Đề cương

Cấu trúc dữ liệu & Giải thuật Cơ bản

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM

Có lý thuyết, bài tập và đề thi.
Bản 6/2025.

MỤC LỤC.

MỤC LỤC. 3

Phần A: GIỚI THIỆU. 6

I. THUẬT TOÁN (ALGORITHM). 6

1. Định nghĩa và tính chất của thuật toán. 6

2. Biểu diễn thuật toán. 6

3. Thời gian thực hiện chương trình. 7

II. CẤU TRÚC DỮ LIỆU (DATA STRUCTURE). 9

1. Giới thiệu bộ nhớ máy tính. 9

2. Biến. 9

3. Nguyên lí truy xuất dữ liệu. 10

4. Con trỏ. 11

5. Tổ chức bộ nhớ của cấu trúc dữ liệu tuyến tính, nguyên lí của vị trí chỉ mục (index). 14

Phần B: CẤU TRÚC DỮ LIỆU TUYẾN TÍNH. 15

I. MẢNG (ARRAY) VÀ VECTOR. 15

1. Giới thiệu. 15

2. Mảng nhiều chiều. 15

3. Mảng động và Vector. 16

4. Chuỗi (xâu). 16

III. NGĂN XẾP (STACK). 17

IV. HÀNG ĐỢI (QUEUE). 18

VI. DANH SÁCH LIÊN KẾT (LINKED LIST). 19

1. Danh sách liên kết đơn (Singly linked list). 19

2. Danh sách liên kết đôi (Doubly linked list). 27

3. Danh sách liên kết vòng (Circular linked list). 27

VII. BẢNG BĂM (HASH TABLE). 28

1. Băm dữ liệu. 28

2. Một số phương pháp giải quyết đụng độ (Collision). 29

3. Một số phương pháp xây dựng hàm băm. 34

4. Cấu trúc map, muiltimap và unorderd_map. 35

Phần B: CẤU TRÚC DỮ LIỆU PHI TUYẾN. 36

I. CẤU TRÚC DỮ LIỆU TẬP HỢP (SET). 36
1. Set, multiset và unordered_set. 36
II. CẤU TRÚC DỮ LIỆU ĐỒ THỊ (GRAPH). 36
1. Giới thiệu. 36
2. Bài toán lưu trữ đồ thị. 37
3. Bài toán duyệt đồ thị. 44
4. Bài toán tìm đường đi ngắn nhất bằng thuật toán Dijkstra. 47
III. CẤU TRÚC DỮ LIỆU CÂY (TREE). 50
1. Cây nhị phân (Binary tree). 52
2. Cây tìm kiếm nhị phân (Binary search tree, BST). 58
3. B-Tree. 64
Phần C: CÁC THUẬT TOÁN TÌM KIẾM CHÍNH XÁC. 70
I. CÁC THUẬT TOÁN CÓ Ý TƯỞNG TÌM KIẾM TUẦN TỰ. 70
1. Thuật toán tìm kiếm tuyến tính (Linear search) và cải tiến, lý giải độ phức tạp tuyến tính. 70
II. CÁC THUẬT TOÁN CÓ Ý TƯỞNG CHIA ĐỂ TRỊ. 72
1. Thuật toán tìm kiếm nhị phân (Binary search), lý giải độ phức tạp logarithm. 72
2. Thuật toán tìm kiếm nội suy (Interbolation search). 78
Phần D: THUẬT TOÁN SẮP XẾP. 80
I. GIỚI THIỆU BÀI TOÁN SẮP XẾP VÀ CÁC THUẬT TOÁN SẮP XẾP. 80
1. Giới thiệu bài toán sắp xếp. 80
2. Stable sort và Online sort. 80
II. CÁC THUẬT TOÁN XỬ LÝ THÔ. 81
1. Bubble sort. 81
2. Selection sort. 82
3. Insertion sort. 83
III. CÁC THUẬT TOÁN CÓ Ý TƯỞNG CHIA ĐỂ TRỊ. 85
1. Merge sort. 85
2. Quick sort. 88
3. Heap sort. 91
V. CÁC THUẬT TOÁN KHÔNG CÓ Ý TƯỞNG SO SÁNH. 93

1. Counting sort. 93
ÔN TẬP KẾT THÚC MÔN. 94
I. THỰC HÀNH. 94
1. Ngăn xếp. 94
2. Hàng đợi. 97
3. Danh sách liên kết. 98
4. Bảng băm. 107
5. Cây nhị phân tìm kiếm. 111
6. Các thuật toán tìm kiếm. 130
7. Các thuật toán sắp xếp. 135
II. LÝ THUYẾT. 142
ĐỀ THI UIT. 146
I. LÝ THUYẾT. 146
II. MÔ PHỎNG THUẬT TOÁN. 148
1. Tìm kiếm. 148
2. Sắp xếp. 149
2. Danh sách liên kết, stack và queue. 152
3. Cây nhị phân, cây nhị phân tìm kiếm. 156
4. Cây B-Tree. 165
5. Bảng băm. 171
6. Đồ thị. 179
III. MÔ HÌNH HÓA BÀI TOÁN VÀ CÀI ĐẶT THUẬT TOÁN. 183

Phần A: GIỚI THIỆU.

[*Đọc thêm*]

I. THUẬT TOÁN (ALGORITHM).

1. Định nghĩa và tính chất của thuật toán.

Lập trình là nghệ thuật cài đặt một hoặc nhiều thuật toán trừu tượng có liên quan với nhau bằng một ngôn ngữ lập trình để tạo ra một chương trình máy tính.

Thuật toán (giải thuật) là tập hợp (dãy) hữu hạn các chỉ thị (hành động) được định nghĩa rõ ràng nhằm giải quyết một bài toán cụ thể nào đó.

- ***Tính chính xác***: Quá trình tính toán hay các thao tác máy tính thực hiện là chính xác.
- ***Tính rõ ràng***: Các câu lệnh minh bạch và được sắp xếp theo thứ tự nhất định.
- ***Tính khách quan***: Được viết bởi nhiều người trên máy tính nhưng kết quả phải như nhau.
- ***Tính phổ dụng***: Có thể áp dụng cho một lớp các bài toán với nhiều đầu vào tương tự nhau.
- ***Tính kết thúc***: Phải hữu hạn bước tính.

2. Biểu diễn thuật toán.

Bài toán:

- Đầu vào: a, b thuộc R.
- Đầu ra: Nghiệm phương trình ax + b = 0.

a.Biểu diễn bằng ngôn ngữ tự nhiên (Natural language).

```
1. Nhập 2 số thực a và b.
2. Nếu a = 0 thì
       2.1. Nếu b = 0 thì
              2.1.1. Phương trình vô số nghiệm
              2.1.2. Kết thúc thuật toán.
       2.2. Ngược lại
              2.2.1. Phương trình vô nghiệm.
              2.2.2. Kết thúc thuật toán.
3. Ngược lại
       3.1. Phương trình có nghiệm.
       3.2. Giá trị của nghiệm đó là x = -b/a
       3.3. Kết thúc thuật toán.
```

b. Biểu diễn bằng lưu đồ (sơ đồ khối) (Flowchart).

c. Biểu diễn bằng mã giả (Pseudo code).

Là biểu diễn dưới hình thức của ngôn ngữ lập trình nhưng có sử dụng từ ngữ của ngôn ngữ tự nhiên.

```
Nếu a = 0 thì {
       Nếu b = 0 thì
```

```
                Xuất "Phương trình vô số nghiệm"
        Nếu không thì:
                Xuất "Phương trình vô nghiệm"
} Nếu không thì
        Xuất "Phương trình có nghiệm x = -b/a"
```

3. Thời gian thực hiện chương trình.

a. Phân tích tiệm cận (Asymptotic analysis).

Phân tích tiệm cận của một thuật toán là khái niệm giúp chúng ta ước tính thời gian chạy của một thuật toán. Sử dụng phân tích tiệm cận, chúng ta có thể đưa ra kết luận tốt nhất về trường hợp tốt nhất, trường hợp trung bình và trường hợp xấu nhất của một thuật toán.

- **Trường hợp tốt nhất**: là thời gian tối thiểu cần thiết để thực hiện chương trình.
- **Trường hợp trung bình**: là thời gian trung bình cần để thực hiện chương trình.
- **Trường hợp xấu nhất**: là thời gian tối đa để thực thi chương trình.

Phân tích tiệm cận là tiệm cận với dữ liệu đầu vào (Input), tức là nếu thuật toán không có Input thì kết luận cuối cùng sẽ là thuật toán sẽ chạy trong một khoảng thời gian cụ thể và không đổi. Ngoài yếu tố Đầu vào, các yếu tố khác được coi là không đổi. Kí hiệu: $f(n)$, n tượng trưng cho dữ liệu đầu vào.

Có 3 cách phân tích tiệm cận (Asymptotic notation), nhưng nổi tiếng nhất là **Big O notation**: ($O\big(f(x)\big)$): Ước tính độ phức tạp thời gian trong trường hợp xấu nhất hoặc khoảng thời gian dài nhất mà một thuật toán yêu cầu (thực hiện từ đầu đến cuối).

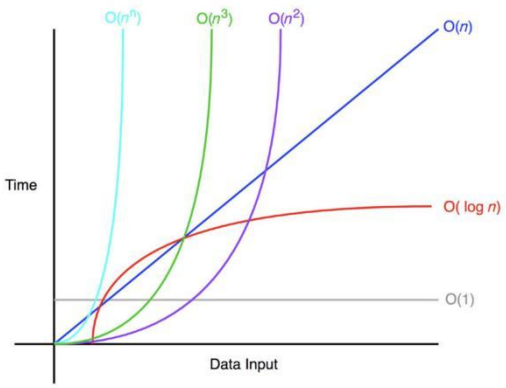
b. Độ phức tạp của thuật toán theo thời gian (Time complexity).

Độ phức tạp theo thời gian thu được bằng cách phân tích tiệm cận.

Ý nghĩa:

- Phỏng đoán tốc độ của thuật toán (nằm trong chương trình), qua đó đưa ra những tối ưu phù hợp nhằm cải tiến sao cho chương trình chạy càng nhanh càng tốt.
- Tương đối thể hiện số lượng phép tính của thuật toán so với khối lượng của dữ liệu vào. Như vậy, độ phức tạp của thuật toán phụ thuộc vào khối lượng của dữ liệu vào.
- Độ phức tạp càng nhỏ (nhỏ nhất là 1) thì thuật toán càng nhanh.

Kí hiệu: $O\big(f(n)\big)$, $f(n)$ là hàm theo n thể hiện khối lượng của dữ liệu vào. Nếu $f(n) = 1$ thì $O\big(f(n)\big) = O(1)$ là độ phức tạp hằng số.



Cách xác định độ phức tạp: Dựa trên số lần lặp nhiều nhất ở mỗi bước.
VD:

Thao tác	Nhận xét	Độ phức tạp	Điều kiện tối đa của n để không TLE
a = b; a = min(a, b);	Lặp 1 lần.	O(1)	Với mọi n.
Lặp từ 1 đến n.	Lặp n lần.	O(n)	$n \leq 10^6$.
Lặp từ 1 đến n: Lặp từ 1 đến m.	Lặp n.m lần.	O(nm) hoặc $O(n^2)$	$n \leq 10^3$.
Lặp n/2 lần: Lặp n/4 lần: …	Lặp $\log_2 n$ lần.	$O(\log_2 n)$	$n \leq 10^{12}$.

b. Hệ số không đổi (Constant factor).

Khái niệm:

- Hệ số không đổi k là hệ số của $f(n)$, trở thành $O(k.f(n))$, thể hiện số lượng phép tính của thuật toán.
- Như vậy, k không phụ thuộc vào khối lượng dữ liệu vào mà phụ thuộc vào số lượng phép tính.

VD1: Có dòng code:

```
a = b; ++ a;
```

Do không lặp nên độ phức tạp là O(1). Tuy nhiên, lại có đến 2 phép tính chứ không phải 1, nên k = 2, tức O(2).

VD2: Trong vòng lặp n lần có dòng code là:

```
a = b; ++ a;
```

Do lặp n lần nên độ phức tạp là O(n). Tuy nhiên, lại có đến 2 phép tính ở mỗi lần lặp, nên k = 2, tức O(2n).

II. CẤU TRÚC DỮ LIỆU (DATA STRUCTURE).

Cấu trúc dữ liệu là cách để lưu trữ dữ liệu.

Một số tiêu chí phân loại cấu trúc dữ liệu:

Cách lưu trữ dữ liệu	Tuyến tính (Linear)	Phi tuyến (Non-linear)
Khả năng lưu trữ dữ liệu	Tĩnh (Static)	Động (Dynamic)
Đặc điểm truy xuất dữ liệu	Ngẫu nhiên (Random)	Tuần tự (Sequential)
Đặc điểm lưu trữ trên bộ nhớ	Bảng băm (Hashtable)	Phi bảng băm (Unhashtable)

1. Giới thiệu bộ nhớ máy tính.

Bộ nhớ máy tính là một dãy các ô nhớ nằm liên tiếp nhau, mỗi một ô nhớ sẽ lưu 1 byte dữ liệu và dữ liệu của bạn sẽ nằm trong các ô nhớ này.

Để truy xuất được vào một ô nhớ bạn cần biết được vị trí (địa chỉ) của ô nhớ đó.

2. Biến.

a. Giới thiệu về biến.

Biến là một ô nhớ hoặc một vùng nhớ được hệ điều hành cấp phát cho chương trình.

Một biến sẽ gồm 2 phần là ***địa chỉ*** của biến (địa chỉ của ô nhớ chứa biến) và ***giá trị*** của biến (giá trị của ô nhớ chứa biến).

Theo mặc định thì địa chỉ được biểu diễn theo hệ cơ số 16. Để chuyển sang hệ cơ số 10, ép kiểu của nó sang số nguyên.

Địa chỉ của biến chính là địa chỉ của ô nhớ chứa biến.

- Cú pháp lấy ra địa chỉ của một biến: `&variable`.
- Cú pháp thay đổi giá trị trong ô nhớ: `*(&variable) = data`.

b. Phân loại biến.

Có 2 loại biến:

Biến tĩnh (Static variable)	**Biến động** (Dynamic variable)
Được khai báo tường minh, có tên gọi.	Không được khai báo tường minh, không có tên gọi. Quản lý thông qua biến con trỏ (pointer).
Tồn tại trong phạm vi khai báo.	Xin khi cần, giải phóng khi sử dụng xong.
Được cấp phát trong stack.	Được cấp phát trong heap.
Kích thước không đổi. → Không sử dụng bộ nhớ hiệu quả.	Linh động về kích thước. → Sử dụng bộ nhớ hiệu quả.

c. Vùng nhớ (Segment) và cấp phát bộ nhớ (Memory allocation).

- **Text segment / Code segment**: Lưu trữ các đoạn mã đã được biên dịch để chương trình thực thi. Đây là vùng nhớ read-only để ngăn chặn việc các đoạn mã bị thay đổi khi chương trình đang chạy. Thông thường text segment là sharable memory, do đó khi bạn mở 1 chương trình nhiều lần thì tất cả cách chương trình này đều sẽ chỉ dùng chung 1 vùng nhớ text segment.
- **Initialized Data Segment**: Lưu trữ các ***tĩnh biến toàn cục***. Vùng nhớ còn được chia thành 2 loại là ***read-only area*** (lưu hằng số) và ***read-write area*** (lưu biến số).
- **Uninitialized Data Segment (BSS)**: Lưu trữ các ***tĩnh biến toàn cục***, nhưng ***chưa được khởi tạo giá trị***.
- **Heap segment**: Dùng cho ***cấp phát động***, vùng nhớ này có đặc điểm là không được quản lý bởi CPU, đó cũng là lý do bạn phải tự quản lý vùng nhớ được cấp phát động để tránh ***memory leak (rò rỉ bộ nhớ)***. Đây là vùng nhớ ***lớn nhất***, nhưng tốc độ truy xuất ***chậm***.
- **Free store**: Tương tự với heap, sự khác biệt không đáng kể.
- **Stack segment**: Lưu trữ các biến cục bộ, các tham số của hàm, địa chỉ của hàm (function address), đó cũng là lý do khi gọi hàm nhiều chương trình sẽ báo lỗi tràn bộ nhớ stack. Vùng nhớ này có tốc độ truy xuất ***nhanh*** nhưng lại có kích thước ***nhỏ***.

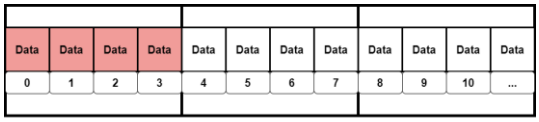
Một biến cục bộ được khai báo sẽ nằm trong vùng nhớ stack. Stack nhỏ nên dễ tràn dữ liệu nên có thể cân nhắc khai báo biến ở free store bằng từ khóa new, phương pháp này gọi là ***cấp phát động (Dynamic memory allocation)***. Nhưng bù lại, bạn phải giải phóng bộ nhớ bằng từ khóa delete hoặc return do vấn đề rò rỉ bộ nhớ (memory leak).

- Khai báo bộ nhớ ở free store: `data-type variable = new data-type;`.
- Giải phóng bộ nhớ bằng delete: `delete(variable);`.
- Giải phóng bộ nhớ bằng return: `return variable;`.

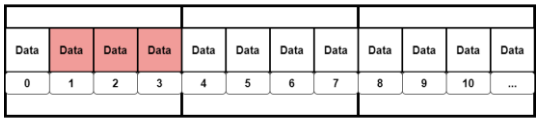
3. Nguyên lí truy xuất dữ liệu.

a. Vấn đề truy xuất dữ liệu.

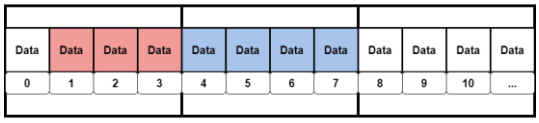
Bộ nhớ máy tính là một dãy các ô nhớ nằm liên tiếp nhau, mỗi một ô nhớ sẽ lưu 1 byte dữ liệu và dữ liệu của bạn sẽ nằm trong các ô nhớ này.

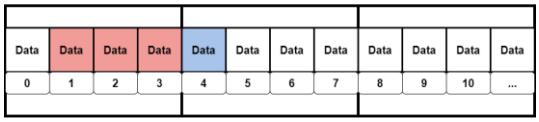


Để truy xuất được vào một ô nhớ bạn cần biết được vị trí (địa chỉ) của ô nhớ đó.



Ví dụ ta muốn lấy giá trị của vùng nhớ có địa chỉ từ 1 tới 4 thì chỉ cần lấy giá trị của từng ô nhớ và tổng hợp lại. Tuy nhiên, việc đọc và ghi với từng ô nhớ như vậy sẽ tốn nhiều thời gian nên CPU sẽ truy xuất tới bộ nhớ theo từng block, mỗi block có thể có kích thước 2, 4, 8, 16, 32, 64 bytes tùy vào CPU. Kích thước này được gọi là ***memory access granularity***.





Các block này được xếp liên tiếp nhau, nên CPU chỉ có thể đọc được dữ liệu từ các vùng nhớ có địa chỉ chia hết cho kích thước của block.

VD. Giả sử các block có kích thước là 4 bytes:



b. Nguyên lí truy xuất dữ liệu.

Để lấy dữ liệu lưu trong các byte từ 1 đến 4, máy tính làm như sau:

- Đọc block 1 (đỏ).
- Loại bỏ các ô không mong muốn.
- Đọc block 2 (xanh).
- Loại bỏ các ô không mong muốn.
- Ghép các ô đã đọc.

Có thể thấy đây là một công việc khá phức tạp, nếu các ô nhớ cần lấy có địa chỉ từ 0 tới 3 thì công việc của CPU sẽ rất đơn giản là chỉ cần đọc một block từ địa chỉ 0.

Do đó, để tránh việc đọc dữ liệu từ các vùng nhớ có địa chỉ không chia hết cho kích thước của block thì trình biên dịch C++ sẽ tự động ***chèn thêm một số byte rỗng*** khi biên dịch để tối ưu tốc độ thực thi.

c. Truy xuất phần tử của cấu trúc dữ liệu.

Một cấu trúc dữ liệu có thể truy xuất ngẫu nhiên và truy xuất tuần tự, hoặc chỉ truy xuất tuần tự.

- ***Truy xuất ngẫu nhiên (Random access)*** (Truy xuất trực tiếp (Direct access)): Là khả năng truy xuất đến một phần tử bất kì thông qua chỉ mục index. Cú pháp: `data-structor[index]`.
- ***Truy xuất tuần tự (Sequential access)***: Để truy cập đến một phần tử nào đó, cần phải duyệt từ phần tử đầu tiên đến phần tử muốn truy xuất.

4. Con trỏ.

a. Giới thiệu con trỏ.

Con trỏ là biến.

Giá trị của con trỏ là địa chỉ của một biến khác. Biến khác ở đây là biến được trỏ.

- Cú pháp khai báo:
  ```
  data-type *pointer
  ```
- Cú pháp trỏ khi biết tên biến:
  ```
  pointer = &pointed-variable
  ```
- Cú pháp trỏ khi biết địa chỉ:
  ```
  pointer = reinterpret_cast<data-type*>(address)
  ```
  Trong đó, address là một dãy số nguyên.
- Cú pháp lấy giá trị tại địa chỉ được trỏ:
  ```
  *pointer
  ```
- Cú pháp thay đổi giá trị của biến được trỏ thông qua con trỏ:
  ```
  *pointer = data
  ```

- Cú pháp giải phóng bộ nhớ con trỏ đơn:
  delete pointer;

Đối với mảng 2 chiều:

Chú ý:

- Không thể trỏ đến biến của một chương trình khác.
- Giả sử có một struct chứa nhiều trỏ. Khi khai báo một biến thuộc struct đó thì các trỏ thành phần vẫn chưa được cấp phát bộ nhớ, muốn truy cập đến các trỏ đó thì phải cấp phát bộ nhớ trước.

VD. Có struct:

```
struct douList{
    node *head;
    node *tail;
};
```

Giả sử có một biến x thuộc kiểu douList. Khi khai báo x, các con trỏ head và tail vẫn chưa được cấp bộ nhớ. Nếu muốn truy cập vào head và tail thì phải cấp bộ nhớ cho nó trước.

b. NULL và nullptr.

- NULL là một số nguyên có giá trị là 0.
- nullptr là một địa chỉ tượng trưng cho một địa chỉ nào đó không có giá trị sử dụng.
- Nếu một con trỏ được gán giá trị NULL, nó được coi như nullptr.

VD. 2 câu lệnh sau là như nhau:

```
int *pointer1 = NULL;
int *pointer2 = nullptr;
```

c. Kích thước của con trỏ (Kiểu dữ liệu của con trỏ).

- Con trỏ trên platform ***x64 (64 bits)*** có kích thước là ***8 bytes***.
- Con trỏ trên platform ***x86 (32 bits)*** có kích thước là ***4 bytes***.
- Kiểu dữ liệu của con trỏ quyết định đến khả năng trỏ của con trỏ. Con trỏ có kiểu dữ liệu có kích thước x bytes sẽ trỏ tới được x bytes đầu tiên trong ô nhớ của biến được trỏ. Giá trị tại ô nhớ được trỏ cũng chính là giá trị trong x bytes đầu tiên của biến được trỏ.

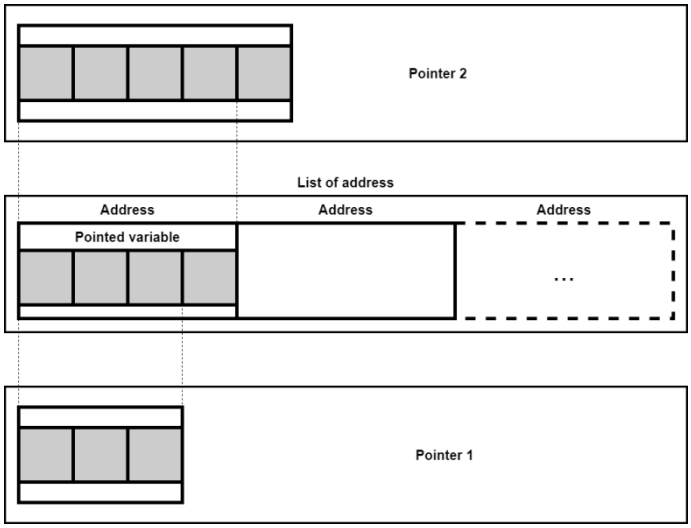
Chú ý: Trên tinh thần, kiểu dữ liệu của trỏ phải cùng với kiểu dữ liệu của biến được trỏ. Cú pháp ép kiểu của biến được trỏ sang kiểu của trỏ: **(pointer-data-type*)&pointed-variable**.

Lý do kết quả có thể là 4 hoặc 8 là do giá trị của một con trỏ luôn là 1 địa chỉ mà ứng dụng x32 sẽ có 232 địa chỉ (232 ô nhớ - tương đương với kiểu dữ liệu có kích thước 4 byte), ứng dụng x64 sẽ có 264 địa chỉ (264 ô nhớ - tương đương với kiểu dữ liệu có kích thước 8 byte) nên con trỏ cần phải được cấp phát đủ bộ nhớ để lưu trữ được các địa chỉ này. Vì thế nên kích thước của con trỏ sẽ phụ thuộc vào platform mà bạn build.

VD.

Trong hình bên, tôi có 1 biến có kích thước 4 bytes.
Kiểu dữ liệu của pointer 1 chỉ có 3 bytes nên chỉ trỏ được 3 bytes đầu tiên của biến.

Kiểu dữ liệu của pointer 2 lớn hơn biến 1 byte nên nó vừa trỏ tới 4 bytes của biến, vừa trỏ tới 1 byte liền kề biến đó.



d. Cấp của con trỏ.

- Con trỏ trỏ tới 1 biến khác (không phải con trỏ) là con trỏ cấp 1 (1 level pointer).
  Cú pháp khai báo: `data-type* pointer`.
- Con trỏ trỏ tới con trỏ cấp 1 là con trỏ cấp 2 (2 level pointer).
  Cú pháp khai báo: `data-type** pointer`.
- Con trỏ trỏ tới con trỏ cấp 2 là con trỏ cấp 3 (3 level pointer).
  Cú pháp khai báo: `data-type*** pointer`.
- …

Bạn có thể khai báo tối đa con trỏ cấp 12.

e. Các thuật toán có sẵn được cài đặt bằng con trỏ.

Library	TC	Cú pháp	Ý nghĩa
`algorithm`	O(nlogn)	`sort(a, b);`	Sắp xếp từ vị trí mà a trỏ đến vị trí liền trướ b trỏ theo thứ tự tăng dần.
`algorithm`	O(nlogn)	`sort(a, b, comp);`	Sắp xếp từ vị trí mà a trỏ đến vị trí liền trướ b trỏ theo thứ tự giảm dần.
`algorithm`	O(nlogn)	`sort(a, b, comparison);`	Sắp xếp từ vị trí mà a trỏ đến vị trí liền trướ b trỏ theo thứ tự tùy theo hàm comparison.
`algorithm`	O(n)	`max_element(a, b)`	Trả về giá trị của phần tử lớn nhất từ vị trí m a trỏ đến vị trí liền trước b trỏ.
`algorithm`	O(n)	`min_element(a, b)`	Trả về giá trị của phần tử nhỏ nhất từ vị trí m a trỏ đến vị trí liền trước b trỏ.
`algorithm`	O(1)	`max(a, b)`	Trả về giá trị lớn nhất giữa a và b.
`algorithm`	O(1)	`min(a, b)`	Trả về giá trị nhỏ nhất giữa a và b.
`algorithm`	O(1)	`swap(a, b)`	Hoán đổi giá trị của a và b.

Hàm comparison có cú pháp khai báo như sau: `bool comparison (data-type a, data-type b) {content}`. Trong đó:

- ***data-type***: Kiểu dữ liệu của các phần tử trong danh sách.
- ***a***, ***b***: 2 tham số tượng trưng cho 2 phần tử bất kì trong danh sách.
- Nếu hàm này trả về ***true***, a sẽ được xếp trước b, và ngược lại.

Bản chất của từ khóa `comp` chính là hàm comparison với nội dung: `return a < b`.

5. Tổ chức bộ nhớ của cấu trúc dữ liệu tuyến tính, nguyên lí của vị trí chỉ mục (index).

a. Địa chỉ của các phần tử trong cấu trúc dữ liệu tuyến tính.

Cấu trúc dữ liệu tuyến tính là một tập các phần tử được nằm trên các ô nhớ liên tiếp. Do đó địa chỉ của các phần tử trong danh sách cũng liên tiếp nhau và cách nhau i đơn vị, với i là số byte của kiểu dữ liệu biểu diễn địa chỉ.

Tên của mảng thật chất là con trỏ trỏ đến địa chỉ phần tử đầu tiên của mảng.

Con trỏ trỏ tới mảng có thể linh động thay đổi kích thước:

- Cấp phát bộ nhớ:

```
*pointer = new data_type[length];
```

- Giải phóng bộ nhớ:

```
delete pointer[];
```

Công thức tính vị trị của phần tử thứ index:

first-address + sizeof(data-type) * index.

Với first-address là phần tử đầu tiên tương ứng với index = 0.

b. Liên hệ dãy địa chỉ với con trỏ.

Khi cộng con trỏ với một giá trị index, bạn thu được con trỏ trỏ tới địa chỉ cách địa chỉ ban đầu:

sizeof(data-type) * index.

Nói cách khác:

first-address + index = first-address + sizeof(data-type) * index.

Phần B: CẤU TRÚC DỮ LIỆU TUYẾN TÍNH.

I. MẢNG (ARRAY) VÀ VECTOR.

[*Đọc thêm*]

1. Giới thiệu.

Mảng là cấu trúc dữ liệu tuyến tính tĩnh (mặc định), có thể được cấp phát động, truy xuất ngẫu nhiên.

Cú pháp	Mảng tĩnh	Mảng động
Khai báo biến	`data-type array[size];`	`data-type* array = new data-type[size]`
Khai báo tham số	`data-type array[]`	`data-type array[]`
Giải phóng bộ nhớ		`delete[] array;`
Ghi chú	Có thể không khai báo size nhưng phải khởi tạo giá trị: + = `{data}` : Mọi phần tử đều = data. + = `{data1, data2}`. + = `{{data11,data12},{data21,data22}}`.	Có thể không khai báo size.

2. Mảng nhiều chiều.

Mảng có thể mở rộng thành nhiều chiều, mỗi phần tử của mảng là một mảng.

a. Mảng tĩnh 2 chiều.

Mảng 2 chiều cũng là một dãy phần tử nằm trên các ô nhớ liên tiếp, phần tử đầu hàng dưới nối tiếp phần tử cuối hàng trên.

Phần tử đầu tiên có vị trí 0, phần tử cuối cùng có vị trí n * m, với n và m là số hàng và cột của mảng.

b. Mảng động 2 chiều.

Mảng 2 chiều được cấp phát động thực ra là mảng một chiều các con trỏ, và các con trỏ này trỏ tới phần tử đầu tiên của mảng được cấp phát động.

Do đó cấp phát động cho mảng 2 chiều thực ra là cấp phát động cho nhiều mảng 1 chiều.

Cụ thể hơn là bạn sẽ dùng 1 con trỏ cấp 2 (con trỏ trỏ tới con trỏ), cấp phát cho nó n con trỏ cấp 1.

- Cú pháp khai báo con trỏ cấp 2: `data-type** arr`.
- Cú pháp cấp phát động:

```
// Cấp phát vùng nhớ động cho mảng 1 chiều các con trỏ kiểu int*
// Có thể hiểu arr là mảng n phần tử, mỗi phần tử là 1 con trỏ kiểu int*
arr =  new  data-type* [n];
for ( int i =  0 ; i < n; i ++ ) {
    arr[i] =  new  data-type [m]; //Cấp phát cho n mảng 1 chiều
```

```
}
```

- Cú pháp giải phóng bộ nhớ:

```
for ( int i = 0 ; i < n; i ++ ) {
    delete []arr[i];  // Giải phóng bộ nhớ con các mảng theo chiều
}
delete []arr;         // Giải phóng bộ nhớ cho mảng các con trỏ
```

Các phần tử không nằm trên các ô nhớ liên tiếp.

3. Mảng động và Vector.

Vector là một cấu trúc dữ liệu tuyến tính động của C++, truy xuất ngẫu nhiên, unhastable, hoạt động với phương thức giống mảng tĩnh nhưng có tính chất giống mảng động.

Library	TC	Cú pháp	Ý nghĩa
vector	O(1)	`vector<data-type> vector;`	Khai báo vector.
vector	O(1)	`vector.push_back(data);`	Đưa data vào vector.
vector	O(1)	`vector.pop_back(data);`	Lấy data khỏi vector.
vector	O(1)	`vector.insert(pointer, data);`	Sao chép data vào vị trí mà pointer trỏ đến.
vector	O(1)	`vector.emplace(pointer, data);`	Chèn data vào vị trí mà pointer trỏ đến.
vector	O(n)	`vector.clear(pointer);`	Xóa phần tử mà pointer trỏ đến.
vector	O(n)	`vector.clear();`	Xóa hết các phần tử của string.
vector	O(1)	`vector.top()`	Trả về phần tử vừa mới đưa vào vector.
vector	O(1)	`vector.empty()`	Trả về true nếu vector rỗng.
vector	O(1)	`vector.size()`	Trả về số lượng phần tử của vector.

4. Chuỗi (xâu).

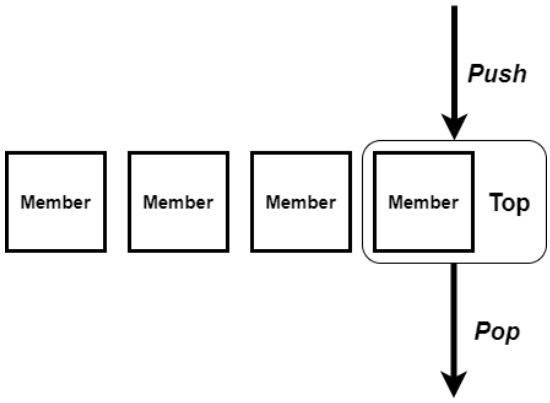
Chuỗi là một danh sách mà mỗi phần tử của chuỗi là một kí tự (char).

Library	TC	Cú pháp	Ý nghĩa
string iostream	O(1)	`getline(cin, string, delimiter);`	Đọc string cho đến khi gặp kí tự delimiter thì dừng. Delimiter có thể không khai báo, mặc định là `\n`.
iostream	O(1)	`fflush(stdin);`	Xóa bộ nhớ đệm, được đặt sau hoặc trước getline để trách đọc vào những kí tự không cần thiết.
string	O(1)	`string.push_back(data);`	Thêm data vào cuối tring.
string	O(1)	`string.pop_back();`	Xóa 1 phần tử ở cuối string.
string	O(n)	`string.clear(pointer);`	Xóa phần tử mà pointer trỏ đến.
string	O(n)	`string.clear();`	Xóa hết các phần tử của string.
string	O(1)	`string.empty()`	Trả về true nếu string rỗng.
string	O(1)	`string.size()` `string.length()`	Trả về số lượng phần tử của string.
string	O(n)	`stoi(string)`	Trả về dạng int của string.
string	O(n)	`stol(string)`	Trả về dạng long của string.

`string`	O(n)	`stoll(string)`	Trả về dạng long long của string.
`string`	O(n)	`stof(string)`	Trả về dạng float của string.
`string`	O(n)	`stod(string)`	Trả về dạng double của string.
`string`	O(n)	`to_string(data)`	Trả về dạng string của data.

III. NGĂN XẾP (STACK).

Stack là một cấu trúc dữ liệu tuyến tính động, chỉ truy xuất ngẫu nhiên phần tử cuối cùng, hashtable, hoạt động theo nguyên tắc LIFO (last in first out) – phần tử nào vào sau thì phần tử đó ra trước ⇒ Các phần tử vào ra từ một đầu của stack, tạm coi như ra vào từ cuối stack.



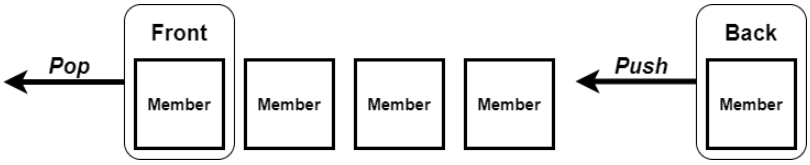
Ứng dụng: Stack là một kho ***lưu trữ tạm thời*** và được xét ***ngược lại với thứ tự ban đầu***. VD:

- Ta đang duyệt mảng A bằng biến i. Ở vị trí i, ta muốn duyệt ngược lại (i-1, i-2, i-3,...).
- Thay vì tốn thêm một vòng for duyệt ngược nữa, ta chỉ cần lần lượt đưa A[i] vào stack. Khi nào muốn duyệt ngược, ta lấy A[i] ra khỏi stack.
- Tuy độ phức tạp không đổi so với for ngược, nhưng thực tế, việc khởi tạo một vòng for có thể tốn nhiều thời gian hơn việc lấy 1 đỉnh ra khỏi stack.

Library	TC	Cú pháp	Ý nghĩa
`stack`	O(1)	`stack<data-type> stack;`	Khai báo stack.
`stack`	O(1)	`stack.push(data);`	Đưa data vào stack.
`stack`	O(1)	`stack.pop(data);`	Lấy data khỏi stack.
`stack`	O(1)	`stack.top()`	Trả về phần tử vừa mới đưa vào stack.
`stack`	O(1)	`stack.empty()`	Trả về true nếu stack rỗng.
`stack`	O(1)	`stack.size()`	Trả về số lượng phần tử của stack.

IV. HÀNG ĐỢI (QUEUE).

Queue là một cấu trúc dữ liệu tuyến tính động, chỉ truy xuất ngẫu nhiên phần tử đầu tiên và phần tử cuối cùng, hashtable, hoạt động theo nguyên tắc FIFO (first in first out) – phần tử nào vào trước thì phần tử đó ra sau ⇒ Các phần tử vào ra từ hai đầu của queue.



Ứng dụng: Queue là một dãy phần tử ***luân phiên đầu - cuối***, phần tử lớn được tạo thành từ các phần tử con.

Library	TC	Cú pháp	Ý nghĩa
queue	O(1)	`queue<data-type> queue;`	Khai báo queue.
queue	O(1)	`queue.push(data);`	Đưa data vào queue.
queue	O(1)	`queue.pop(data);`	Lấy data ra khỏi queue.
queue	O(1)	`queue.front()`	Trả về phần tử đưa vào queue đầu tiên.
queue	O(1)	`queue.back()`	Trả về phần tử vừa mới đưa vào queue.
queue	O(1)	`queue.empty()`	Trả về true nếu danh sách rỗng.
queue	O(1)	`queue.size()`	Trả về số lượng phần tử của danh sách.

VI. DANH SÁCH LIÊN KẾT (LINKED LIST).

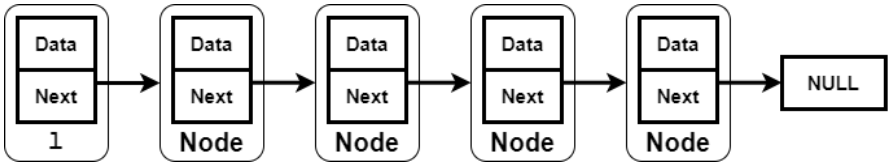
Danh sách liên kết đơn là cấu trúc dữ liệu tuyến tính động, truy xuất tuần tự, hashtable. Nhưng các ô nhớ của danh sách là ngẫu nhiên.

Danh sách liên kết có 3 loại:

1. Danh sách liên kết đơn (Singly linked list).

Cấu trúc: Mỗi phần tử của danh sách là 1 node. Node bao gồm:

- Data: Giá trị của phần tử.
- Next: Con trỏ trỏ đến node liền sau. Node có next trỏ đến NULL là node cuối cùng.



Quy tắc:

- Trên thực tế, ta chỉ thao tác với địa chỉ của node, tức là ***con trỏ***. Nên phía trước tên node trong khai báo hoặc tham số, đều phải có toán tử *****.
- Các hàm thao tác với danh sách liên kết nên là một con trỏ, tham số là l và giá trị trả về ứng với l.

- Gọi giá trị của node p: **`p->data`** hoặc **`(*p).data`**.
- Dịch chuyển p đến node liền sau p: **`p = p->next`** hoặc liền trước p: **`p = p->pre`**.
- Toán tử **`->`** cũng có thể trỏ tuần hoàn **`->next->next->`** … .
- Chỉ có thể truy xuất theo chiều xuôi qua toán tử **`->next`** hoặc **`->pre`**.
- Do chỉ có thể truy xuất theo chiều xuôi (giả sử là chiều sang phải) nên phải cân nhắc điều chỉnh thứ tự xử lý các liên kết giữa các node, trong bài toán cần phải thêm hoặc bớt node.

a. Tạo và hủy danh sách.

- Khai báo node.

```
struct node{
    int data;
    node *next;
};
```

- Để thuận tiện cho việc gắn thêm phần tử vào đầu và cuối danh sách, ta nên lưu trữ kiểu dữ liệu danh sách liên kết để lưu luôn đầu và cuối danh sách:

```
struct list {
    node* head; // Trỏ đến đầu danh sách
    node* tail; // Trỏ đến cuối danh sách
};
```

- Khởi tạo list:

```
void CreateEmptyList(list &l) {
    l.head = NULL;
    l.tail = NULL;
}
```

- Khai báo node mới dựa trên dữ liệu đã có.

```
node *CreateNode(int data){
    node *p = new node; // Tạo node mới.
    p->next = NULL;     // Cho node này trỏ đến NULL.
    p->data = data;     // Cho giá trị cho node.
    return p;
}
```

Sau khi kết thúc, vùng nhớ của con trỏ p được giải phóng. Thứ hàm trả về là ***vùng nhớ được cấp phát động***. Đây là 2 loại vùng nhớ khác nhau.

- Hủy danh sách.

```
void RemoveList(list &L) {
    list * p;
    while (L.head != NULL) {
        p = L.head;
        L.head = p->next; // Thu gọn danh sách theo đầu head
        delete p;
    }
    L.tail = NULL;
}
```

b. Duyệt tuyến tính.

- Duyệt toàn bộ danh sách bằng từ khóa NULL / nullptr.

```
node *p = L.head;

while (p != NULL){
    p = p->next;
}
```

- Duyệt đến vị trí k.

```
node *p = L.head;

for (int i = 0; i < k; i++){
    p = p->next;
}
```

c. Chèn thêm node.

Giả sử ta cần chèn node p tại vị trí của node q.
Phương pháp:

1. Tạo node p (nếu chưa có node p).
2. Nối node p vào node sau q (p->next = q->next).
3. Nối node q vào p (q->next = p).
4. Cập nhật head và tail (nếu cần).

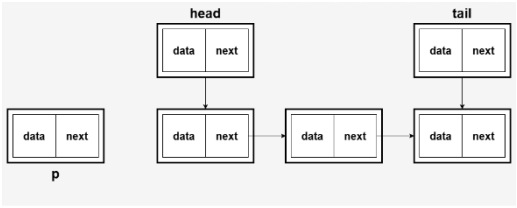
- Thêm một node mới và nối vào đầu danh sách.

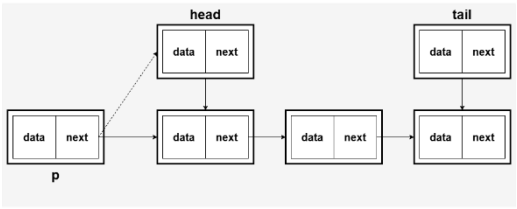
  Trường hợp tổng quát:

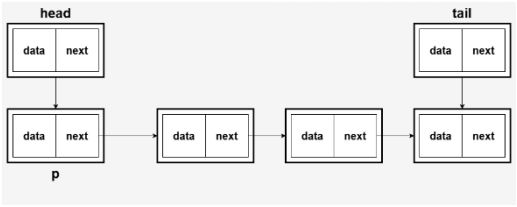
```
void AddHead(list &L, node* p) {
    // Nếu list đang rỗng thì head và tail cùng trỏ đến một vị trí
    if (L.head == NULL) {
        L.head = p;
        L.tail = L.head;
    }
    // Nếu list không rỗng thì nối p vào trước head, sau đó gán head là p
    else {
        p->next = L.head;
        L.head = p;
    }
}
```

  Trường hợp cụ thể:

```
void AddHead(list &L, int x) {
    node* p = CreateNode(x); // Tạo node mới có giá trị là x
    if (L.head == NULL) {
        L.head = p;
        L.tail = L.head;
    } else {
        p->next = L.head;
        L.head = p;
    }
}
```



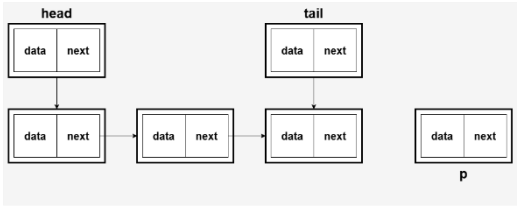


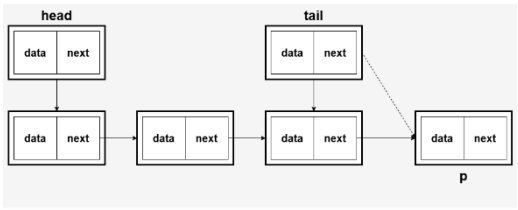


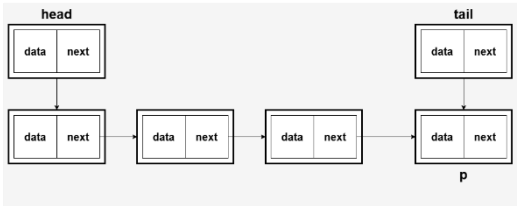
- Thêm một node mới và nối vào cuối danh sách:

Trường hợp tổng quát:

```
void AddTail(list &L, node* p) {
    // Nếu list đang rỗng thì head và tail cùng trỏ đến một vị trí
    if (L.pHead == NULL) {
        L.head = p;
        L.tail = L.head;
    }
    // Nếu list không rỗng thì nối p vào sau tail, sau đó gán tail là p
    else {
        L.tail->next = p;
        L.tail = p;
    }
}
```

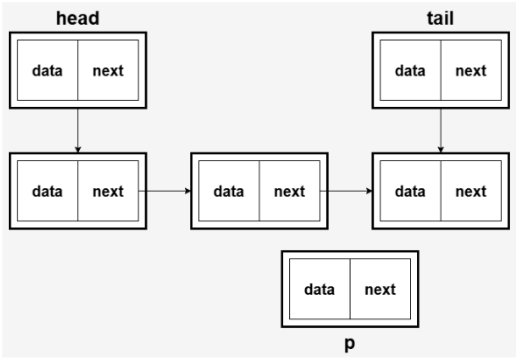

head
tail
data
next
p

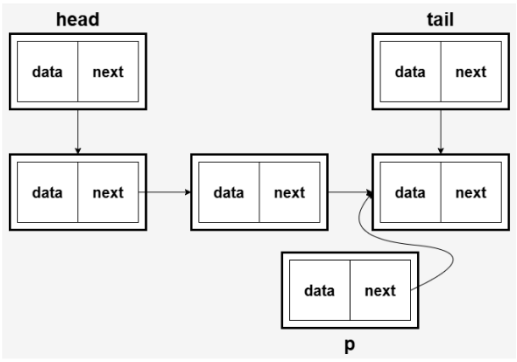

head
tail
data
next
p

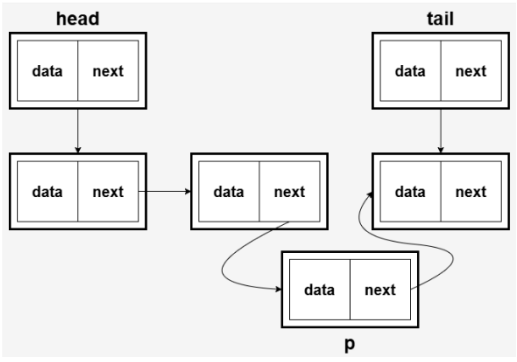

head
tail
data
next
p

- Chèn node vào liền sau một node khác:

```
void AddAfterQ(list &L, node* p, node* q) {
    // Nếu đã tồn tại Q
    if (Q != NULL) {
        p->next = q->next; // Nối p với phần tử sau q
        q->next = p;       // Nối q vào p
        if (L.tail == q)   // Trong trường hợp tail là q là đổi lại tail là p
            L.tail = p;
    }
    // Nếu chưa tồn tại q
    else
        AddHead(L, p);
}
```







- Chèn 1 node mới ở vị trí k.

```
void *AddAt(list &L, int k, int x){
    // Di chuyển đến node liền trước node k (k - 1).
    node *p = L.head;
    for (int i = 0; i < k-1; i++){
        p = p->next;
    }

    node *temp = new node; // Tạo node mới
    temp->data = x;
    temp->next = p->next;  // Nối temp với phần tử sau p
    p->next = temp;        // Nối p đến temp

    if (L.tail == p)       // Trong trường hợp tail là p là đổi lại tail là temp
        L.tail = temp;
}
```

d. Xóa node.

Giả sử ta cần xóa node p nằm sau node q.
Phương pháp:

1. Cô lập p bằng các nối q vào node sau p (q->next = p->next = q->next->next).
2. Giải phóng bộ nhớ p.
3. Cập nhật head và tail (nếu cần).

- Xóa node đầu tiên khỏi danh sách.

```
bool RemoveHead(list &L) {
    node *p;
    if (L.head != NULL) {
        p = L.head;            // Truyền địa chỉ head cho p
        L.head = L.head->next; // Gán node sau head là head
        if (L.head == NULL)    // Trường hợp list có 1 node
            L.tail = NULL;
        delete p;              // Xóa p (head cũ)
        return true;
    }
    return false;
}
```

- Xóa node vào liền sau một node khác:

```
bool RemoveAfterQ(list &L, node *Q) {
    node *p;
    if (Q != NULL) {
        p = Q->next;
        if (p != NULL) {
            if (p == L.tail)   // Nếu p là tail thì cập nhật lại tail là Q
                L.tail = Q;
            Q->next = p->next; // Nối Q đến node sau p
            delete p;
        }
        return true;
    }
    return false;
```

```
}
```

- Xóa node ở vị trí k.

```
void *deleteAt(list *L, int k){
    // Di chuyển đến node liền trước node k(k - 1)
    node *p = L.head;
    for (int i = 0; i < k-1; i++){
            p = p->next;
    }
    node *temp = p->next;
    if (temp != NULL) {
        p->next = p->next->next; // Nối node sau p đến node sau của node
sau p
    } else
        L.tail = p;

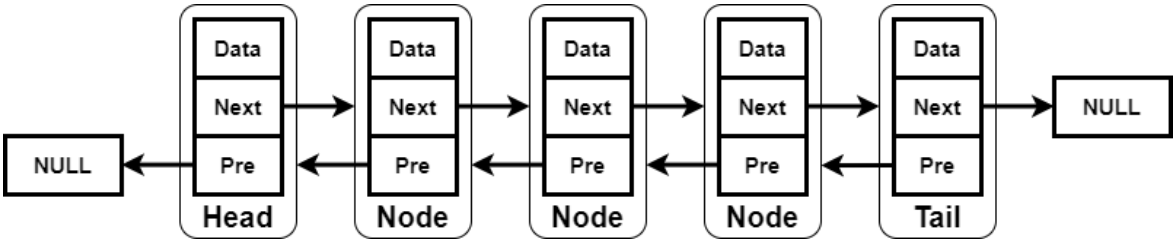
    delete(temp);
}
```

2. Danh sách liên kết đôi (Doubly linked list).

[*Đọc thêm*]

Cấu trúc: Mỗi phần tử của danh sách là 1 node. Node bao gồm:

- Data: Giá trị của phần tử.
- Next: Con trỏ trỏ đến node liền sau. Node có next trỏ đến NULL là node cuối cùng.
- Pre: Con trỏ trỏ đến node liền trước. Node có pre trỏ đến NULL là node đầu tiên.



3. Danh sách liên kết vòng (Circular linked list).

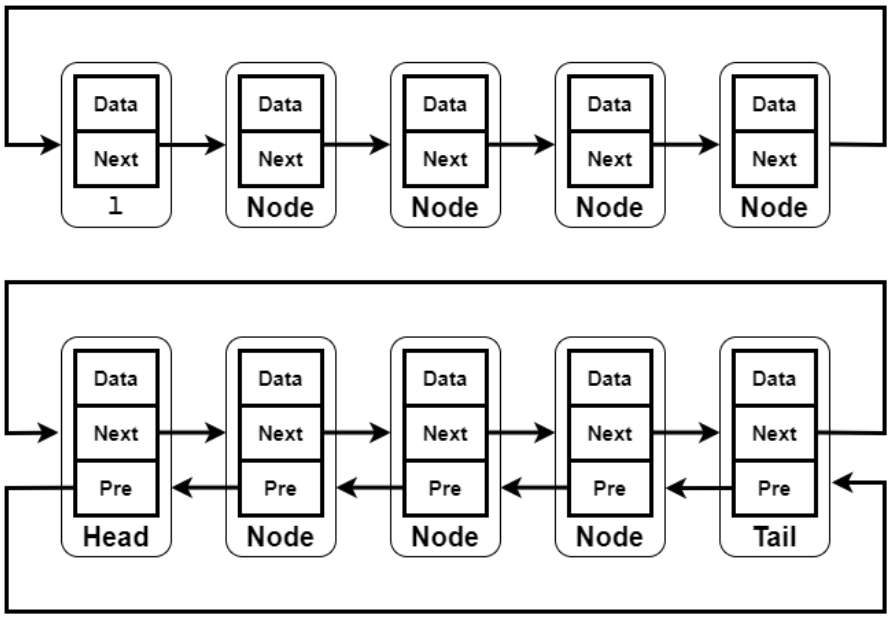
[*Đọc thêm*]

Danh sách liên kết vòng là danh sách liên kết mà node cuối trỏ tới node đầu.

Danh sách liên kết vòng có thể là danh sách đơn (1 vòng) hoặc kép (2 vòng).

Chú ý:

- Phần tử của danh sách liên kết vòng có 1 phần tử là node tự trỏ đến chính nó:
  - `p->next = temp;`.
  - `p->pre = temp;`.
- Cú pháp dịch chuyển con trỏ p đến cuối node: `while (p->next != l) {p = p->next}`.



VII. BẢNG BĂM (HASH TABLE).

1. Băm dữ liệu.

Băm dữ liệu (*Data hashing*) là đưa dữ liệu từ một tập hợp nào đó sang một danh sách (ánh xạ). Việc một phần tử được đưa vào vị trí nào của danh sách được quyết định bởi hàm băm (*hash function*). Một phần tử của bảng băm còn được gọi là một *bucket*.

Một số ký hiệu:

- **k**: Là một phần tử trong tập hợp.
- **m**: Kích thước của bảng băm.
- **n**: Số lượng phần tử hiện có trong bảng băm.
- **T**: Bảng băm.
- $\boldsymbol{h(k)}$: Hàm băm.

Như vậy:

$$T[h(k)] = k$$

Hệ số tải (Load factor): Thể hiện mức độ không gian được sử dụng của bảng băm hiện có cao bao nhiêu (*số phần tử trung bình được lưu ở mỗi bucket*):

$$\lambda = \frac{n}{m}$$

2. Một số phương pháp giải quyết đụng độ (Collision).

2 phần tử đụng độ với nhau khi cả 2 đều được ánh xạ vào 1 vị trí trong bảng băm.

VD: Có hàm băm: $h(k) = k \% 2$, nhưng vậy thì 2 và 4 sẽ đụng độ vì $h(2) = h(4) = 0$.

a. Phương pháp nối kết (Separate chaining).

Nối kết trực tiếp (Direct chaining):

- Mỗi phần tử trong bảng băm lúc này sẽ là một danh sách động.
- Khi có nhiều phần tử đụng độ, ta đưa chúng vào danh sách tại vị trí đụng độ đó.

Cài đặt bằng danh sách liên kết:

```
struct NODE {
    int key;
    NODE* pNext;
};

typedef NODE* HASHTABLE[M];
```

→ Do bảng băm có kích thước cố định nên M là một hằng số.

Khởi tạo:

```
NODE* CreateNode(int x) {
    NODE* p;
    p = new NODE;
    p->key = x;
    p->pNext = NULL;
    return p;
}

void AddTail(NODE* &head, int x) {
    NODE *p=CreateNode(x);
    if (head == NULL) head = p;
    else {
        NODE* i=head;
        while(i->pNext!=NULL){
            i=i->pNext;
        }
        i->pNext=p;
    }
}

void init(HASHTABLE &H, int numbucket) {
    for (int i = 0; i < numbucket; i++)
        H[i] = NULL;
}
```

→ Khi thêm một key **x** tại **H[vt]**, ta chỉ cần **AddTail(H[vt], x);** là xong.

Xóa key:

```
void removeX(NODE* &p, int x) {
    if (!p)
        return;

    // x ở đầu H[vt]
    if (p->key == x) {
        NODE* temp = p;
        p = p->pNext;
        delete temp;
        return;
    }

    // Tìm x
    NODE* prev = p;
    NODE* curr = p->pNext;
    while (curr != nullptr && curr->key != x) {
        prev = curr;
        curr = curr->pNext;
    }

    // Nếu tìm thấy x
    if (curr) {
        prev->pNext = curr->pNext;
        delete curr;
    }
}

// Trả về true nếu x có thể bị xóa (x có tồn tại trong bảng băm)
bool Remove(HASHTABLE &H, int &numBucket, int x) {
    int vt = HF(numBucket, x);
    NODE* p = H[vt];

    // Tìm x
    while (p && p->key != x)
        p = p->pNext;

    // Nếu tìm thấy
    if (p) {
        removeX(H[vt], x);
        return true;
    } else
        return false;
}
```

Nối tiếp hợp nhất (Coalesced chaining):

[*Đọc thêm*]

- Mỗi phần tử trong bảng băm lúc này sẽ là một cặp (key, pNext). pNext là vị trí của phần tử đụng độ trong bảng băm.
- Khi có nhiều phần tử bị đụng độ, ta đưa chúng vào vị trí trống gần vị trí cuối của bảng băm nhất, sau đó gán pNext là vị trí mà phần tử bị đụng độ được đưa vào.

VD: Hãy đưa các phần tử {1; 2; 3; 4, 5} vào bảng băm có 5 phần tử theo phương pháp Coalesced chaining và $h(k) = k \% 2$:

	0	*1*	*2*	*3*	*4*
key		**1**			
pNext	-1	-1	-1	-1	-1
key	**2**	**1**			
pNext	-1	-1	-1	-1	-1
key	**2**	**1**			**3**
pNext	-1	4	-1	-1	-1
key	**2**	**1**		**4**	**3**
pNext	3	4	-1	-1	-1
key	**2**	**1**	**5**	**4**	**3**
pNext	3	4	-1	-1	2

b. Phương pháp băm lại (Open addressing).

Khi 2 phần tử đụng độ thì: *Tiếp tục băm* phần tử vào sau cho đến khi phần tử vào sau được đưa vào bảng băm.

Gọi $h'(k, i)$ là hàm băm lại lần thứ i, i bắt đầu từ 1, có 3 phương pháp xây dựng hàm băm lại:

- ***Phương pháp dò tuyến tính (Linear probing)***:
$$h'(k, i) = (h(k) + i) \% m$$
Để cho dễ, linear probing chính là đặt phần tử bị đụng độ vào *vị trí trống gần nhất bên phải* vị trí bị đụng độ.
- ***Phương pháp dò bậc 2 (Quadratic probing)***:
$$h'(k, i) = [h(k) + i^2] \% m$$
- ***Phương pháp băm kép (Double hashing)***:
$$h'(k, i) = [h(k) + i.h_1(k)] \% m$$
Trong đó, $h_1(k)$ là một hàm băm phụ, bổ trợ cho băm lại.

Quy trình băm lại:

1. Băm bằng hàm băm thông thường.
2. Nếu đụng độ thì sử dụng hàm băm lại cho đến khi không đụng độ.

Đối với phương pháp băm lại, người ta thường thêm ràng buộc *load factor* để tránh hiện tượng băm lại quá nhiều lần. Sau khi thêm một số phần tử mà vượt quá load factor thì các phần tử phía sau nó sẽ không được thêm vào nữa.

VD:

- Cho hàm băm dò tuyến tính:

$$H(k,i) = (k\ \%\ 10 + i)\ \%\ 10$$

- Hãy đưa các giá trị 41, 45, 51, 30, 62, 80, 82, 90, 100, 120 vào bảng băm sao cho load factor không quá 0.6.

0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	Load factor	
	41									0.1	*41 % 10 = 1*
	41				**45**					0.2	*45 % 10 = 5*
	41				45					0.2	*51 % 10 = 1 (đụng độ)*
	41	**51**			45					0.3	*(1 + 1) % 10 = 2*
30	41	51			45					0.4	*30 % 10 = 0*
40	41	51			45					0.4	*62 % 10 = 2 (đụng độ)*
30	41	51	**62**		45					0.5	*(2 + 1) % 10 = 3*
30	41	51	62		45					0.5	*80 % 10 = 0 (đụng độ)*
30	41	51	62		45					0.5	*(0 + 1) % 10 = 1 (đụng độ)*
30	41	51	62		45					0.5	*(1 + 1) % 10 = 2 (đụng độ)*
30	41	51	62		45					0.5	*(2 + 1) % 10 = 3 (đụng độ)*
30	41	51	62	**80**	45					0.6	*(3 + 1) % 10 = 4*

Ở góc độ kỹ thuật lập trình, người ta thường gắn thêm cho mỗi bucket một trường có tên là **`flag`** để xác định trạng thái của bucket:

```
#define EMPTY 0
#define DELETED -1
#define OCCUPIED 1

struct NODE {
    int flag;
    int key;
};

struct HASHTABLE {
    int M;       // Kích thước của bảng
    int n;       // Số phần tử hiện có
    NODE* table;
};

void init(int M) {
    H.n = 0;
    H.M = M;
    H.table = new NODE[H.M];
    for (int i = 0; i < H.M; i++)
        H.table[i].flag = EMPTY;
}
```

Trong đó:

- **`EMPTY`**: Tượng trưng cho trạng thái bucket trống (mới khởi tạo xong).
- **`DELETED`**: Tượng trưng cho trạng thái bucket vốn có key nhưng đã bị xóa.
- **`OCCUPIED`**: Tượng trưng cho trạng thái bucket đã có key.

Trong nhiều trường hợp, flag `EMPTY` và `DELETED` có thể được coi là một trường là **`isEmpty`**.

Khởi tạo và chèn key vào bảng băm:

```
void Insert(HASHTABLE& H, int x) {
    int vt = HF(H, x); // Hàm băm chính
    int i = 1;

    while (true) {
        // bucket đang trống
        if (H.table[vt].flag == EMPTY || H.table[vt].flag == DELETED) {
            H.table[vt].key = x;
            H.table[vt].flag = OCCUPIED;
            H.n++;
            return;

        // key bị trùng
        } else if (H.table[vt].key == x)
            return;

        else
            vt = reHF(H, x, i++); // Hàm băm lại
    }
}

void CreateHashTable(HASHTABLE& H){
    H.n = 0;

    cin >> H.M;

    int x;

    while(true){
        cin >> x;
        if(x == -1)
            return;
        if(float(H.n + 1) / H.M <= 0.7) // Kiểm tra hệ số tải
            Insert(H, x);
    }
}
```

→ Để tránh băm lại vô hạn:

- Trước khi đưa 1 key vào table cần kiểm tra hệ số tải.
- Nếu key đã tồn tại trong table thì không cần băm lại.
- Cần giới hạn số lần băm lại (trong code trên không có chi tiết này).

Xóa key:

```
void Delete(HASHTABLE& H, int x) {
    int vt = HF(H, x);
    int i = 1;

    while (true) {
        // key x không tồn tại
        if (H.table[vt].flag == EMPTY || H.table[vt].flag == DELETED)
            return;

        // key x có tồn tại
        else if (H.table[vt].key == x) {
            H.table[vt].flag = DELETED;
            H.n--;
            return;
        } else
            vt = HF_LinearProbing(H, x, i++);
    }
}
```

3. Một số phương pháp xây dựng hàm băm.

Có 3 phương pháp chính:

- **Phương pháp chia**:

$$h(k) = k \ \% \ m$$

Nếu m là số nguyên tố thì hiệu quả của bảng băm này cao.

- **Phương pháp nhân**:

$$h(k) = \lfloor m.(k.A \ \% \ 1) \rfloor$$

Trong đó:

  - $\lfloor \ \ \rfloor$ là ký hiệu của phép làm tròn xuống, tương ứng với hàm `floor()`.
  - $k.A \ \% \ 1$ là phần thập phân của kA khi $0 < A < 1$. Theo Knuth, hàm băm sẽ hoạt động tốt khi $A = \frac{1}{2}.(\sqrt{5} - 1)$.

- **Phương pháp dùng định lý họ hàm băm phổ quát**:

$$h(k) = \big((ak + p) \ \% \ p\big) \ \% \ m$$

Trong đó:

  - p: Số nguyên tố tùy ý lớn hơn tất cả k.
  - a, b: Hai số tự nhiên tùy ý sao cho $1 \le a \le p - 1, \ 0 \le b \le p - 1$.

Phương pháp này đảm bảo xác suất xảy ra đụng độ $\le \frac{n}{m}$.

Đối với chuỗi, ta có một số phương pháp băm như sau:

$$h(s) = \sum_{i=0}^{|s|-1} \big(int(s_i)\big) \ \% \ m$$

Trong đó, $int(c)$ là số nguyên ASCII ứng với ký tự c.

4. Cấu trúc map, muiltimap và unorderd_map.

Trong C++, cả map, unordered_map và multimap đều là những cấu trúc dựa trên ý tưởng của bảng băm, nghĩa là bạn có thể truy xuất đến 1 phần tử trong bảng qua key (không nhất thiết phải là index).

	map	**multimap**	**unordered_map**
Thư viện	`map`	`map`	`unorderd_map`
Khai báo	`map<KeyType, ValueType> myMap;`	`multimap<KeyType, ValueType> myMap;`	`unordered_map<KeyType, ValueType> myMap`
Cho phép trùng key	Không.	Có.	Không.
Thứ tự các key	Tăng dần.	Theo thứ tự key được đưa vào.	Tăng dần.
Cấu trúc bên trong	Cây đỏ đen.	Cây đỏ đen.	Bảng băm xử lý đụng độ bằng direct chaining.

Về mặt lập trình, cả 3 cấu trúc trên đều được xây dựng là tập hợp các cặp pair {key, value}. Key là giá trị khóa để truy xuất đến value.

Một số phương thức:

`myMap[key];`	Tạo key không có value. (Không dùng với multimap).
`myMap[key] = value;`	Nếu key chưa tồn tại, tạo key và gán cho value. Nếu key đã tồn tại, sửa value của nó. (Không dùng với multimap).
`myMap.insert({key, value})`	Chèn cặp {key, value}.
`myMap.count(key)`	Đếm số lần xuất hiện của key.
`myMap.erase(key)`	Xóa key.

Cây đỏ đen là một cây nhị phân tìm kiếm tự cân bằng. Mỗi thao tác trên cây đều có độ phức tạp $O(\log n)$. Cây đỏ đen có 4 quy tắc xây dựng:

1. Mỗi node có màu hoặc đỏ (RED) hoặc đen (BLACK).
2. Gốc và Các node lá (NULL hoặc NIL) là đen.
3. Không có 2 node đỏ liền kề.
4. Từ một node đến tất cả các lá con của nó, số lượng node đen phải giống nhau (black-height).

Phần B: CẤU TRÚC DỮ LIỆU PHI TUYẾN.

I. CẤU TRÚC DỮ LIỆU TẬP HỢP (SET).

1. Set, multiset và unordered_set.

Set (thư viện `set`) là một cấu trúc dữ liệu có sẵn trong C++ dựa trên cây đỏ đen, dùng để lưu trữ các giá trị không trùng nhau và được tự động sắp xếp tăng dần.

Một số phương thức:

`set <type> mySet;`	Tạo set.
`mySet.insert(x);`	Chèn x vào set.
`mySet.count(x)`	Trả về số lần xuất hiện của x trong set.
`mySet.size()`	Trả về kích thước của set.
`mySet.clear();`	Xóa các phần tử của set.

Set cũng có multiset và unordered_set tương tự như map:

- **multiset**: Cho phép các phần tử trùng nhau.
- **unordered_set**: Các phần tử giữ thứ tự như khi chúng được đưa vào, không xếp tăng dần.

II. CẤU TRÚC DỮ LIỆU ĐỒ THỊ (GRAPH).

1. Giới thiệu.

Đồ thị (graph) là cấu trúc dữ liệu phi tuyến, truy xuất tuần tự hoặc ngẫu nhiên (tùy cách lưu trữ), bao gồm các đỉnh (verticle) để lưu dữ liệu và các cạnh (edge) để kết nối các đỉnh.

[*Xem trong môn **Cấu trúc rời rạc***]

2. Bài toán lưu trữ đồ thị.

a. Các phương pháp lưu trữ đồ thị.

Có rất nhiều cách để lưu trữ đồ thị trong máy tính.

	Ma trận kề *(Adjacency matrix)*	**Danh sách kề** *(Adjacency list)*	**Danh sách cạnh** *(Edge list)*	**Ma trận liên thuộc** *(Incidence matrix)*
Bản chất	Danh sách 2 chiều.	Danh sách 1 chiều.	Danh sách 1 chiều.	Danh sách 2 chiều.
xKhông trọng số	$A_{uv} = true \Leftrightarrow \exists(u,v)$	$A_u = \{v\} \Leftrightarrow \exists(u,v)$	$A_i = \{(u,v)\} \Leftrightarrow \exists(u,v)$	$A_{ue} = \begin{cases} -1, & e = (u,v) \\ 0, & e = \emptyset \\ 1, & e = (v,u) \end{cases}$
Có trọng số	$A_{uv} = \lvert(u,v)\rvert$	$A_u = \{(v, \lvert(u,v)\rvert)\}$	$A_i = \{(u,v, \lvert(u,v)\rvert)\}$	
Độ phức tạp thời gian	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Độ phức tạp không gian	$O(n^2)$	$\leq O(n^2)$	$O(n)$	$O(n^2)$
Kết luận	Tốt khi đồ thị dày.			

Trong phạm vi học tập môn Cấu trúc dữ liệu và Giải thuật, không nghiên cứu về *Ma trận liên thuộc*.

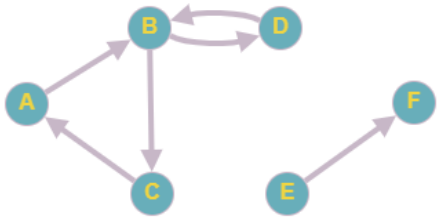
Trên thực tế, đồ thị có thể được lưu trữ bằng một số cấu trúc dữ liệu sau:

	Không trọng số	**Có trọng số**
Ma trận kề	`bool graph[v][v];` `vector <vector <bool>> graph;`	`int graph[v][v];` `vector <vector <int>> graph;`
Danh sách kề	`map <char, set<char> > graph;` `map <char, vector<char> > graph;`	`map <char, set<pair<char, int> > > graph;` `map <char, vector<pair<char, int> > > graph;`
Danh sách cạnh	`vector<pair<char, char>> graph;`	`vector<tuple <char, char, int> > graph;`

Chú ý:

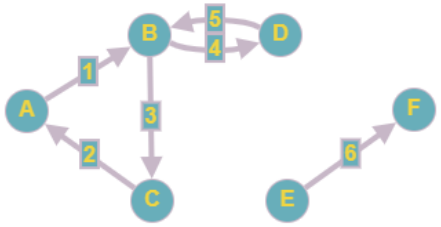
- Giả sử các đỉnh được ký hiệu là ký tự **char** → Đối với ma trận kề, tên gọi các đỉnh buộc phải được ánh xạ sang các số tự nhiên từ *0 đến v – 1*.
- Giữa các phương pháp lưu trữ trên (ma trận kề, danh sách kề, danh sách cạnh) có thể được chuyển đổi qua lại.
- Ngoài ra còn một số cấu trúc khác như con trỏ nhiều chiều hay danh sách liên kết nhiều chiều, nhưng do xử lý phức tạp nên không đề cập đến trong phần này.

VD:
Biểu diễn đồ thị sau bằng 3 cách:



Ma trận kề	Danh sách kề	Danh sách cạnh
A B C D E F A 0 1 0 0 0 0 B 0 0 1 1 0 0 C 1 0 0 0 0 0 D 0 1 0 0 0 0 E 0 0 0 0 0 1 F 0 0 0 0 0 0	A : B B : C D C : A D : B E : F F :	A B B C B D C A D B E F

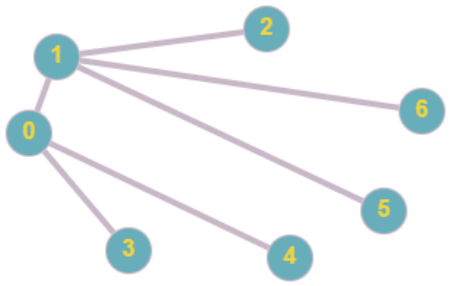
VD:
Biểu diễn đồ thị sau bằng 3 cách:



Ma trận kề	Danh sách kề	Danh sách cạnh
A B C D E F A 0 1 0 0 0 0 B 0 0 3 4 0 0 C 2 0 0 0 0 0 D 0 5 0 0 0 0 E 0 0 0 0 0 6 F 0 0 0 0 0 0	A : (B 1) B : (C 3) (D 4) C : (A 2) D : (B 5) E : (F 6) F :	A B 1 B C 3 B D 4 C A 2 D B 5 E F 6

VD1: ***Liệt kê các đỉnh của đồ thị***.

Cho một đồ thị vô hướng G = (V, E) với V là tập đỉnh và E là tập cạnh. Quy ước đánh số các đỉnh là số nguyên (0,1,…). Giả sử thông tin đầu vào của bài toán gồm số cạnh và danh sách các cạnh của đồ thị được nối từ 2 đỉnh. Hãy liệt kê các đỉnh của đồ thị.

Input	Output	Đồ thị
12 0 3 0 4 0 1 1 0 1 2 1 5 1 6 2 1 3 0 4 0 5 1 6 1	0 1 2 3 4 5 6	

Code mẫu:

```cpp
#include <iostream>
#include <map>
#include <set>

using namespace std;

int main()
{
    map <int, set<int>> graph;

    // Đọc vào số cạnh
    int m;
    cin >> m;

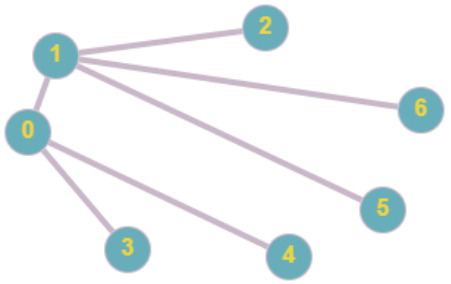
    // Đọc vào từng cạnh (mỗi cạnh biểu thị bằng 2 đỉnh u và v)
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].insert(v);
        graph[v].insert(u); // Do là đồ thị vô hướng nên (u, v) là (v, u)
    }

    // Xuất đồ thị
    for (auto it = graph.begin(); it != graph.end(); it++)
        cout << it->first << ' ';

    return 0;
}
```

VD2: ***Xuất ma trận kề của đồ thị***.

Cho một đồ thị vô hướng G = (V, E) với V là tập đỉnh và E là tập cạnh. Quy ước đánh số các đỉnh là số nguyên (0,1,…). Giả sử thông tin đầu vào của bài toán gồm số cạnh, số đỉnh và danh sách các cạnh của đồ thị được nối từ 2 đỉnh. Hãy xây dựng 2 hàm đọc dữ liệu và xuất ra ma trận kề của đồ thị này.

Input	Output	Đồ thị
12 7 0 3 0 4 0 1 1 0 1 2 1 5 1 6 2 1 3 0 4 0 5 1 6 1	0 1 2 3 4 5 6	

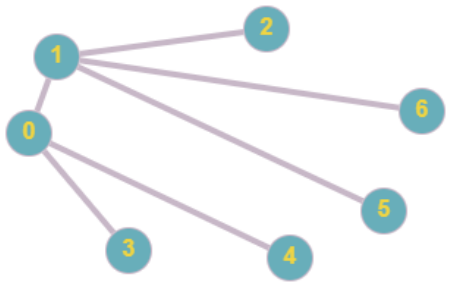
Code mẫu:

```
void Output(vector<vector<int>> G, int v) {
    for (int i = 0; i < v; i++) {
        for (int j = 0; j < v; j++)
            cout << G[i][j] << ' ';
        cout << '\n';
    }
}

void Input(vector<vector<int>>& G, int v, int e) {
    cin >> e >> v;
    for (int i = 0; i < e; i++) {
        int a, b;
        cin >> a >> b;
        G[a][b] = 1;
        G[b][a] = 1;
    }
}
```

VD2: ***Xuất ma trận kề của đồ thị***.

Cho một đồ thị vô hướng G = (V, E) với V là tập đỉnh và E là tập cạnh. Quy ước đánh số các đỉnh là số nguyên (0,1,…). Giả sử thông tin đầu vào của bài toán gồm số cạnh, số đỉnh và danh sách các cạnh của đồ thị được nối từ 2 đỉnh. Hãy xây dựng 2 hàm đọc dữ liệu và xuất ra ma trận kề của đồ thị này.

Input	Output	Đồ thị
12 7 0 3 0 4 0 1 1 0 1 2 1 5 1 6 2 1 3 0 4 0 5 1 6 1	0 1 0 1 1 0 0 1 0 1 0 0 1 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0	

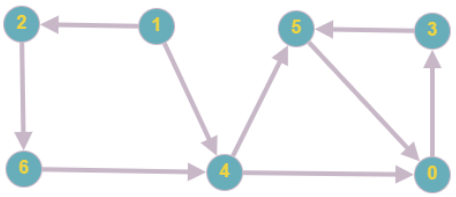
Code mẫu:

```
void Output(vector<vector<int>> G, int v) {
    for (int i = 0; i < v; i++) {
        for (int j = 0; j < v; j++)
            cout << G[i][j] << ' ';
        cout << '\n';
    }
}

void Input(vector<vector<int>>& G, int v, int e) {
    cin >> e >> e;
    for (int i = 0; i < e; i++) {
        int a, b;
        cin >> a >> b;
        G[a][b] = 1;
        G[b][a] = 1;
    }
}
```

VD3: ***Tính bậc của đỉnh trong đồ thị vô hướng***.

Cho một đồ thị có hướng G=(V, E) với V là tập đỉnh và E là tập cạnh. Quy ước đánh số các đỉnh là số nguyên (0,1,…). Giả sử thông tin đầu vào của bài toán gồm số cạnh và danh sách các cạnh của đồ thị được nối từ 2 đỉnh. Hãy cho biết bậc 1 đỉnh bất kỳ của đồ thị này.

Input	Output	Đồ thị
10 5 0 3 5 0 3 4 5 6 4 2 6 1 2 1 4 4 0 0 1 5	3	

Code mẫu:

```
#include <iostream>
#include <map>
#include <set>

using namespace std;

void Input(map<int, set<int>>& graph) {
    int e;
    cin >> e;

    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].insert(v);
    }
}

int deg(map<int, set<int>> graph, int k) {
    int ans = 0;

    for (auto it : graph)
        if (it.first == k)
            ans += it.second.size(); // Deg out
        else
            for (auto v : it.second)
                if (v == k)
                    ans++; // Degin
    return ans;
}

int main()
{
        map<int, set<int>> graph;

        Input(graph);

        int k;
```

```
        cin >> k;
        cout << deg(graph, k);

        return 0;
}
```

3. Bài toán duyệt đồ thị.

a. Duyệt theo chiều sâu (DFS, Deep-Frist Search).

Ý tưởng: Xét đỉnh u:

1. Đưa u vào đỉnh đã duyệt.
2. Kiểm tra các đỉnh kề với u xem có đỉnh nào chưa duyệt không, nếu có, xét đỉnh đó.

Code mẫu với đệ quy:

```
void DFS(map<string, set<string>> graph, set<string>& visited, string u) {
    visited.insert(u);
    for (auto v : graph[u]) {
        if (visited.find(v) == visited.end())
            DFS(graph, visited, v);
    }
}
```

b. Duyệt theo chiều rộng (BFS, Breath-Frist Search).

Ý tưởng: Xét đỉnh u:

1. Đưa u vào đỉnh đã duyệt.
2. Lần lượt đưa các đỉnh kề với u mà chưa được duyệt vào queue.
3. Xét các đỉnh trong queue theo thứ tự.

Code mẫu với queue:

```
void BFS(const map<string, set<string>>& graph, const string& start) {
    set<string> visited;
    queue<string> q;

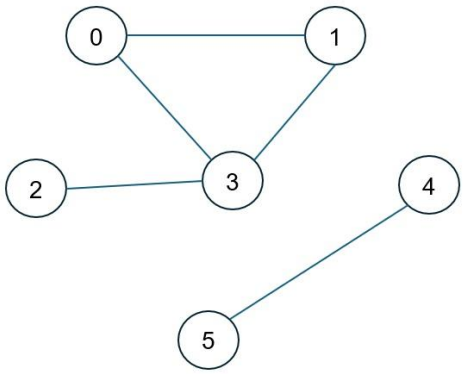
    visited.insert(start);
    q.push(start);

    while (!q.empty()) {
        string u = q.front(); q.pop();

        for (auto v : graph[u]) {
            if (visited.find(v) == visited.end()) {
                visited.insert(v);
                q.push(v);
            }
        }
    }
```

VD1: ***Kiểm tra đường đi***.

Cho một đồ thị vô hướng G có n cạnh và m đỉnh. Quy ước đánh số các đỉnh là số nguyên (0,1,…). Với đỉnh thứ u (u > 0), hãy xây dựng các hàm kiểm tra xem có tồn tại đường đi từ đỉnh 0 đến đỉnh u không.

Input	Output	Đồ thị
6 5 0 1 0 3 1 3 2 3 4 5	CO CO CO KHONG KHONG	

Ý tưởng:

- Sử dụng thuật toán DFS / BFS để tìm đường đi. Xét đỉnh thứ u:
  - Nếu u là đỉnh đích thì trả về **`True`**.
  - Duyệt các đỉnh kề với u, nếu có đỉnh chưa được duyệt mà liên thông tới u thì trả về **`True`**, ngược lại là **`False`**.
- **Chú ý**:
  - Trong trường hợp sử dụng DFS, nếu liên tục truyền vào `map <int, set <int>> graph` và `set <int> visited` sẽ tạo ra rất nhiều bản sao, gây *lãng phí bộ nhớ và thời gian*.
  - Cách khắc phục: Truyền tham chiếu **`const map <int, set <int>>& graph`** và **`set <int>& visited`**.
  - Đối với `graph` là tham chiếu hằng, ta cần sử dụng phương thức `graph.at(u)` tương đương với `graph[u]`, vì `graph[u]` sẽ tạo mới `u` trong graph nếu `u` không tồn tại, mâu thuẫn với từ khóa `const`. Nhưng nếu muốn sử dụng `.at(u)`, ta cần kiểm tra xem `u` có tồn tại trong `graph` không trước.

Code mẫu:

```
bool HavePathTo(int u, int e, const map<int, set<int>>& graph, set<int>& visited) {
    if (graph.find(u) == graph.end())
        return false;

    if (u == e)
        return true;

    visited.insert(u);

    for (auto v : graph.at(u))
        if (visited.find(v) == visited.end())
            if (HavePathTo(v, e, graph, visited))
                return true;

    return false;
}
```

```
void Output(const map <int, set <int>>& graph, int m) {
    for (int i = 1; i < m; i++) {
        set <int> visited;
        cout << (HavePathTo(0, i, graph, visited) ? "CO\n" : "KHONG\n");
    }

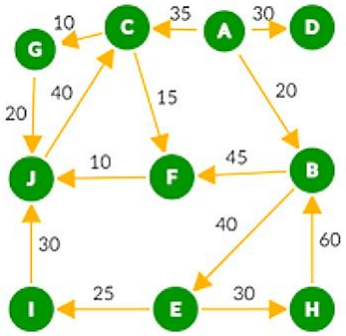
}
```

4. Bài toán tìm đường đi ngắn nhất bằng thuật toán Dijkstra.

Ý tưởng: [*Xem trong môn **Cấu trúc rời rạc***].

VD1: ***Tìm đường đi ngắn nhất 1***.

Viết chương trình nhập vào 1 đồ thị có n đỉnh à e cạnh và m cặp đỉnh đầu, cuối cần tìm đường đi như ví dụ bên dưới. Chương trình xuất ra m chuỗi đường đi ngắn nhất dùng Dijkstra và độ dài đường đi ngắn nhất, nếu không có đường đi thì ghi ra `no_path`.

Input	Output	Đồ thị
10 14 A B C D E F G H I J A B 20 A C 35 A D 30 B E 40 B F 45 C F 15 C G 10 E H 30 E I 25 F J 10 G J 20 H B 60 I J 30 J C 40 2 A J A F	A C F J 60 A C F 50	

Code mẫu:

```
#include <iostream>
#include <map>
#include <set>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

#define w first
#define v second

void readGraph(map <char, set < pair<int, char> > >& graph) {
    int n, e;
    cin >> n >> e;

    for (int i = 0; i < n; i++) {
        char u;
        cin >> u;
        graph[u];
    }

    for (int i = 0; i < e; i++) {
        char u, v;
        int w;
        cin >> u >> v >> w;
        graph[u].insert({w, v});
```

```
    }
}


pair <vector <char>, int> minPath2(char e, char s, const map <char, set <pair <int,
char> > >& graph) {
    // Khởi tạo đường đi đến các đỉnh là vô cùng
    map <char, pair <int, char> > minPaths;
    for (auto it = graph.begin(); it != graph.end(); it++)
        minPaths[it->first] = {INT_MAX, s};

    // Xét đỉnh hiện tại
    set <pair <int, char> > currVs;
    currVs.insert({0, s});

    while (!currVs.empty()) {
        auto currV = *currVs.begin();
        currVs.erase(currVs.begin());

        // Xét những đỉnh kề với đỉnh hiện tại
        for (auto adjV : graph.at(currV.v)) {
            int newW = currV.w + adjV.w;
            if (minPaths[adjV.v].w > newW) {
                minPaths[adjV.v] = {newW, currV.v};
                currVs.insert({newW, adjV.v});
            }
        }
    }

    // Truy vết để tìm các đỉnh trên đường đi
    int minW = minPaths[e].w;
    vector <char> path;
    while (e != s) {
        path.push_back(e);
        e = minPaths[e].v;
    }
    path.push_back(s);
    reverse(path.begin(), path.end());

    return {path, minW};
}
```

```
void Output(const map <char, set <pair <int, char> > >& graph) {
    int m;
    cin >> m;

    for (int i = 0; i < m; i++) {
        char s, e;
        cin >> s >> e;

        pair <vector <char>, int> minPath = minPath2(e, s, graph);
        if (minPath.second == INT_MAX)
            cout << "no_path";
        else {
            for (auto v : minPath.first)
                cout << v << ' ';
            cout << minPath.second;
        }
        cout << '\n';
    }
}


int main()
{
    map <char, set <pair <int, char> > > graph;
    readGraph(graph);
    Output(graph);
    return 0;
}
```

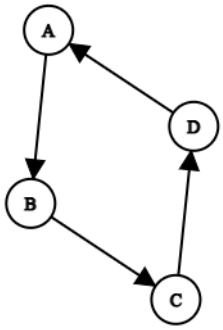
III. CẤU TRÚC DỮ LIỆU CÂY (TREE).

Cây (tree) là một trường hợp đặc biệt của đồ thị, bao gồm các nút (node) để lưu dữ liệu và các cạnh (edge) để kết nối các node. Trong số các node, node đầu tiên được tạo được gọi là gốc (root). Mỗi cây chỉ có một root.

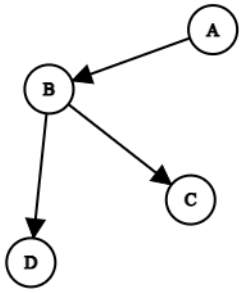
- **Cây con (Subtree)**: Là một phần của cây, bao gồm các nút con.

Đặc điểm:
- Các cạnh chỉ có 1 chiều, không có chu kỳ.
- Các node đều được liên kết với nhau.
- Không có node tự trỏ đến chính nó.

VD.



Không phải cây.



Cây.

Phân loại cành:
- **Cạnh (Edge)**: Là đường nối giữa 2 nút.
- **Đường đi (Path)**: Là tập hợp các node có chung cạnh.
- **Độ dài đường đi (Length of path)**: Là số cạnh của đường đi.
- **Độ sâu của nút (Depth of node)**: Là độ lệch của bậc node đó và bậc node gốc.
- **Chiều cao của cây (Height of tree)**: Là độ lệch của bậc node gốc và bậc node sâu nhất.
- **Độ cao của nút (Height of node)**: Là độ lệch của bậc node đó và bậc node sâu nhất.
- **Độ cao của nút (Height of tree)**: Là độ lệch của bậc node gốc và bậc node sâu nhất.

Suy ra:
- Tổng số node của 1 cây là tổng số node của các cây con + 1.
- Chiều cao của 1 cây là chiều cao lớn nhất của các cây con + 1.

Phân loạt node:
- **Cha / Tổ tiên (Parent / Ancestor)**: Là nút có cạnh hướng đến nút khác.
- **Con (Subnode / Descendant)**: Là nút được một nút khác có cạnh hướng đến.
- **Gốc (Root)**: Là nút đầu tiên, trên cùng của cây.
- **Lá (Leaf)**: Là nút không có nút con.

Bậc / Mức (Level, đếm từ bậc 0 đến n):

- **Bậc của node**: Là số cây con của node.
- **Bậc của cây**: Là bậc lớn nhất trong số các bậc của các node trong cây.

Lá luôn có bậc cao nhất. Gốc luôn có bậc thấp nhất.

- **Khóa (Key)**: Là giá trị của một nút.

1. Cây nhị phân (Binary tree).

a. Giới thiệu.

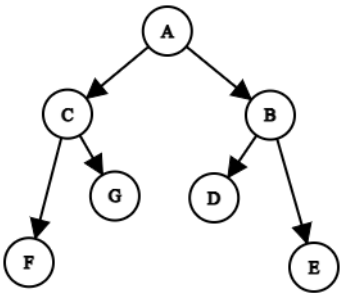
Cây nhị phân là cây mà mỗi node cha có tối đa 2 node con, gọi là node trái và node phải.

Trong cây nhị phân, xuất phát từ một node chỉ có 1 path đến node khác.

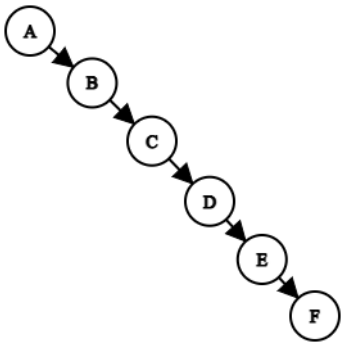
Tính chất:

- Bậc thứ i có tối đa 2^i node.
- Cây có chiều cao h có tối đa $2^{h+1} - 1$ node.
- Chiều cao của cây ít nhất là $\log_2 n$ và cao nhất là $n - 1$. Trong đó, chiều cao của cây tại 1 nút T sẽ được tính là:

$$\mathrm{h}(T) = \max(\mathrm{h}(T_l), \mathrm{h}(T_r)) + 1$$



Cây nhị phân có chiều cao thấp nhất.



Cây nhị phân có chiều cao cao nhất.

b. Các tính chất đặc biệt liên quan đến số lượng node, cây nhị phân đầy đủ và cây nhị phân hoàn chỉnh.

Cây nhị phân đầy đủ (Full binary tree) là cây mà mỗi node cha có đủ 2 node con.

Cây nhị phân hoàn chỉnh (Complete tree) là cây mà mỗi ở bậc thứ i có đủ 2^i node, ngoại trừ bậc cuối cùng. Trong trường hợp các node lá không có đủ $2^{\max i}$ node thì các node lá sẽ lệch về bên trái.

Nếu mức thứ i là đầy đủ thì số lượng node của mức thứ i là 2^i (số node tối đa). Số node trái và node phải sẽ bằng nhau:

$$|t_l| = |t_r| = \frac{2^i}{2} = 2^{i-1}$$

- Nếu $|t_l| = |t_r|$: Cây cân bằng tại t.
- Nếu $|t_l| < |t_r|$: Cây lệch trái tại t.
- Nếu $|t_l| > |t_r|$: Cây lệch phải tại t.

c. Cài đặt cấu trúc dữ liệu cây nhị phân.

```
struct TNode {
    int key;
    TNode *pLeft;
    TNode *pRight;
};

typedef TNode *TREE;
```

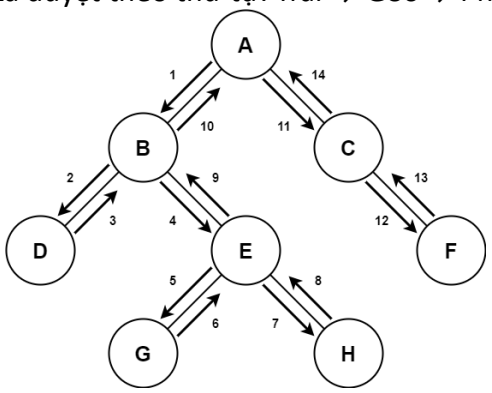
d. Duyệt cây.

Duyệt theo chiều sâu, có 3 cách duyệt cây:

- ***Duyệt trung thứ tự (In-order tranversal – LNR)***:
  - Nếu có con trái thì đi xuống con trái.
  - In con hiện tại.
  - Nếu có con phải thì đi xuống con phải.
- ***Duyệt tiền thứ tự (Pre-order tranversal – NLR)***:
  - In con hiện tại.
  - Nếu có con trái thì đi xuống con trái.
  - Nếu có con phải thì đi xuống con phải.
- ***Duyệt hậu thứ tự (Post-order tranversal – LRN)***:
  - Nếu có con trái thì đi xuống con trái.
  - Nếu có con phải thì đi xuống con phải.
  - In con hiện tại.

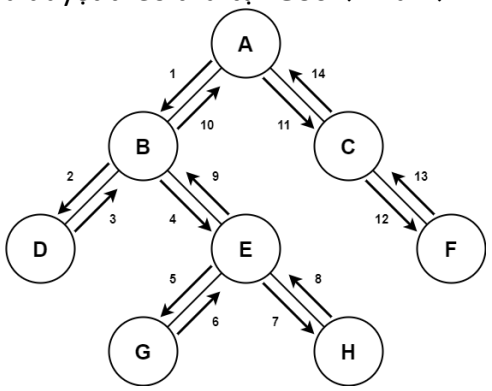
VD: Cho cây:

- LNR: Là duyệt theo thứ tự: Trái → **Gốc** → Phải.



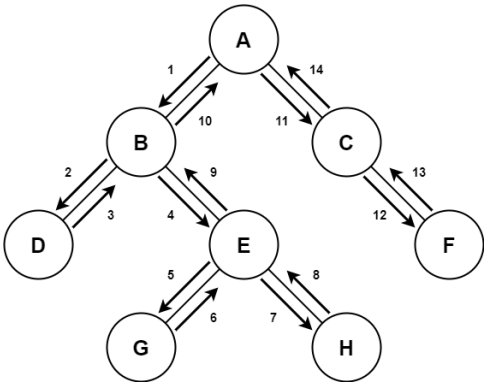
Thứ tự sau khi duyệt là: D, B, G, E, H, A, F, C.

- NLR: Là duyệt theo thứ tự: **Gốc** → Trái → Phải.



Thứ tự sau khi duyệt là: A, B, D, E, G, H, C, F.

- LRN: Là duyệt theo thứ tự: Trái → Phải → **Gốc**.



Thứ tự sau khi duyệt là: D, G, H, E, B, F, C, A.

Cùng một danh sách thứ tự duyệt và cùng một kiểu duyệt nhưng vẫn có thể suy ra nhiều cây khác nhau.

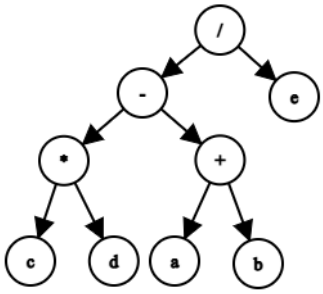
Chú ý: Thứ tự của duyệt trung thứ tự cũng chính là thứ tự sắp xếp ***không giảm dần*** của các khóa của cây.

Liên hệ với biểu thức:

Mỗi biểu thức có thể coi như là một thứ tự duyệt trong cây nhị phân.

- Node lá: ***Toán hạng***.
- Không phải node lá: ***Toán tử***.

Giả sử có cây biểu thức như sau:



- ***Biểu thức trung tự***: Thu được nhờ duyệt trung tự:
  `((c * d) – (a + b)) / e`
- ***Biểu thức tiền tự***: Thu được nhờ duyệt tiền tự:
  `/ - * c d + a b e`
- ***Biểu thức hậu tự***: Thu được nhờ duyệt hậu tự:
  `c d * a b + - e /`

Cách cài đặt:

```
void preorder(TREE root) {
    if (root) {
        cout << root->key << " ";
        preorder(root->pLeft);
        preorder(root->pRight);
    }
}
void inorder(TREE root) {
    if (root) {
        inorder(root->pLeft);
        cout << root->key << " ";
        inorder(root->pRight);
    }
}
void postorder(TREE root) {
    if (root) {
        postorder(root->pLeft);
        postorder(root->pRight);
        cout << root->key << " ";
    }
}
```

Đối với phương pháp đệ quy, cách làm khá đơn giản, ta chỉ cần gọi đệ quy dựa trên thứ tự của cách duyệt.

e. Một số cài đặt khác.

Đếm số node trong cây:

$$|T| = |T_{left}| + |T_{right}| + 1$$

```
int DemNode(TREE t) {
    if (t == NULL)
        return 0;
    int a = DemNode(t->pLeft);
    int b = DemNode(t->pRight);
    return a + b + 1;
}
```

Tìm chiều cao của cây:

$$\text{Height}(T) = \max\left(\text{Height}(T_{left}), \text{Height}(T_{right})\right) + 1$$

```
int Height(TNODE* T) {
    if (!T)
        return -1;
    int a = Height(T->pLeft);
    int b = Height(T->pRight);
    return (a > b ? a : b) + 1;
}
```

2. Cây tìm kiếm nhị phân (Binary search tree, BST).

a. Giới thiệu cây tìm kiếm nhị phân.

Cây tìm kiếm nhị phân là cây nhị phân mà ***|node con trái| < |node cha| < |node con phải|***. Dấu bằng (=) có thể xảy ra ở một trong hai vế, tùy vào yêu cầu cài đặt.

Ý nghĩa: Cây cho phép truy xuất tuần tự với tốc độ $O(\log n)$.

Tính chất đặc biệt:

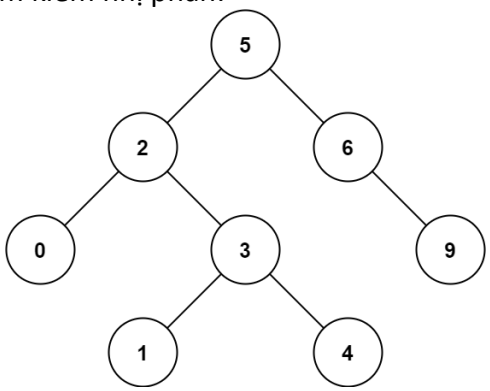
- Lá là nút có left == right == nullptr.
- **Inorder** cho ra danh sách các khóa được xếp theo thứ tự **tăng dần**.
- |Mọi node trong cây con trái| < |Node cha| < |Mọi node trong cây con phải|.

Tính chất:

- Bậc thứ i có tối đa 2^i node.
- Cây có chiều cao h có tối đa $2^{h+1} - 1$ node.
- Chiều cao của cây ít nhất là $\log_2 n$ và cao nhất là $n - 1$.

VD. Danh sách sau có thể được chuyển đổi thành cây tìm kiếm nhị phân:

5; 2; 6; 0; 3; 1; 4; 9.



b. Một số thao tác với BST.

Kiểu dữ liệu cây tìm kiếm nhị phân:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Tạo node:

```
TNODE* CreateTNode(int x) {
    TNODE *p;
    p = new TNODE;

    p->key = x;
    p->pLeft = NULL;
    p->pRight = NULL;

    return p;
}
```

Thêm node:

- Bản đệ quy:

```
int Insert(TREE &T, int x) {

    if (T) {
        // Nếu x đã tồn tại thì không cần Insert nữa
        if (T->key == x)
            return 0;

        // Nếu x nằm ở cây con trái
        if (T->key > x)
            return Insert(T->pLeft, x);

        // Nếu x nằm ở cây con phải
        else
            return Insert(T->pRight, x);
    }

    // Nếu cây rỗng thì x là root
    T = CreateTNode(x);

    return 1;
}
```

- Bản khử đệ quy:

```
int Insert(TREE &Root, int x) {

    // Trường hợp cây rỗng
    if (Root == NULL)
        Root = CreateTNode(x);

    TREE T = Root;

    // Bắt đầu duyệt cây
```

```
    while (T) {
        // Nếu x đã tồn tại thì không cần Insert nữa
        if (T->key == x)
            return 0;

        // Nếu x nằm ở cây con trái
        if (x < T->key) {
            if (T->pLeft == NULL)
                T->pLeft = CreateTNode(x);
            else T = T->pLeft;
        }

        // Nếu x nằm ở cây con phải
        else {
            if (T->pRight == NULL)
                T->pRight = CreateTNode(x);
            else T = T->pRight;
        }
    }

    return 1;
}
```

Tìm node: Trong cây tìm kiếm nhị phân, việc tìm kiếm một node có độ phức tạp trung bình $O(\log n)$ (tương tự tìm kiếm nhị phân, ta không cần kiểm tra toàn bộ node mà lần lượt so sánh giá trị của giá trị cần tìm với node đang xét):

```
TNODE* searchNode(TREE T, int x) {
    if (T != NULL) {
        if (T->key == x)
            return T;
        if (T->key > x)
            return searchNode(T->pLeft, x);
        else
            return searchNode(T->pRight, x);
    }
    return NULL;
}
```

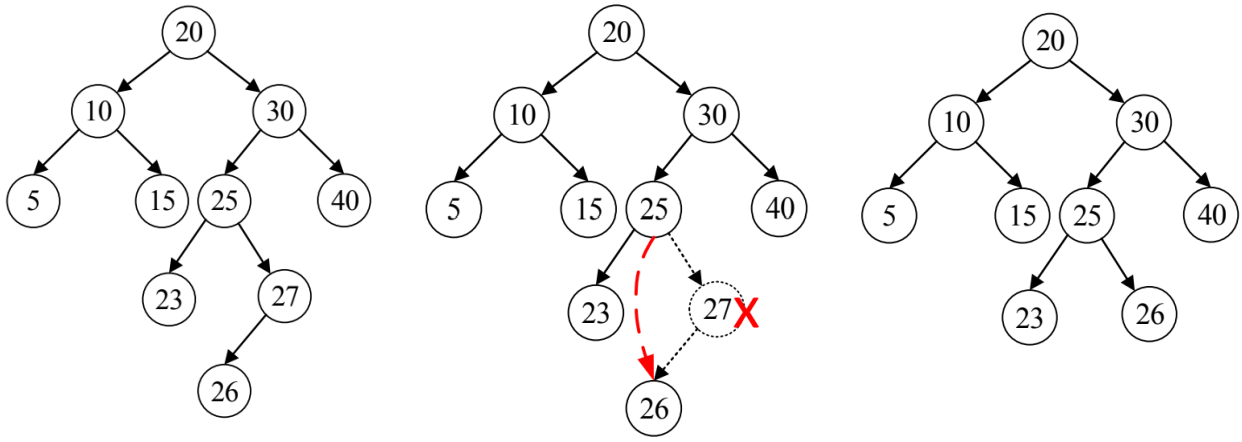
Xóa node (bảo toàn cây nhất có thể): Gọi target là node cần xóa.

- TH$_1$: ***Target không có con*** → Xóa luôn.
- TH$_2$: ***Target có 1 con*** → Nối con đó với node cha của target và xóa target.
- TH$_3$: ***Target có 2 con***:
    - Tìm một node để thay thế target.
    - Node này có thể là:
        - Node lớn nhất của cây con trái.
        - Node nhỏ nhất của cây con phải.
    - Sau khi thay thế, quy về 2 trường hợp trên với node dùng để thay thế.

Độ phức tạp: $O(h)$.

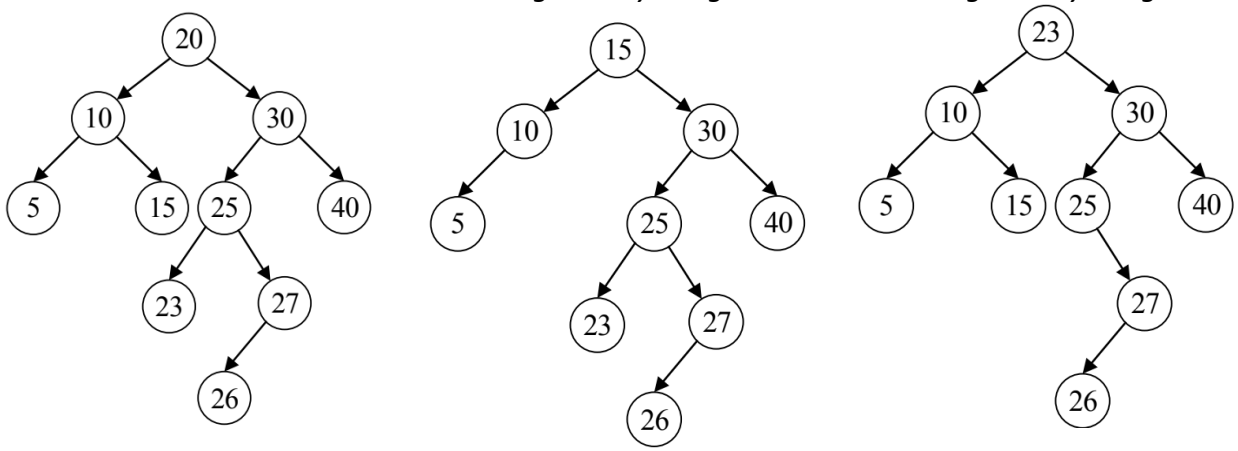
VD1: Target có 1 con: Xóa node 27 trong cây sau:

Ban đầu. *Nối con target với cha target.* *Xóa target.*



VD2: Target có 2 con: Xóa node 20 trong cây sau:

Ban đầu. *Hướng 1: Thay bằng 15.* *Hướng 2: Thay bằng 23.*



Chú ý: Trong trường hợp cây bị duỗi thẳng:

- Node nhỏ nhất của cây con phải có thể nằm ở bên trái.
- Node lớn nhất của cây con trái có thể nằm ở bên phải.

Code mẫu:

```
void deleteNode(int x, TNODE*& T) {
    if (!T)
        return;

    // Find x

    if (x > T->key)
        deleteNode(x, T->pRight);
    else if (x < T->key)
        deleteNode(x, T->pLeft);
    else {
        // Case 1: x not have any children

        if (!T->pLeft && !T->pRight) {
            delete T;
            T = nullptr;
            return;
        }

        TNODE* i = T;

        // Case 2: x have only 1 chilren

        if (!T->pRight) {
            T = T->pLeft;
            delete i;
            i = nullptr;
            return;
        }

        if (!T->pLeft) {
            T = T->pRight;
            delete i;
            i = nullptr;
            return;
        }

        // Case 3: x have 2 chilrens

        TNODE* p = T->pLeft;
        TNODE* parentP = T;
        while (p->pRight) {
            parentP = p;
            p = p->pRight;
        }

        T->key = p->key;

        if (parentP == T) // Xóa node trái khi p là con trái
            deleteNode(p->key, T->pLeft);
        else
            deleteNode(p->key, parentP->pRight);
    }
}
```

3. B-Tree.

a. Giới thiệu cây B-Tree.

Cây B-Tree là một cây tìm kiếm nhị phân tự cân bằng, mỗi node của cây có thể có nhiều giá trị (key).

Cây B-tree bậc m thì bên trong mỗi node có tối đa $m - 1$ key.

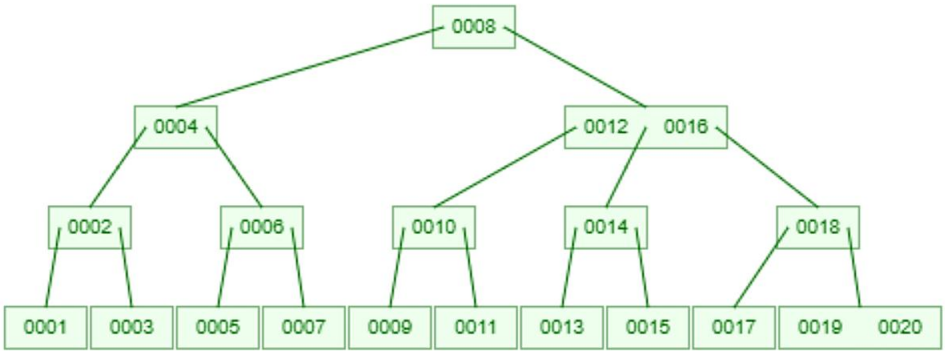
Cây B-Tree thỏa mãn các tính chất sau:

- Các key trong mỗi node được xếp theo thứ tự tăng dần.
- Các key trong node bên trái thì nhỏ hơn các key trong node bên phải.
- Các node lá thì có cùng độ sâu.
- Node có m – 1 key thì có m node con.

Tổng hợp:

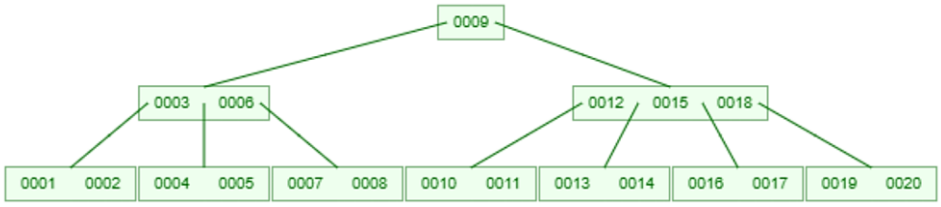
	Của root	Không của root
Số key	$\in [1;\ m-1]$	$\in \left[\left\lfloor\frac{m}{2}\right\rfloor;\ m-1\right]$
Số node con	$\in [1+1;\ m-1+1]$	$\in \left[\left\lfloor\frac{m}{2}\right\rfloor+1;\ m-1+1\right]$

VD1:



Một cây B-tree bậc 3 chứa các key từ 1 đến 20.

VD2:



Một cây B-tree bậc 5 chứa các key từ 1 đến 20.

b. Khởi tạo B-tree.

Kiểu dữ liệu B-tree:

```
struct node {
    vector<int> keys;
    vector<node*> children;
};

struct BTree {
    node* root;
    int m;
};
```

→ keys[i] sẽ chứa 2 node con là:

- **`children[i]`**: Chứa các key < keys[i].
- **`children[i+1]`**: Chứa các key > keys[i].

Khởi tạo cây B-tree chưa có key, có bậc là m. Nếu m chẵn thì đưa về lẻ.

```
void init(Btree tree, int m) {
    if (m % 2 == 0)
        m = m / 2 + 1;
    tree.m = m;
    tree.root = nullptr;
}
```

c. Tìm kiếm trên B-tree.

1. Tại một node, tìm key_i đầu tiên ≥ x. Xét keyi:
    a. Nếu key_i = x: X có tồn tại trong cây.
    b. Nếu node đang xét là lá: X không tồn tại trong cây.
    c. Do key_i key đầu tiên ≥ x, nên nếu key_i ≠ x và key_i không phải lá, thì kiểm tra cây con chilren[i].

```
bool search(node* root, int x) {
    if (root == nullptr)
        return false;

    int i = 0;
    int n = root->keys.size();

    while (i < n && x > root->keys[i])
        i++;

    if (i < n && x == root->keys[i])
        return true;

    if (root->children.empty())
        return false;

    return search(root->children[i], x);
}
```

Việc tìm kiếm một key trong node có thể được tiến hành bằng linear search hoặc binary search, nhưng việc quyết định node nào sẽ được tìm kiếm tiếp theo phải dựa trên phép so sánh key.

d. Chèn thêm key vào B-tree.

1. **Nếu cây rỗng**: Đưa x thành root của key.
2. **Tìm vị trí thích hợp để chèn**:
    a. Tại một node, tìm kiếm key đầu tiên ≥ x để định vị trí chèn.
    b. Nếu node đã là lá mà không tìm thấy vị trí chèn đảm bảo các tính chất thì tách node.

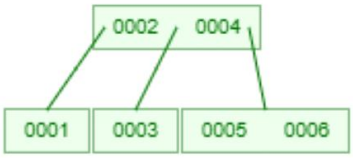
Tách node: Gọi mid là vị trí *trung vị*.

1. Tạm đưa x vào node hiện tại.
2. Đưa mid lên node cha.
3. Các key bên trái mid sẽ làm con trái mid, các key bên phải mid sẽ là con phải mid.

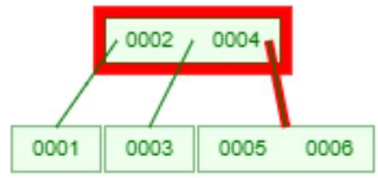
→ Cây B-tree phát triển theo chiều từ dưới lên thay vì từ trên xuống như cây tìm kiếm nhị phân.

VD:

Chèn key 7 vào cây sau:

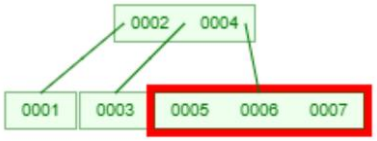


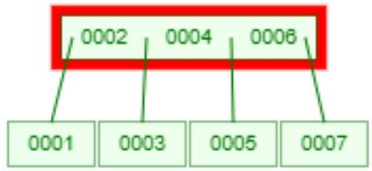
Tìm vị trí chèn phù hợp:



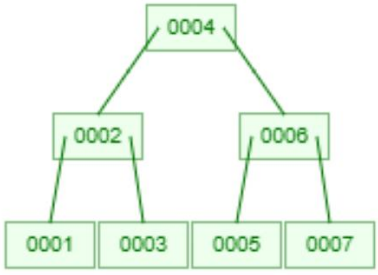


Không thể tìm ra vị trí hợp lệ, bắt đầu tách:





Sau khi tách xong vẫn không hợp lệ, tách lan truyền:

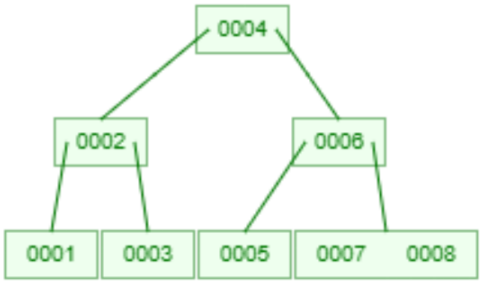


e. Xóa key.

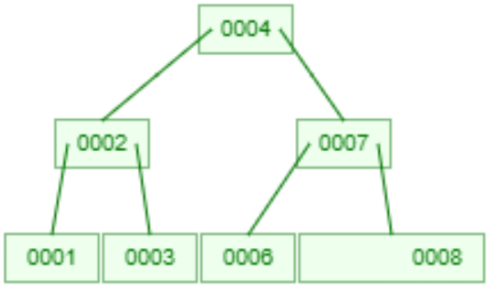
Khi xóa 1 key nào đó, có khả năng node đó sẽ thiếu key (|keys| < k). Lúc này, ta phải tìm cách bù thêm key vào node đã thiếu.

- **Trường hợp: Key cần xóa ở lá**. Tuần tự thực hiện 2 thao tác sau:
  - ***Underflow***:
    - Tìm một node adj *cùng cấp liền kề* sao cho $|keys_{adj}| > k$ (dư key). Nếu có, chọn key gần với key cần xóa nhất nằm ở node đó.
    - Tiến hành xoay cây:
      - Lấy key từ cha thế vào key cần xóa.
      - Đưa key đó lên cha đã bị lấy ở bước trên.
    - Nếu key cùng cấp liền kề gần nhất có con phải nhất / trái nhất (node chứa key này không phải lá) thì con đó cũng phải bọ xoay theo và gắn vào vị trí phù hợp.
  - ***Catenation***: Thực hiện bước này khi không thể underflow:
    - Gộp các thành sau thành 1 node duy nhất thế vào node chứa key bị xóa:
      - Node có key cần xóa.
      - Node trái gần nhất (hoặc node phải gần nhất).
      - Key nằm giữa 2 node kể trên (nằm ở cha).
    - Lúc này, do ta đã lấy key từ cha, cha có thể thiếu key. Ta bù key cho cha bằng cách tuần tự thực hiện lại các bước trên.
- **Trường hợp: Key cần xóa không ở lá**.
  - Thay thế k bằng key ở lá lớn nhất nhỏ hơn k hoặc nhỏ nhất lớn hơn k.
  - Quay về trường hợp key cần xóa ở lá.

VD: Cho cây:

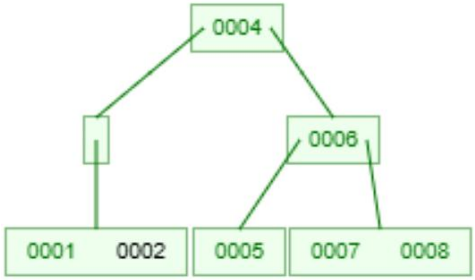


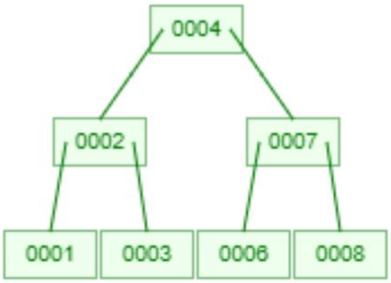
Xóa node 5.
Ta tìm thấy ở node (7; 8) có thừa 1 key. Do đó, ta lấy key gần nhất là 7 đưa lên 6, sau đó lấy 6 thế vào 5.



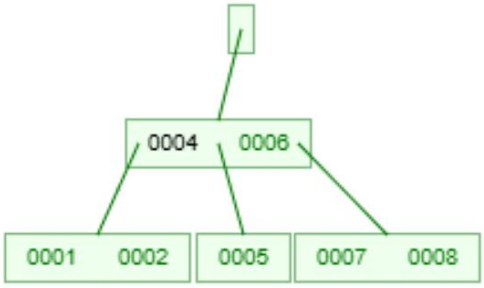
Hoàn tất xóa:

Xóa node 3.
Do không tìm thấy node cùng cấp liền kề nào có dư key. Ta gộp (1) với (2) lại thành một node chung.





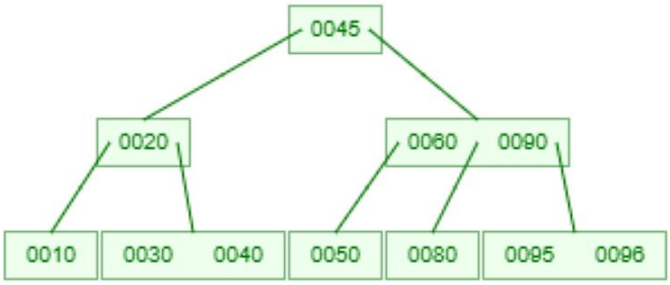
Lúc này, node cha đã bị rỗng. Ta phải tìm key để bù vào cha. Và cũng do không có node cùng cấp liền kề nào dư key, ta gộp (6) với (4) lại thành một node chung.



Do node cha rỗng và đồng thời là root, ta xóa luôn root này mà không cần bù gì cả.

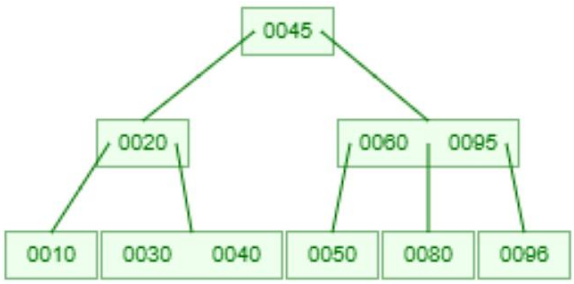


VD: Cho cây:



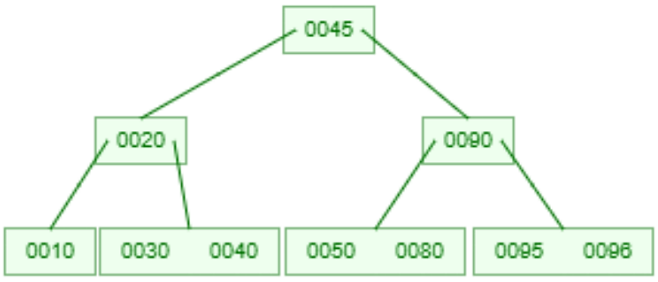
Xóa node 90.

Ta có thể thế 90 bằng 80 hoặc 95. Giả sử thế bằng 95, do node ở 95 dư khóa nên ta cứ thế mà xóa hẳn.



Xóa node 60.

Ta có thể thế 60 bằng 50 hoặc 80. Giả sử thế bằng 50. Khi đó ta xoay cây và thu được kết quả như dưới:



Phần C: CÁC THUẬT TOÁN TÌM KIẾM CHÍNH XÁC.

Bài toán: Cho danh sách A có n phần tử, hãy kiểm tra xem x có phải phần tử thuộc A không.

I. CÁC THUẬT TOÁN CÓ Ý TƯỞNG TÌM KIẾM TUẦN TỰ.

1. Thuật toán tìm kiếm tuyến tính (Linear search) và cải tiến, lý giải độ phức tạp tuyến tính.

Tìm kiếm tuyến tính là tìm kiếm theo tuần tự phần tử từ đầu đến cuối danh sách.

Độ phức tạp: $O(n)$.

a. Các phương pháp duyệt danh sách.

Sử dụng index: Danh sách có n phần tử được đánh số index từ 0 đến n-1.

- Mảng 1 chiều:

```
for (int i = 0; i < n; i++) {...}
```

- Mảng 2 chiều:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; i < 3; i++) {...}
}
```

Sử dụng con trỏ:

- Mảng 1 chiều:

```
int* p = &A[0];
for (int i = 0; i < 5; i++) {
    cout << *(p + i) << " ";
}
```

- Mảng 2 chiều:

```
int* p = &A[0][0];
for (int i = 0; i < 9; i++) {
    cout << p[i] << " ";
}
```

Sử dụng iterator: Iterator là một con trỏ chuyên dùng để thao tác với các danh sách động.

Giá trị của iterator gắn với danh sách bằng một số hàm sau đây:

A.begin() **A**	Trỏ đến index 0 của A. Tên của danh sách A được coi như A.begin().
A.end()	Trỏ đến index n của A.
A.rbegin()	Trỏ đến index n – 1 của A.
A.rend()	Trỏ đến index – 1 của A.
A.begin() + i	Trỏ đến index i của A.

Do là con trỏ, sử dụng toán tử * phía trước iterator để lấy giá trị ở địa chỉ được trỏ.

Tuy độ phức tạp 2 thuật toán đều tuyến tính $O(n)$, nhưng số bước thực hiện thuật toán của 2 ý tưởng có sự khác biệt:

Số phép	**Linear search**	**Sentinel linear search**	**Xấp xỉ**
Gán	n	2 + n + 1	n
So sánh	2n + 1	n + 2	n
Tổng cộng	3n + 1	2n + 5	n

b. Thuật toán tìm kiếm tuyến tính thông thường.

```
int linearSearch(int n, int x) {
    for (int i = 0; i < n; i++) {
        if (A[i] == x) {
            return i;
        }
    }
    return -1;
}
```

c. Thuật toán tìm kiếm tuyến tính cải tiến (Sentinel linear search).

```
int sentinel_LinearSearch(int n, int x) {
    int last_element = A[n-1];
    A[n-1] = x;

    int i = 0;
    while (A[i] != x)       // finds x
        i++;

    A[n-1] = last_element;

    if (i < n-1             // x is founded before n-1
        || A[n-1] == x)     // last_element equal to x
        return i;
    return -1;
}
```

II. CÁC THUẬT TOÁN CÓ Ý TƯỞNG CHIA ĐỂ TRỊ.

1. Thuật toán tìm kiếm nhị phân (Binary search), lý giải độ phức tạp logarithm.

Tìm kiếm nhị phân là thuật toán xây dựng dựa trên phương pháp chia để trị (Divide and Conquer).

Điều kiện áp dụng thuật toán: Danh sách đã được sắp xếp theo thứ tự không tăng hoặc không giảm.

Độ phức tạp: $O(\log n)$.

Các hàm có sẵn trong C++:

Library	TC	Cú pháp	Ý nghĩa
`algorithm`	$O(\log n)$	`binary_search(a, b, x)`	Hàm trả về true nếu x có trong phạm vi bắt đầu tự vị trí mà a trỏ đến liền trước b trỏ. Điều kiện: Danh sách đã được xếp không giảm.
`algorithm`	$O(\log n)$	`lower_bound(a, b, x)`	- Hàm trả về con trỏ trỏ đến ***phần tử đầu tiên nhỏ nhất lớn hơn x*** trong phạm vi bắt đầu tự vị trí mà a trỏ đến liền trước b trỏ. - Nếu không tìm thấy phần tử thỏa mãn, trỏ đến ***phần tử đầu tiên bằng x***. - Nếu không tìm thấy x, trả về ***NULL***. Điều kiện: Danh sách đã được xếp không giảm.
`algorithm`	$O(\log n)$	`upper_bound(a, b, x)`	- Hàm trả về con trỏ trỏ đến ***phần tử đầu tiên lớn nhất nhỏ hơn x*** trong phạm vi bắt đầu tự vị trí mà a trỏ đến liền trước b trỏ. - Nếu không tìm thấy phần tử thỏa mãn, trỏ đến ***phần tử đầu tiên bằng x***. - Nếu không tìm thấy x, trả về ***NULL***. Điều kiện: Danh sách đã được xếp không giảm.
`algorithm`	$O(\log n)$	`equal_range(a, b, x)`	- Hàm trả về một pair chứa 2 con trỏ là first và last lần lượt trỏ đến ***phạm vi của dãy con bằng x*** trong phạm vi bắt đầu tự vị trí mà a trỏ đến liền trước b trỏ. - Nếu không tìm thấy, ***first = last***. Điều kiện: Danh sách đã được xếp không giảm.

a. Ý tưởng sơ khai.

Giả thiết: Danh sách đã được sắp xếp không giảm.

Mô tả:

- Thuật toán tìm kiếm một phần tử cụ thể bằng cách so sánh phần tử tại vị trí giữa nhất của tập dữ liệu. Nếu tìm thấy x thì chỉ mục của phần tử được trả về.
- Nếu phần tử cần tìm là lớn hơn giá trị phần tử giữa thì phần tử cần tìm được tìm trong mảng con nằm ở bên phải phần tử giữa, nếu không thì sẽ tìm ở trong mảng con nằm ở bên trái phần tử giữa.
- Tiến trình sẽ tiếp tục như vậy trên mảng con cho tới khi tìm hết mọi phần tử trên mảng con này.

Số bước thực hiện:

- Ở lần lặp thứ nhất, xét trên đoạn dài $\frac{n}{2}$.
- Ở lần lặp thứ hai, xét trên đoạn dài $\frac{n}{2^2}$.
- ...
- Ở lần lặp thứ k, xét trên đoạn dài $\frac{n}{2^k}$.
- Số lần thực hiện thuật toán là:

$$\frac{n}{2^k} = 1 \Leftrightarrow n = 2^k \Leftrightarrow k = \log_2 n = \log n$$

Trong Tin học, $\log n = \log_2 n$, không phải cơ số 10.

Gọi:

- l, r: Là index trái cùng và phải cùng của phạm vi tìm kiếm. Ban đầu thì l = 0 và r = n-1, tức là tìm kiếm trên phạm vi toàn bộ danh sách.
- mid: Vị trí giữa l và r, coi như mid = (l + r) / 2.

Chú ý: Trong một số trường hợp, (l + r) / 2 có thể tràn số. Khi đó, ta có thể thay bằng:

$$l + \frac{r - l}{2}$$

Các bước:

1. Nếu l > r, tức là lỗi phạm vi, ta không cần xét thêm nữa, thuật toán kết thúc.
2. Tính mid = (l + r) / 2.
3. Xét A[mid]:
   a) Nếu x > A[mid], tức là x thuộc khoảng (mid, r]. Quay về bước đầu với l = mid + 1.
   b) Nếu x < A[mid], tức là x thuộc khoảng [r, mid). Quay về bước đầu với r = mid - 1.
   c) Nếu x = A[mid], tức là đã xác định x thuộc A, kết thúc thuật toán.

Code mẫu: Hàm trả về true nếu x thuộc A:

- Code có đệ quy:

```
int binarySearch(int l, int r, int x){
    if(l > r)
        return -1;
    int mid = (l + r) / 2;
    if(x > A[mid])
        return binSearch(mid+1, r, x);
    else if(x < A[mid])
       return binSearch(l, mid-1, x);
    else
        return mid;
}
```

- Code khử đệ quy:

```
bool binarySearch(int l, int r, int x){
    while (l <= r) {
        int mid = (l + r) / 2;
        if (x > A[mid])
            l = mid + 1;
        else if (x < A[mid])
            r = mid - 1;
        else
            return true;
    }
    return false;
}
```

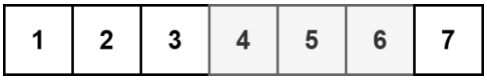
b. Lower-bound và Upper-bound.

Bài toán: Tìm phần tử nhỏ hơn x lớn nhất, x không nhất thiết phải nằm trong danh sách.

Giả sử có một danh sách như sau:



Khi x = 5, đây là sẽ là các mid:



Đáp án của bài toán trên là 4.

Nhận xét:

- Đáp án là một trong số các mid nằm trước mid kết quả.
- Khi cập nhật right để đổi cận thì đồng thời cập nhật đáp số là mid.

Code mẫu:

```
int lowerBound(int *a, int n, int x) {
    int l = 0, r = n - 1, res = l;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (x >= a[mid]) {
            res = mid;
            l = mid + 1;
        }
        else
            r = mid - 1;
    }
    return res;
}
```

Tương tự với bài toán tìm phần tử lớn hơn x nhỏ nhất, x không nhất thiết phải nằm trong danh sách.

```
int upperBound(int *a, int n, int x) {
    int l = 0, r = n - 1, res = r;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (x <= a[mid]) {
            res = mid;
            r = mid - 1;
        }
        else
            l = mid + 1;
    }
    return res;
}
```

Các hàm trên đều giả sử trong mảng không có các phần tử bị trùng.

c. Equal range.

Equal range là một đoạn của danh sách sao cho tất cả các phần tử trên đoạn đó đều bằng với x.

Code mẫu: Hàm gán lại cho l và r là phạm vi của equal range (Nếu l > r thì không có equal range):

```
pair<int, int> equalRange(const vector<int>& A, int x) {
    int l = 0, r = A.size() - 1;
    int left = -1, right = -1;

    while (l <= r) {
        int mid = (r + l) / 2;

        if (A[mid] > x) {
            r = mid - 1;

        } else if (A[mid] < x) {
            l = mid + 1;

        } else {
            left = mid;

            while (left > 0 && A[left - 1] == x) {
                left--;
            }
            right = mid;

            while (right < A.size() - 1 && A[right + 1] == x) {
                right++;
            }

            return {left, right};
        }
    }

    return {-1, -1};
}
```

Một cách code khác đơn giản hơn, nhưng sẽ chậm hơn một chút:

```
pair<int, int> equalRange(const vector<int>& nums, int target) {
    auto lower = lower_bound(nums.begin(), nums.end(), target);
    auto upper = upper_bound(nums.begin(), nums.end(), target);

    if (lower == nums.end() || *lower != target)
        return {-1, -1};

    return {distance(nums.begin(), lower),
            distance(nums.begin(), upper) - 1};
}
```

d. Rotated array.

Cho một danh sách nums có n phần tử. nums được chia thành 2 phần:

- Phần đầu từ 0 đến k.
- Phần sau từ k + 1 đến n – 1 (0 < k < n).

Cả 2 phần đều đã được sắp xếp tăng dần và mọi phần tử của phần đầu đều lớn hơn mọi phần tử của phần cuối.

Nums được gọi mảng xoay.

VD:

4 5 6 7 0 1 2

Nhận xét về mảng xoay:

- Bài toán cung cấp mảng gần như đã được sắp xếp sẵn và yêu cầu tìm kiếm phần tử nằm trong mảng đó, ta nghĩ ngay đến binary search. Tuy nhiên, bài này không thể dùng trực tiếp binary search được vì mảng không hoàn toàn được sắp xếp, ta cần có chút biến tấu.
- Do tính chất "*các phần tử phân biệt*" và "*cả 2 đoạn đều đã được sắp xếp tăng dần*", nên nếu l < r và nums[l] < nums[r] thì trong đoạn [l; r], mảng đã được sắp xếp tăng dần.
- Phần đầu luôn lớn hơn phần sau.

2. Thuật toán tìm kiếm nội suy (Interbolation search).

[*Đọc thêm*]

a. Nội suy (Interbolation) và Nội suy tuyến tính (Linear interbolation).

Nội suy là ước lượng các giá trị chưa biết trong một dãy dữ liệu dựa trên các giá trị đã biết.

Nội suy tuyến tính (nội suy 1 chiều) là phép nội suy áp dụng cho dãy dữ liệu tuyến tính (không tăng hoặc không gian).

VD. Ta có một hàm ẩn y = f(x) và được cung cấp n = 3 cặp giá trị x và y tương ứng sau đây:

x	y
1	10
2	20
3	30
x_i	y_i

Giả sử cho một giá trị x_i, hỏi y_i có thể là bao nhiêu?

Nội suy tuyến tính hướng đến việc chọn 2 điểm A(x_1, y_1) và B(x_2, y_2) tùy ý. Do tập dữ liệu tuyến tính, những điểm dữ liệu đó có thể nằm trên một hàm số tuyến tính, tức là một đường thẳng đi qua A và B, đường thẳng đó có dạng tổng quát:

$$y = ax + b$$

Nhờ vào 2 điểm A, B có thể suy ra a và b:

$$\begin{cases} y_1 = ax_1 + b \\ y_2 = ax_2 + b \end{cases} \Leftrightarrow \begin{cases} b = y_1 - ax_1 \\ y_2 = ax_2 + y_1 - ax_1 \Leftrightarrow y_2 - y_1 = a(x_2 - x_1) \Leftrightarrow a = \dfrac{y_2 - y_1}{x_2 - x_1} \end{cases}$$

Phương trình trở thành:

$$y = \frac{y_2 - y_1}{x_2 - x_1}x + y_1 - \frac{y_2 - y_1}{x_2 - x_1}x_1 = y_1 + (x - x_1)\frac{y_2 - y_1}{x_2 - x_1}$$

Như vậy, ta thu được hàm số y theo x, công thức này được gọi là công thức nội suy tuyến tính, với y được gọi là giá trị cần nội suy.

$$f(x) = y = y_1 + (x - x_1)\frac{y_2 - y_1}{x_2 - x_1}$$

Công thức trên đặc biệt chính xác đối với tập dữ liệu phân bố đều.

b.Thuật toán tìm kiếm nội suy.

Tìm kiếm nội suy là sự cải tiến của tìm kiếm nhị phân nhờ việc ứng dụng công thức nội suy tuyến tính.

Điều kiện áp dụng thuật toán: Danh sách đã được sắp xếp theo thứ tự không tăng hoặc không giảm.

Độ phức tạp: O(log(logn)).

Mô tả:

- Cũng tương tự tìm kiếm nhị phân nhưng thay vì dùng mid = (l + r) / 2, ta dùng vị trí dò (Probe position) với công thức như sau:

$$Probe\ position = mid\ = l + (x - A[l])\ \frac{r - l}{A[r] - A[l]}$$

- Công thức trên là một công thức suy ra từ công thức nội suy tuyến tính, với mid là giá trị cần nội suy.
- Thuật toán này đặc biệt chính xác nếu giá trị các phần tử của danh sách phân bố đều. Nếu danh sách phân bố quá lệch, ***thuật toán có thể thậm hơn tìm kiếm nhị phân***.

Phần D: THUẬT TOÁN SẮP XẾP.

I. GIỚI THIỆU BÀI TOÁN SẮP XẾP VÀ CÁC THUẬT TOÁN SẮP XẾP.

1. Giới thiệu bài toán sắp xếp.

Bài toán: Cho danh sách A có n phần tử, hãy sắp xếp các phần tử của A theo thứ tự không tăng hoặc không giảm (coi như tăng dần).

2. Stable sort và Online sort.

Phân loại các thuật toán dựa trên kết quả sắp xếp: **Stable sort**:

- Một thuật toán được gọi là stable nếu sau khi hoàn thành thuật toán, các phần tử có giá trị bằng nhau trong danh sách vẫn giữ nguyên vị trí tương ứng so với trước khi sắp xếp.

Phân loại các thuật toán dựa trên phương pháp sắp xếp: **Online sort**:

- Một thuật toán được gọi là online nếu việc so sánh được thực hiện tuần tự theo chiều dài mảng. Có nghĩa là, một mảng có thể vừa được sắp xếp vừa được bổ sung dữ liệu.

	Buble	Select	Insert	Shell	Heap	Merge	Quick	Count	Radix
Stable									
Online									

Trong số các thuật toán quan trọng thì:

- Chỉ có insertion sort, merge sort là ***stable***.
- Chỉ có insertion sort là ***online***.

II. CÁC THUẬT TOÁN XỬ LÝ THÔ.

1. Bubble sort.

[*Đọc thêm*]

a. Ý tưởng ban đầu.

Độ phức tạp: $O(n^2)$.

Mô tả:

- Ở bước sắp xếp 1, ta đưa phần tử lớn nhất tới cuối danh sách bằng cách so sánh các cặp phần tử liên kề từ đầu đến cuối.
- Ở bước sắp xếp 2, ta đưa phần tử lớn thứ 2 tới áp chót danh sách bằng cách so sánh các cặp phần tử liên kề từ đầu đến vị trí áp chót.
- …
- Như vậy, ta cần thực hiện n - 1 bước sắp xếp như thế để đưa lần lượt các phần tử lớn nhất, lớn thứ 2, thứ 3,… vào các vị trị tương ứng.

Các bước:

1. Cho i chạy từ n-1 đến 1 (Bước sắp xếp):
   a) Cho j chạy từ 0 đến i-1:
      i. Nếu A[j] > A[j+1] thì hoán đổi vị trí A[j] và A[j+1].

Code mẫu:

```
void bubbleSort(int a[], int n){
    for (int i = n-1; i >= 1; i--){
            for (int j = 0; j < i; j++){
                    if (a[j] > a[j+1])
                            swap(a[j], a[j+1]);
            }
    }
}
```

b. Ý tưởng cải tiến.

Chúng ta thấy rằng giải thuật sắp xếp nổi bọt so sánh mỗi cặp phần tử trong mảng trừ khi cả toàn bộ mảng đó đã hoàn toàn được sắp xếp theo thứ tự tăng dần. Để tránh việc này xảy ra, chúng ta có thể sử dụng một biến check chẳng hạn để giúp chúng ta biết có cần thực hiện thao tác tráo đổi thứ tự hay không. Nếu không cần thiết thì thoát khỏi vòng lặp.

Ở mỗi i, swapped = false. Nếu trong trình so sánh mà có hoán đổi, swapped = true. Nếu đã kết thúc toàn bộ quá trình so sánh mà swapped vẫn bằng false thì tức là thuật toán không hoán đổi bất cứ cặp phần tử nào, cũng có nghĩa là dãy đã được sắp xếp, kết thúc thuật toán.

Code mẫu:

```
void bubbleSort(int a[], int n){
    bool swapped = false;
    for (int i = n-1; i >= 1; i--){
        for (int j = 0; j < i; j++){
            if (a[j] > a[j+1]) {
                swap(a[j], a[j+1]);
                swapped = true;
            }
        }
        if (swapped == false)
            break;
    }
}
```

2. Selection sort.

Độ phức tạp: $O(n^2)$.

Mô tả:

- Ta chia danh sách thành 2 phần, phần đầu là các phần tử đã được sắp xếp (sorted), phần sau là các phần tử chưa được sắp xếp (unsorted). Xét phần tử thứ i, phần trước i là phần đầu, sau i là phần sau.
- Xét unsorted và đưa phần tử nhỏ nhất của unsorted lên đầu unsorted.
    - Khi unsorted = mảng đang xét, phần tử nhỏ nhất của unsorted là phần tử nhỏ nhất của mảng đang xét.
    - Khi unsorted = mảng đang xét từ index 1 đến n – 1, phần tử nhỏ nhất của unsorted là phần tử nhỏ thứ hai của mảng đang xét.
    - …
    - Như vậy, khi mảng unsorted càng thu nhỏ về sau, sorted càng được mở rộng.

Các bước:

1. Cho i chạy từ 0 đến n-1:
    a) Tìm vị trí của phần tử nhỏ nhất trong đoạn từ i đến n - 1 và gán nó cho indexOfMin.
    b) Hoán đổi A[i] và A[indexOfMin].

Số phép tính thực hiện trong tất cả các input đều là như nhau:

- Thuật toán tìm min: Tốn n bước.
- Ở bước i, số phép so sánh là $n - i$.
- Tổng số phép so sánh: $1 + 2 + 3 + \cdots + n - 1 = \frac{n.(n-1)}{2} \approx n^2$.

Code mẫu:

```
for (int i = 0; i < n-1; i++) {
    int indexOfMin = i;
        for (int j = i + 1; j < n; j++) {
            if (A[j] < A[indexOfMin]) {
                indexOfMin = j;
            }
        }
    swap(A[i], A[indexOfMin]);
}
```

3. Insertion sort.

Độ phức tạp: $O(n^2)$.

Mô tả:

- Ta chia danh sách thành 2 phần, phần đầu là các phần tử đã được sắp xếp (sorted), phần sau là các phần tử chưa được sắp xếp (unsorted). Xét phần tử thứ i, phần trước i là phần đầu, sau i là phần sau.
- Cùng ý tưởng với Selection sort, nhưng ta đưa phần tử đầu tiên của unsorted (phần tử thứ i) vào sorted sao cho giữ nguyên tính chất sắp xếp của sorted. Có 2 cách thực hiện:
    - Vừa so sánh vừa dịch:
        - So sánh 2 cặp A[index-1] và A[index], nếu như A[index-1] > A[index] thì dịch A[index-1] sang phải một đơn vị.
    - Dùng Binary search (Binary insertion sort):
        - Do sorted có tính tăng dần, ta dùng Binary search tìm ra vị trí k là vị trí cần chèn.
        - Lần lượt dịch các phần tử của sorted trước k qua phải một đơn vị.

Số bước thực hiện:

- Đối với dãy đã tăng dần: Cần $n - 1 \approx n$ lần so sánh.
- Đối với dãy đã giảm dần: Cần $1 + 2 + \cdots + n - 1 = \frac{n.(n-1)}{2} \approx n^2$ lần so sánh.

Code mẫu:

```
for (int i = 1; i < n; i++) {
    int index = i;
    while (index-1 >= 0 && a[index-1] > a[index]) {
        swap(a[index-1], a[index]);
        index--;
    }
}
```

Sử dụng hàm swap có thể tốn thêm thời gian, tốt nhất nên khử swap:

```
void insertionSort(int *a, int n) {
    for (int i = 1; i < n; i++) {
        int selected = a[i];
        int index = i;
        while (index > 0 && a[index - 1] > key) {
            a[index] = a[index - 1];
            index--;
        }
        a[index] = selected;
    }
}
```

Cải tiến bằng cách dùng Binary search:

```
int binarySearch(int *a, int l, int r, int x) {
    int res = l;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (x > a[mid]) {
            res = mid;
            l = mid + 1;
        }
        else
            r = mid - 1;
    }
    return res + 1;
}



void binaryInsertionSort(int *a, int n) {
    for (int i = 1; i < n; i++) {
        int selected = a[i];
        int pos = binarySearch(a, 0, i, selected);
        for (int j = i; j > pos; j--) {
            a[j] = a[j - 1];
        }

        a[pos] = selected;
    }
}
```

III. CÁC THUẬT TOÁN CÓ Ý TƯỞNG CHIA ĐỂ TRỊ.

1. Merge sort.

Độ phức tạp: $O(n \log n)$.

a. Bài toán hợp nhất danh sách theo thứ tự.

Mấu chốt của thuật toán này nằm ở bài toán sau:

Bài toán: Cho 1 mảng từ l đến r, mảng này được chia thành hai nửa. Nửa từ l đến mid, và từ mid + 1 đến r, cả 2 nửa đã được sắp xếp theo thứ tự. Hãy sắp xếp lại toàn bộ mảng theo thứ tự.

Giải quyết:

- Có thể dùng các hàm sort, nhưng độ phức tạp có thể lên đến n^2.
- **Nhận xét**: Do 2 nửa đã được sắp xếp tăng dần, ta chỉ cần dùng 2 con trỏ trỏ đến 2 nửa, lần lượt ghép từng phần tử vào một mảng mới, sau đó lấy mảng mới này gán lại cho mảng ban đầu.
- Phương pháp trên tốn $O(n)$ thời gian và không gian (***cần mảng phụ***).

```
void myMerge(int *a, int l, int mid, int r) {
    int i = l, j = mid + 1;

    vector<int> temp;

    while(i <= mid && j <= r) {
        if(a[i] <= a[j]) {
            temp.push_back(a[i]);
            i++;
        } else {
            temp.push_back(a[j]);
            j++;
        }
    }

    while(i <= mid) {
        temp.push_back(a[i]);
        i++;
    }

    while(j <= r) {
        temp.push_back(a[j]);
        j++;
    }

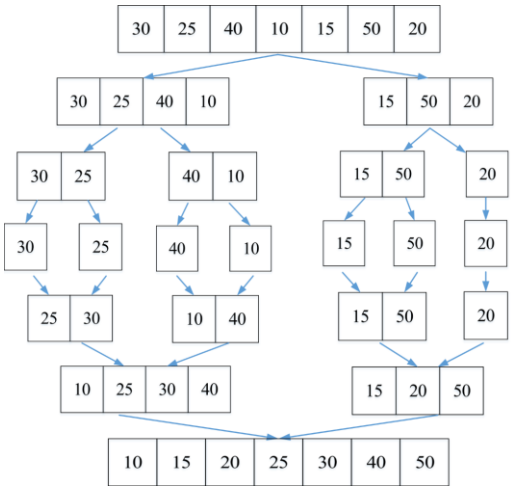
    for(int k = l; k <= r; k++) {
        a[k] = temp[k];
    }
}
```

Có 2 cách tiếp cận thuật toán merge sort:

b. Cách tiếp cận từ lớn đến nhỏ (top-down).

- Bước 1: **Devide**: Ngầm chia mảng thành hai mảng con cho đến khi không thể chia được nữa (mỗi mảng con có kích thước là một).
- Bước 2: **Merge**: Ghép các mảng con lại.

```
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int mid = (l + r) / 2;
        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, r);
        merge(arr, l, mid, r);
    }
}
```



c. Cách tiếp cận từ nhỏ đến lớn (khử đệ quy) (bottom-up).

- Ban đầu, ngầm chia mỗi phần tử của mảng.
- Sau đó, lần lượt ghép hai phần tử / nhóm phần tử liên tiếp với nhau lại:
- Gọi:
  - **`width`**: Chiều dài 1 mảng con đang xét:

$$width \in \{[1; n)|2k\}$$

  - **`l`**: Index trỏ đến phần tử đầu tiên của mảng con đang xét.

$$l \in \{[0; n - width)|2.width\}$$

  - **`r`**: Index trỏ đến phần tử cuối cùng của 2 mảng con đang xét.

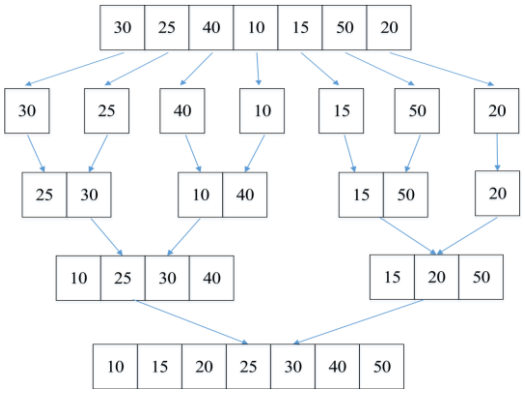
$$r = \min(n - 1;\ \ l + 2.width - 1)$$

  - **`mid`**: Index trỏ đến trung vị.

$$mid = \left\lfloor \frac{l + r}{2} \right\rfloor = l + width - 1$$

Biến	Ví dụ						Min	Max	Bước nhảy
width	=	1	2	4	8	...	1	n	width *= 2
l	=	0	0	0	0	...	0	n – width	l += 2 * width
	=	2	4	8	16	...			
	=	4	8	16	24	...			
	=	...	...	...	...	...			
r	=	1	3			...	~	n – 1	r = min(l + 2 * width – 1, n – 1)
	=	3	7			...			
	=	5	11			...			
	=	...	...	...	...	...			

```
void mergeSort(int arr[], int n) {
    for (int width = 1; width < n; width *= 2) {
        for (int l = 0; l < n - width; l += 2 * width) {
            int mid = l + width - 1;
            int r = min(l + 2 * width - 1, n - 1);
            merge(arr, l, mid, r);
        }
    }
}
```



2. Quick sort.

Độ phức tạp: $O(n.\log n)$.

a. Cải tiến bài toán hợp nhất danh sách theo thứ tự.

Mô tả:

- Quick sort là một cải tiến của merge sort.
- Cải tiến nằm ở cách merge:
    - Thuật toán merge sort sẽ merge 2 danh sách con không có thứ tự về một danh sách lớn có thứ tự với chi phí là $O(n)$.
    - Tuy nhiên, nếu 2 danh sách con đó đã có thứ tự sẵn, ta chỉ cần ghép chúng lại thì sẽ tạo ra danh sách lớn hơn có thứ tự.
    - Gọi phần tử trung vị trong danh sách lớn là pivot, các phần tử phía trước pivot phải < pivot, và các phần tử ở phía sau pivot phải ≥ pivot. Việc đưa các phần tử trong danh sách lớn theo thứ tự mô tả trên được gọi là phân hoạch (partition).

b. Các phương pháp phân hoạch.

Có 2 phương pháp phân hoạch:

- ***Thuật toán Hoare***:
  - Chọn pivot là phần tử ***bất kỳ*** trong danh sách, thường là trung vị.
  - Tìm phần tử đầu tiên bên trái **≥** pivot (không thể tìm phần tử > pivot vì có khả năng con trỏ trỏ đến nó sẽ tràn ra khỏi danh sách).
  - Tìm phần tử đầu tiên bên phải **≤** pivot (không thể tìm phần tử < pivot vì có khả năng con trỏ trỏ đến nó sẽ tràn ra khỏi danh sách).
  - Nếu phần tử bên trái ***nằm trước*** phần tử bên phải thì hoán đổi chúng.

```
void partitionHoare(int arr[], int left, int right) {
    int pivot = arr[left];
    int i = left, j = right;

    while (i <= j) {
        while (arr[i] < pivot) i++;
        while (arr[j] > pivot) j--;

        if (i <= j) {
            swap(arr[i], arr[j]);
            i++; j--;
        }
    }
}
```

  - Kết thúc thuật toán, j luôn trỏ đến vị trí cuối cùng của danh sách con trước pivot; i luôn trỏ đến vị trí đầu tiên của danh sách con sau pivot (i > j).

3	**5**	**2**	**1**	**4**	**6**
	i	j			
1	**5**	**2**	**3**	**4**	**6**
	j	i			

Trường hợp mảng chưa theo thứ tự.

1	2	3	4	5	6
j				i	

Trường hợp mảng đã theo thứ tự.

- ***Luật toán Lomuto***: [*Đọc thêm*]
    - Chọn pivot là phần tử nằm sau cùng danh sách.
    - Duyệt mảng từ đầu về cuối, Nếu gặp phần tử nhỏ hơn pivot, hoán đổi nó với phần tử đứng trước.
    - Đưa pivot về đúng vị trí bằng cách đổi chỗ với phần tử đầu tiên của vùng lớn hơn

```
int partitionLomuto(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            swap(arr[i], arr[j]);
            i++;
        }
    }
    swap(arr[i], arr[high]);
    return i;                 // Trả về index của pivot sau khi phân hoạch
}
```

    - Do phải hoán đổi nhiều lần, độ phức tạp cao, thuật toán này không được ưa chuộng.

c. Tổng thể thuật toán quick sort.

Code mẫu theo phương pháp Hoare:

```
void QuickSort(int a[], int left, int right) {
    int i, j, pivot;
    pivot = a[(left + right) / 2];
    i = left; j = right;
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            Swap(a[i], a[j]);
            i++; j--;
        }
    }
    if (left < j) QuickSort(a, left, j);   // Gọi ds con từ left đến j
    if (i < right) QuickSort(a, i, right); // Gọi ds con từ i đến right
}
```

3. Heap sort.

Heap sort là phương pháp sử dụng đặc trưng của cây heap để sắp xếp.

a. Danh sách có tính heap.

Một danh sách có ***tính heap*** là khi:

- Thứ tự các phần tử trong danh sách là kết quả khi duyệt cây theo chiều rộng. Cụ thể, với mỗi phần tử ***i*** sẽ có 2 con là ***2i + 1*** và ***2i + 2*** (i *liền đới* với 2i + 1 và 2i + 2) và *2i + 1, 2i + 2 < n*:
  - Max heap: A[i] > A[2i + 1], A[i] > A[2i + 2].
  - Min heap: A[i] < A[2i + 1], A[i] < A[2i + 2].
- Thứ tự chuyển đổi từ danh sách sang cây heap: *Từ trên xuống, từ trái sang*.

Danh sách có tính heap *Cây heap*

Bài toán: ***Đưa một dãy từ không heap về heap (Hiệu chỉnh – Heapify)***:

- Hiệu chỉnh tại vị trí i là quá trình chọn và hoán đổi i và các con của nó sao cho thỏa cây heap.
- Trong cây heap, ta cần xét cặp cha – con nên cần xét tất cả những node có con. Biết node có con cuối cùng (từ trên xuống, từ trái sang) có index là:

$$\left\lfloor \frac{n}{2} \right\rfloor - 1$$

- Lần lượt xét các node có con từ dưới lên, từ phải sang. Xét cặp cha – con:
  - ***Đưa về max heap***: Nếu |cha| < |con| thì: Hoán đổi cha và con.
  - ***Đưa về min heap***: Nếu |cha| > |con| thì: Hoán đổi cha và con.
  - Hiệu chỉnh tại node con sau bị hoán đổi (nếu bị hoán đổi).

```
void Heapify(int a[], int n, int i) {
    int childLeft = i*2+1;
    int childRight = i*2+2;
    int max = i;

    if(childLeft < n && a[max] < a[childLeft])
        max = childLeft;
    if(childRight < n && a[max] < a[childRight])
        max = childRight;

    if(max != i) {
        swap(a[max], a[i]);
        Heapify(a, n, max);
    }
}

void buildHeap(int a[], int n) {
    for(int i = n/2-1; i >= 0; i--)
        Heapify(a, n, i);
}
```

Độ phức tạp:

- ***buildHeap***: $O(n)$ (luôn luôn).
- ***Heapify***: $O(\log n)$ (trung bình) hoặc $O(1)$ (tốt nhất, khi danh sách đã có tính heap).

→ $O(n)$ (tốt nhất) hoặc $O(n.\log n)$ (trung bình).

b. Heap sort.

Thuật toán Heap sort có ý tưởng tương tự như Selection sort, đó là ta lần lượt đưa các phần tử lớn nhất, lớn thứ hai,… về cuối danh sách unsorted. Tuy nhiên, sự khác nhau nằm ở cơ chế tìm và hoán đổi phần tử lớn nhất với phần tử cuối cùng của unsorted:

- Trong thuật toán Selection sort, cần $O(n)$ thời gian để tìm và hoán đổi vị trí phần tử lớn nhất về cuối unsorted.
- Trong thuật toán Heap sort:
    - Giả sử unsorted đã có tính heap, như vậy phần tử đầu tiên luôn là lớn nhất → Trực tiếp hoán đổi phần tử lớn nhất với phần tử cuối cùng của sorted.
    - Sau khi hoán đổi phần tử lớn nhất với phần tử cuối cùng của unsorted, unsorted đã không bao gồm phần tử lớn nhất ấy. Khi đó, unsorted không còn tính heap.
    - Để đưa unsorted về tính heap ở vị trí ta đã hoán đổi, ta cần hiệu chỉnh tại vị trí đó, tốn $O(\log n)$. Sau khi hiệu chỉnh, quay lại bước ban đầu.

Độ phức tạp: $O(n.\log_2 n)$.

Code mẫu:

```
void HeapSort(int a[], int n) {
    buildHeap(a, n);

    for(int i = n - 1; i > 0; i--) {
        swap(a[0], a[i]);
        Heapify(a, i, 0);
    }
}
```

V. CÁC THUẬT TOÁN KHÔNG CÓ Ý TƯỞNG SO SÁNH.

[*Đọc thêm*]

1. Counting sort.

Độ phức tạp: $O(\max A - \min A + 1) \approx O(n)$.

Counting Sort là một thuật toán sắp xếp không dựa trên so sánh, hoạt động bằng cách đếm số lần xuất hiện của từng phần tử trong tập dữ liệu và sử dụng thông tin đó để sắp xếp mảng.

Mô tả:

- Counting sort là một thuật toán sắp xếp không dựa trên so sánh, hoạt động bằng cách đếm số lần xuất hiện của từng phần tử trong tập dữ liệu và sử dụng thông tin đó để sắp xếp mảng.

Các bước thực hiện:

- Tìm giá trị nhỏ nhất và lớn nhất trong mảng để xác định phạm vi giá trị cần đếm (minVal và maxVal).
- Tạo ra một mảng count có kích thước (minVal + maxVal). count[i] chính là số lần xuất hiện của i trong A. Ban đầu, count[i] = 0. Lần lượt duyệt qua từng phần tử của mảng ban đầu, count[A[i]]++.
- Gọi mảng đã được sắp xếp là sorted. Lần lượt duyệt từ minVal đến maxVal (hoặc ngược lại) và đưa vào mảng sorted count[i] phần tử i.

Nhược điểm:

- Thuật toán không hiệu quả nếu như maxVal – minVal quá lớn, tốn một lượng bộ nhớ lớn để lưu trữ.

Code mẫu:

```
void countingSort(vector<int>& A) {
    int minVal = *min_element(A.begin(), A.end());
    int maxVal = *max_element(A.begin(), A.end());
    vector<int> count(maxVal - minVal + 1, 0);

    for (int num : A) {
        count[num]++;
    }

    int index = 0;
    for (int i = minVal; i <= maxVal; i++) {
        while (count[i] > 0) { // Có tồn tại i trong A
            A[index++] = i;    // Gán i vào A[index]; index++
            count[i]--;        // Do đã lấy i ra, nên giảm count[i] xuống
        }
    }
}
```

ÔN TẬP KẾT THÚC MÔN.

I. THỰC HÀNH.

1. Ngăn xếp.

VD1: ***Chuỗi ngoặc hợp lệ (Valid parentheses)***.

Chuỗi ngoặc hợp lệ là chuỗi ngoặc mà với mỗi dấu mở ngoặc thì có dấu đóng ngoặc tương ứng. Cho một chuỗi ngoặc có thể có "(", "[", "{" hặc dấu đóng ngoặc tương ứng. Xác định xem chuỗi ngoặc đó có hợp lệ không.

Input	Output	Giải thích
([]{})	true	
({{	false	

Ý tưởng:

- Duyệt tuần tự chuỗi:
- Nếu gặp ký tự mở ngoặc thì đưa nó vào stack.
- Nếu gặp ký tự đóng ngoặc thì kiểm tra ký tự đầu tiên của stack, nếu là dấu mở ngoặc tương ứng.
- Như vậy, sau khi duyệt hết chuỗi, stack phải rỗng nếu chuỗi là dãy ngoặc hợp lệ.

Code mẫu:

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '{' || c == '[') {
            st.push(c);
        } else {
            if (st.empty())
                return false;
            char top = st.top();
            if ((c == ')' && top != '(') ||
                (c == '}' && top != '{') ||
                (c == ']' && top != '[')) {
                return false;
            }
            st.pop();
        }
    }
    return st.empty();
}
```

VD2: ***Biểu thức hậu tố*** *(Postfix expression)* ***và biểu thức trung tố*** *(Infix expression)*.

Biểu thức hậu tố (Ký pháp Ba Lan) là biểu thức mà tại mỗi biểu thức con có toàn bộ toán tử nằm phía sau toán hạng.

Ưu điểm của biểu thức hậu tố:

- ***Không cần dấu ngoặc***: Do không cần sử dụng dấu ngoặc để xác định thứ tự các phép toán, biểu thức hậu tố dễ dàng hơn khi xử lý bởi máy tính.
- ***Dễ dàng tính toán bằng Stack***: Biểu thức hậu tố rất phù hợp để xử lý bằng các cấu trúc dữ liệu như stack.
- ***Loại bỏ sự mơ hồ***: Thứ tự thực hiện phép toán được quy định rõ ràng, không phụ thuộc vào ưu tiên của các dấu ngoặc.

Yêu cầu 1: Cho một biểu thức hậu tố mà mỗi toán hạng chỉ có 1 chữ số, tính giá trị của biểu thức đó.

Input	Output	Giải thích
`1 2 + 3 *`	`9`	1 2 + → 1 + 2 = 3 1 2 + 3 * → (1 + 2) * 3 = 9
`1 2 / 3 4 * -`	`-11.5`	1 2 / → 1 / 2 = 0.5 3 4 * → 3 * 4 = 12 1 2 / 3 4 * - → 1 / 2 – 3 * 4 = -11.5
`1 2 3 4 * - /`	`-0.1`	3 4 * → 3 * 4 = 12 2 3 4 * - → 2 – 3 * 4 = -10 1 2 3 4 * - / → 1 / (2 – 3 * 4) = -0.1
`1 2 3 - / 4 *`	`-4`	2 3 - → 2 – 3 = -1 1 2 3 - / → 1 / (2 - 3) = -1 1 2 3 - / 4 * → 1 / (2 – 3) * 4 = -4

Ý tưởng:

- Duyệt tuần tự chuỗi.
- Nếu gặp ký tự chữ số thì đưa vào stack.
- Nếu gặp ký tự toán tử gì thì lấy 2 phần tử đầu tiên của stack ra để thực hiện toán tử đó. Sau đó đưa kết quả này vào stack.
- Như vậy, sau khi duyệt hết chuỗi, stack chỉ còn duy nhất 1 phần tử là giá trị của biểu thức.

Code mẫu:

```
float evaluatePostfix(string expr) {
    stack<float> st;
    for (char c : expr) {
        if (isdigit(c)) {
            st.push(c - '0');
        } else {
            float b = st.top(); st.pop();
            float a = st.top(); st.pop();
            switch (c) {
                case '+': st.push(a + b); break;
                case '-': st.push(a - b); break;
                case '*': st.push(a * b); break;
                case '/': st.push(a / b); break;
            }
        }
    }
    return st.top();
}
```

Yêu cầu 2: Chuyển biểu thức trung tố (infix) thành biểu thức hậu tố:

Input	Output
A*B+C*((D-E)+F)/G	A B * C E - F + * G / +

Ý tưởng:

- Phân tách các toán hạng, toán tử và các dấu ngoặc khỏi chuỗi, gọi là token.
- Duyệt các token:

token[i]	Hành vi
Là toán hạng.	• Đưa token vào output.
Là dấu mở ngoặc.	• Đưa token vào stack.
Là dấu đóng ngoặc.	• Lấy các phần tử từ stack ra và thêm vào output cho đến khi gặp dấu mở ngoặc. • Lấy dấu mở ngoặc đó ra khỏi stack.
Là toán tử	• Trong khi ngăn xếp không rỗng và toán tử trên đỉnh ngăn xếp có độ ưu tiên lớn hơn hoặc bằng toán tử hiện tại thì lấy toán tử ra khỏi stack và đưa vào output. • Đưa token vào stack.

- Sau khi duyệt hết biểu thức infix, nếu trong stack còn phần tử thì lấy các token trong đó ra và cho lần lượt vào output.

Code mẫu:

```
string toInfix(string postfix) {
    string infix;
    stack<char> st;
    for (char token : postfix) {
        if ('A' <= token && token <= 'Z') {
            infix += token; infix += ' ';
        }
        else if (token == '(') {
            st.push(token);
        }
        else if (token == ')') {
            while (!st.empty() && st.top() != '(') {
                infix += st.top(); infix += ' ';
                st.pop();
            }
            st.pop();
        }
        else {
            while (!st.empty()
                   && ((st.top() == '*' || st.top() == '/') &&
                       (token == '*' || token == '/' || token == '+' || token == '-
'))) {
                infix += st.top(); infix += ' ';
                st.pop();
            }
            st.push(token);
        }
    }
    return infix;
}
```

2. Hàng đợi.

3. Danh sách liên kết.

VD1: ***Khai báo danh sách liên kết đơn kéo dài về phía đuôi***.

Hãy viết hàm `void CreateEmptyList(LIST& L)` và `void PrintList(node* L.pHead)` để tạo và xuất danh sách liên kết đơn lưu trữ các số nguyên (giá trị số nguyên < 1000). Các số được lần lượt thêm vào cuối danh sách, kết thúc khi gặp giá trị **-1**.

Input	Output
6 4 8 2 5 1 8 4 2 4 -1	6 4 8 2 5 1 8 4 2 4
-1	Empty List.

Code mẫu:

```
struct node {
    int data;
    node *pNext;
};

struct LIST{
    node* pHead;
    node* pTail;
};

void CreateEmptyList(LIST &L) {
    L.pHead = NULL;
    L.pTail = NULL;
}

node *createNode(int data) {
    node *p = new node;
    p->pNext = NULL;
    p->data = data;
    return p;
}

void CreateList(LIST &L) {
    int tempData;
    cin >> tempData;

    while (tempData != -1) {
        node *pNew = createNode(tempData);
        if (L.pHead == NULL) {
            L.pHead = pNew;
            L.pTail = pNew;
        } else {
            L.pTail->pNext = pNew;
            L.pTail = pNew;
        }
        cin >> tempData;
    }
}

void PrintList(node *l) {
    if (l == NULL) {
        cout << "Empty List.";
        return;
    } else {
        while (l != NULL) {
            cout << l->data << ' ';
```

```
            l = l->pNext;
        }
    }
}
```

VD2: ***Khai báo danh sách liên kết đơn kéo dài về phía đầu***.

Hãy viết hàm **`void CreateEmptyList(LIST& L)`** để tạo và xuất danh sách liên kết đơn lưu trữ các số nguyên (giá trị số nguyên < 1000). Các số được lần lượt thêm vào đầu danh sách, kết thúc khi gặp giá trị **`-1`**.

Input	Output
6 4 8 2 5 1 8 4 2 4 -1	4 2 4 8 1 5 2 8 4 6
-1	Empty List.

Code mẫu:

```
struct node {
    int data;
    node *pNext;
};

struct LIST{
    node* pHead;
    node* pTail;
};

void CreateEmptyList(LIST &L) {
    L.pHead = NULL;
    L.pTail = NULL;
}

node *createNode(int data) {
    node *p = new node;
    p->pNext = NULL;
    p->data = data;
    return p;
}

void CreateList(LIST &L) {
    int tempData;
    cin >> tempData;

    while (tempData != -1) {
        node *pNew = createNode(tempData);
        if (L.pHead == NULL) {
            L.pHead = pNew;
            L.pTail = pNew;
        } else {
            pNew->pNext = L.pHead;
            L.pHead = pNew;
        }
        cin >> tempData;
    }
}
```

VD3: ***Khai báo danh sách liên kết đơn kéo dài về phía đuôi (chỉ dùng một con trỏ để lưu danh sách)***.

Hãy viết hàm **`void CreateEmptyList(NODE*& pHead)`** và **`void PrintList(NODE* pHead)`** để tạo và xuất danh sách liên kết đơn lưu trữ các số nguyên (giá trị số nguyên < 1000). Các số được lần lượt thêm vào đầu danh sách, kết thúc khi gặp giá trị **`-1`**.

Input	Output
6 4 8 2 5 1 8 4 2 4 -1	4 2 4 8 1 5 2 8 4 6
-1	Empty List.

Code mẫu:

```
struct NODE {
    int infor;
    NODE *PNext;
};


NODE *CreateNode (int infor){
    NODE *p = new NODE;
    p->infor = infor;
    p->PNext = NULL;
    return p;
}

void CreateEmptyList(NODE*& pHead) {
    pHead = NULL;
}

void CreateList(NODE*& pHead) {
    int tempData;
    cin >> tempData;

    NODE *pTemp = pHead;

    while (tempData != -1) {
        NODE *pNew = CreateNode(tempData);
        if (pTemp == NULL) {
            pHead = pNew;
            pTemp = pNew; // Việc thay đổi pTemp không thay đổi pHead
        } else {
            pTemp->PNext = pNew;
            pTemp = pNew;
        }
        cin >> tempData;
    }
}

void PrintList (NODE *pHead) {
    NODE *pTemp = pHead;
    if (pTemp == NULL) {
        cout << "Empty List.";
    } else {
        while (pTemp != NULL) {
            cout << pTemp->infor << ' ';
            pTemp = pTemp->PNext;
        }
    }
}
```

VD4: ***Chuyển phần tử giữa lên làm phần tử đầu***.

Cho danh sách liên kết đơn có cấu trúc như sau:

```
struct NODE {
    int info;
    NODE* pNext;
};

struct LIST {
    NODE* pHead;
    NODE* pTail;
};
```

Hãy viết hàm **`void setMiddleHead(NODE*& pHead)`** chuyển giá trị ở middle lên làm phần tử đầu danh sách liên kết L.

Yêu cầu kỹ thuật: Chỉ dùng 1 con trỏ để lưu trữ danh sách (pHead).

Input	Output
1 2 3 4 5 -1	3 1 2 4 5
1 2 3 4 5 6 -1	3 1 2 4 5 6

Code mẫu:

```
// Lấy middle
int getMiddle(NODE* pHead) {
    int count = 0;
    NODE* temp = pHead;
    while (temp) {
        count++;
        temp = temp->pNext;
    }
    return (count % 2 == 0 ? count - 1 : count) / 2;
}

void setMiddleHead(NODE*& pHead) {
    if (pHead == nullptr || pHead->pNext == nullptr)
        return;

    int m = getMiddle(pHead);
    if (m == 0)
        return;  // Phần tử đầu đã là middle

    NODE* prev = pHead;
    for (int j = 0; j < m - 1; ++j)
        prev = prev->pNext;

    // Cách ly middle
    NODE* middle = prev->pNext;
    prev->pNext = middle->pNext;

    // Hoàn thiện pHead
    middle->pNext = pHead;
    pHead = middle;
}
```

VD4: ***Chèn phần tử nhưng vẫn đảm bảo thứ tự***.

Cho danh sách liên kết đơn có cấu trúc như sau:

```
struct NODE {
    int key;
    NODE* pNext;
};

struct LIST {
    NODE* pHead;
    NODE* pTail;
};
```

Cho một danh sách liên kết đã được sắp xếp theo thứ tự giảm dần và một **NODE* X**. Hãy xây dựng hàm **void ChenGiam(LIST &L, NODE* x)** chèn **X** để vào danh sách sao cho vẫn đảm bảo thứ tự giảm dần.

Code mẫu:

```
void ChenGiam(LIST &L, NODE* x){

    // Ds chỉ có 1 phần tử
    if (!L.pHead) {
        L.pHead = x;
        L.pTail = x;
        return;
    }

    // Chèn đầu
    if (x->key > L.pHead->key) {
        x->pNext = L.pHead;
        L.pHead = x;
        return;
    }

    // Chèn sau vị trí đầu
    NODE* p = L.pHead;
    while (p->pNext && p->pNext->key.MASV > sv.MASV)
        p = p->pNext;

    x->pNext = p->pNext;
    p->pNext = x;

    if (p == L.pTail)
        L.pTail = x;
}
```

VD3: ***Đảo ngược danh sách liên kết***.

Cho danh sách liên kết đơn có cấu trúc như sau:

```
struct NODE {
    int info;
    NODE* pNext;
};

struct LIST {
    NODE* pHead;
    NODE* pTail;
};
```

Hãy viết hàm **`void Reverse_LinkedList(LIST& L)`** đảo ngược danh sách liên kết **`L`**.

Yêu cầu kỹ thuật: Không tạo danh sách phụ.

Input	Output
`1 2 3 4 5 -1`	`5 4 3 2 1`
`-1`	`Empty List.`

Ý tưởng:

- Lần lượt gọi **`curr`**, **`prev`** và **`next`** là 3 con trỏ trỏ đến node hiện tại, node liền trước node hiện tại (`prev->pNext = curr`) và node liền sau node hiện tại (`next = curr->pNext`).
- Lần lượt duyệt danh sách bằng `curr`:
    - Gọi `next` (để chuẩn bị dịch chuyển đến node tiếp theo).
    - Đảo chiều `curr` trỏ đến `prev`.
    - Dịch chuyển đến node tiếp theo bằng cách đưa `prev` làm `curr` và đưa `curr` làm `next`.

Code mẫu:

```
void Reverse_LinkedList(LIST& L) {
    NODE* prev = nullptr;
    NODE* curr = L.pHead;
    NODE* next = curr->pNext;

    while (curr != nullptr) {
        next = curr->pNext;
        // Đảo chiều
        curr->pNext = prev;
        // Dịch chuyển các node lên trên 1 nấc
        prev = curr;
        curr = next;
    }

    // Tinh chỉnh lại pHead và pTail
    L.pTail = L.pHead;
    L.pHead = prev;
}
```

VD4: ***Duyệt ngược từ cuối lên***.

Cho danh sách liên kết đơn có cấu trúc như sau:

```
struct NODE {
    int info;
    NODE* pNext;
};

struct LIST {
    NODE* pHead;
    NODE* pTail;
};
```

Cho biết giá trị tại vị trí thứ k (tính từ cuối danh sách) trong danh sách L.

Input	Output
6 4 -8 2 -5 8 1 4 2 0 -1 5	8
1 8 9 -1 4	-1

Ý tưởng:

- Duyệt lần 1: Lấy chiều dài danh sách (tốn n bước).
- Duyệt lần 2: Duyệt n – k bước.

Code mẫu:

```
int searchXget_nth_node_from_tail (NODE *pHead, int k) {
    // Lấy l là chiều dài của danh sách
    int l = 0;
    NODE *p = pHead;
    while (p != nullptr) {
        l++;
        p = p->pNext;
    }

    // Trường hợp đặc biệt
    if (k > l || k <= 0)
        return -1;

    // Tìm node cần tìm
    p = pHead;
    for (int i = 0; i < l - k; i++) {
        p = p->pNext;
    }

    return p->info;
}
```

VD5: ***Hoare quick sort trên linked list***.

Cho danh sách liên kết đơn có cấu trúc như sau:

```
struct NODE {
    int info;
    NODE* pNext;
};

struct LIST {
    NODE* pHead;
    NODE* pTail;
};
```

Hãy dùng thuật toán Hoare Quick sort để sắp xếp danh sách liên kết L theo thứ tự tăng dần.

Yêu cầu kỹ thuật: Không phát sinh bộ nhớ để lưu các node trung gian.

Input	Output
7 8 5 3 9 0 1 2	7 0 1 2 3 5 8 9
0	Empty List.

Ý tưởng:

- Ta cần xây dựng các hàm sau:
  - **`SeparateHead`**: Có chức năng tách head ra khỏi linked list.
  - **`Partition`**: Tạo ra 2 mảng **`L1`** (≤ pivot) và **`L2`** (> pivot). Để đơn giản, ta chọn luôn pivot là head.
  - **`Join`**: Nối **`L1`**, pivot và **`L2`** lại.
- Do yêu cầu không phát sinh thêm bộ nhớ lưu các node trung gian, ta sẽ sử dụng tham chiếu.

Code mẫu:

```
NODE* SeparateHead(LIST &L) {
    NODE* p = L.pHead;
    if (p == nullptr)
        return p;
    L.pHead = p->pNext;
    if (L.pHead == nullptr)
        L.pTail = nullptr;
    p->pNext = nullptr;
    return p;
}

void Partition(LIST &L, LIST &L1, NODE*& pivot, LIST &L2) {
    pivot = SeparateHead(L);
    while (L.pHead != nullptr) {
        NODE *p = SeparateHead(L);
        if (p->info <= pivot->info)
            addTail(L1, p);
        else
            addTail(L2, p);
    }
    L.pTail = nullptr;
}

void Join(LIST &L, LIST &L1, NODE*& pivot, LIST &L2) {
    while (L1.pHead != nullptr) {
        NODE *p = SeparateHead(L1);
        addTail(L, p);
    }

    addTail(L, pivot);

    while (L2.pHead != nullptr) {
```

```
        NODE *p = SeparateHead(L2);
        addTail(L, p);
    }
}

void quicksort_ascending(LIST& L) {
    LIST L1, L2;
    CreateEmptyList(L1);
    CreateEmptyList(L2);
    NODE *pivot;

    Partition(L, L1, pivot, L2);

    if (L1.pHead && L1.pHead != L1.pTail)
        quicksort_ascending(L1);
    if (L2.pHead && L2.pHead != L2.pTail)
        quicksort_ascending(L2);

    Join(L, L1, pivot, L2);
}
```

4. Bảng băm.

VD1: ***Kiểm tra đụng độ đối với bảng băm xử lý đụng độ bằng phương pháp nối kết trực tiếp***.

Cho bảng băm nối kết, có hàm băm là **h(key) = key % M**, trong đó key là giá trị khóa, M là kích thước bảng băm. Hãy viết chương trình in ra các địa chỉ trên bảng băm có đụng độ nhưng xảy ra ít đụng độ nhất, theo một bộ dữ liệu đầu vào.

Đầu vào:

- Dòng đầu tiên là hai số n, M. Cho biết bộ dữ liệu này có n khóa được nhập vào, và bảng băm có kích thước M.
- n số kế tiếp là các giá trị khóa key được thêm vào bảng băm. Giá trị của key trong đoạn [10000, 100000].

Đầu ra: Một dòng duy nhất, gồm k địa chỉ trên bảng băm đã xảy ra đụng độ nhưng với số lượng ít nhất. **Các địa chỉ này được in theo thứ tự tăng dần**.

Lưu ý: Bảng băm không giới hạn số lượng khóa được nhập vào.

Input	Output
5 7 10012 10019 10021 10026 10035	4

Ý tưởng:

- Giả sử mỗi node trong bảng băm là 1 vector. Khi đó, tại một node có xảy ra đụng độ thì tức là độ dài vector ≥ 2.

Code mẫu:

```
int n, m;
cin >> n >> m;

vector <vector <int>> hashTable (m);

for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    hashTable[H(x, m)].push_back(x);
}

int minCollide = INT_MAX;
for (int i = 0; i < m; i++)
    if (hashTable[i].size() >= 2 && hashTable[i].size() < minCollide)
    minCollide = hashTable[i].size();

for (int i = 0; i < m; i++)
    if (hashTable[i].size() == minCollide)
        cout << i << ' ';
```

VD2: ***Ứng dụng thực tế của bảng băm***.

Để giúp user thuận tiện trong việc đăng nhập thì trang web A cần cài đặt cơ chế tự động điền mật khẩu của một số tài khoản đã có trong cơ sở dữ liệu. Em hãy hỗ trợ cài đặt cơ chế trên.

Input:

- ***Dòng đầu tiên***: Chứa 2 số N và M. Trong đó:
  - N ($1 \le N \le 10000$) là số lượng (tài khoản, mật khẩu) trong cơ sở dữ liệu.
  - M ($1 \le M \le 10000$) là số lần người dùng nào đó nhập tài khoản.
- ***N dòng tiếp theo***: Mỗi dòng có hai chuỗi ký tự user ($1 \le |user| \le 30$) và pass ($1 \le |pass| \le 30$).
- ***M dòng cuối***: Mỗi dòng gồm một chuỗi ký tự str là tài khoản của người dùng đang đăng nhập ($1 \le |str| \le 30$).

Output:

- Gồm M dòng: Mật khẩu tương ứng với người dùng đang đăng nhập.
- Nếu người dùng cập nhật mật khẩu nhiều lần thì ***in ra mật khẩu mới nhất***.
- Nếu người dùng chưa đăng ký mật khẩu thì in ra **`Unregistered User.`**.

Input	Output
6 3 PMCL 1234 MMCL 1234 PMCL 1234abcXYZ MMCL abcZYX1234 MMCL 1234QAZ@# KHMT mnbjhg KHMT1 PMCL MMCL	Unregistered User. 1234abcXYZ 1234QAZ@#

Ý tưởng:

- Nhận xét: Xét cặp username – password, ta thấy username có tính độc nhất, password thì có thể trùng. Với mỗi username thì chỉ có một password tương ứng → Tìm cách băm username để lưu password là tối ưu nhất.
- Như vậy, ta tạo một bảng băm có cấu trúc mỗi node là một cặp (username – password). Khi nhận một username x:
  - Nếu như x tồn tại trong các username → Thay đổi password.
  - Nếu như x chưa tồn tại → Cập nhật username và password mới.

Code mẫu:

```
#include <iostream>
#include <string.h>

using namespace std;

struct NODE {
    string password;
    string username;
};
```

```cpp
struct HASHTABLE {
    int M;
    NODE* table;
};

int HF(HASHTABLE H, string str) {
    int sum = 0;
    for (auto c : str)
        sum += c;
    return sum % H.M;
}

int reHF(HASHTABLE H, int vt, int i) {
    return (vt + i) % H.M;
}

void Insert(HASHTABLE& H, string username, string password) {
    int vt = HF(H, username);
    int i = 1;

    while (true) {
        // Nếu như vị trí sau khi băm trống hoặc username đã tồn tại thì cập nhật
username và password
        if (H.table[vt].username == "" || H.table[vt].username == username) {
            H.table[vt].username = username;
            H.table[vt].password = password;
            return;
        } else
            vt = reHF(H, vt, i++);
    }
}

string getPassword(HASHTABLE H, string username) {
    int vt = HF(H, username);
    int i = 1;

    while (true) {
        if (H.table[vt].username == "")
            return "Unregistered User."; // Nếu như username trống thì tức là chưa đk
        else if (H.table[vt].username == username)
            return H.table[vt].password;
        else
            vt = reHF(H, vt, i++);
    }
}

int main() {

    HASHTABLE H;

    int n, m;
    cin >> n >> m;

    // init
    H.M = n;
    H.table = new NODE[H.M];
    for (int i = 0; i < n; i++)
        H.table[i].username = "";
```

```
    for (int i = 0; i < m; i++) {
        string username, password;
        cin >> username >> password;
        Insert(H, username, password);
    }

    for (int i = 0; i < m; i++) {
        string username;
        cin >> username;
        cout << getPassword(H, username) << '\n';
    }

    return 0;
}
```

5. Cây nhị phân tìm kiếm.

VD1: ***Duyệt cây bằng NLR***.

Cho cây BST có cấu trúc như sau:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Cho một danh sách số nguyên kết thúc bằng -1. Hãy tạo 1 cây BST là các số nguyên trên, sau đó in cây này ra theo thứ tự NLR.

Input	Output
6 5 7 2 8 1 -1	1 2 5 6 7 8
-1	Empty List.

Code mẫu:

```
TNODE* CreateNode(int x) {
    TNODE* p = new TNODE;
    p->key = x;
    p->pRight = nullptr;
    p->pLeft = nullptr;
    return p;
}

void Insert(TREE& Root, int x) {
    TREE T = Root;

    if (!Root)
        Root = CreateNode(x);
    else while(T) {
        if (x == T->key)
            return;
        if (x < T->key) {
            if (T->pLeft)
                T = T->pLeft;
            else {
                T->pLeft = CreateNode(x);
                return;
            }
        } else {
            if (T->pRight)
                T = T->pRight;
            else {
                T->pRight = CreateNode(x);
                return;
            }
        }
    }
}
```

```
void CreateTree(TREE& T) {
    while (true) {
        int x;
        cin >> x;

        if (x == -1)
            break;

        Insert(T, x);
    }
}

void LNR(TREE T) {
    if (T->pLeft)
        LNR(T->pLeft);
    cout << T->key << ' ';
    if (T->pRight)
        LNR(T->pRight);
}

void PrintTree(TREE T) {
    if (!T)
        cout << "Empty Tree.";
    else LNR(T);
}

int main() {
        TREE T = nullptr;
        CreateTree(T);
        PrintTree(T);
        return 0;
}
```

VD2: ***Duyệt cây bằng BFS***.

Cho cây BST có cấu trúc như sau:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Cho một danh sách số nguyên kết thúc bằng -1. Hãy tạo 1 cây BST là các số nguyên trên, sau đó in cây này ra theo thứ tự BFS.

Input	Output
9 16 6 7 2 12 4 11 25 1 20 -1	9 6 16 2 7 12 25 1 4 11 20
-1	Empty Tree.

Ý tưởng:

- Dùng queue.

Code mẫu:

```
void PrintTreeBFS(TREE T) {
    if (!T) {
        cout << "Empty Tree.";
        return;
    }

    queue <TNODE*> q;
    q.push(T);

    while (!q.empty()) {
        TNODE* p = q.front();
        q.pop();

        cout << p->key << ' ';

        if (p->pLeft)
            q.push(p->pLeft);
        if (p->pRight)
            q.push(p->pRight);
    }
}
```

VD2: ***Duyệt cây bằng BFS (có phân biệt cấp)***.

Cho cây BST có cấu trúc như sau:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Cho một danh sách số nguyên kết thúc bằng -1. Hãy tạo 1 cây BST là các số nguyên trên, sau đó in cây này ra theo thứ tự BFS, các node cùng cấp thì in trên cùng một hàng.

Input	Output
9 16 6 7 2 12 4 11 25 1 20 -1	9 6 16 2 7 12 25 1 4 11 20
-1	Empty Tree.

Ý tưởng:

- Dùng queue, nhưng ta sẽ chèn **`nullptr`** vào để phân biệt giữa các cấp.

Code mẫu:

```
void PrintTreeBFS(TREE T) {
    if (!T) {
        cout << "Empty Tree.";
        return;
    }

    queue <TNODE*> q;
    q.push(T);
    q.push(nullptr);


    while (!q.empty()) {
```

```
        TNODE* p = q.front();
        q.pop();

        if (!p) {
            cout << '\n';
            if (!q.empty())
                q.push(nullptr);
        } else {
            cout << p->key << ' ';
            if (p->pLeft)
                q.push(p->pLeft);
            if (p->pRight)
                q.push(p->pRight);
        }
    }
}
```

VD2: ***Tìm mức của một khóa***.

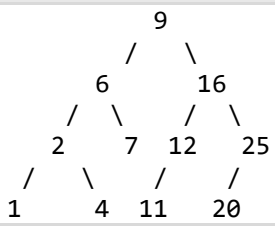
Cho cây BST có cấu trúc như sau:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Xây dựng hàm **`void PrintLevel(TREE T, int x, int& level)`** để về mức của node `x`. Nếu `x` không tồn tại trong cây thì trả về `-1`.

Yêu cầu kỹ thuật: Dùng queue.

Input	Output	Minh họa cây
7 9 16 6 7 2 12 4 11 25 1 20 -1	2	
30 9 16 6 7 2 12 4 11 25 1 20 -1	-1	

Ý tưởng:

- Đây thực chất chỉ là 1 biến thể của ví dụ trên.
- Mỗi lần gặp `nullptr` thì đã kết thúc một level khác 0.

Code mẫu:

```
void PrintLevel(TREE T, int x, int& level) {
    if (!T)
        return;

    queue<TNODE*> nodes;
    nodes.push(T);
    nodes.push(nullptr);
    level = 0;

    while (!nodes.empty()) {
        TNODE* p = nodes.front();
        nodes.pop();
```

```
        if (!p) {
            level++;
            if (!nodes.empty())
                nodes.push(nullptr);
        } else {
            if (p->key == x)
                return;

            if (p->pLeft)
                nodes.push(p->pLeft);
            if (p->pRight)
                nodes.push(p->pRight);
        }
    }

    level = -1;
}
```

VD3: ***Tổng node trong***.

Cho cây BST có cấu trúc như sau:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Cho một danh sách số nguyên kết thúc bằng -1. Xây dựng hàm trả về tổng giá trị các node không phải lá.

Input	Output
9 16 6 7 2 12 4 11 25 -1	45
-1	0

Ý tưởng:

- Ngoại trừ các node lá, còn lại thì ta đều nhận key làm tổng.

Code mẫu:

```
int SumInternalNode (TREE T) {
    if (!T)
        return 0;
    if (!T->pLeft && !T->pRight)
        return 0;
    else
        return T->key + SumInternalNode(T->pLeft) + SumInternalNode(T->pRight);
}
```

VD3: ***Tổng nhóm cha con lớn nhất***.

Cho cây BST có cấu trúc như sau:

```
struct TNode {
    int key;
    TNode * left, * right;
};

typedef TNode* TREE;
```

Hãy hoàn thành hàm **`int MaxGroup(TREE)`** để trả về tổng giá trị khóa của một nút và hai nút con trên cây nhị phân.

Input	Output
10 1 4 13 2 17 8 15 6 3 11 13 4 17 8 2 1 15 6 11 3	25

Ý tưởng:

- Tại một node, hàm MaxGroup sẽ trả về max của cặp cha – con node đó, cặp cha – con của cây con phải và cặp cha – con của cây con trái.

Code mẫu:

```
int MaxGroup(TREE T) {
    if (!T)
        return 0;
    else {
        int thisSum = T->key +
                      (T->left ? T->left->key : 0) +
                      (T->right ? T->right->key : 0);
        int rightSum = MaxGroup(T->right);
        int leftSum = MaxGroup(T->left);

        if (thisSum > rightSum && thisSum > leftSum)
            return thisSum;
        else if (rightSum > thisSum && rightSum > leftSum)
            return rightSum;
        else
            return leftSum;
    }
}
```

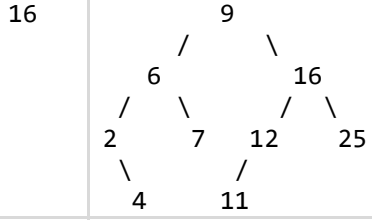
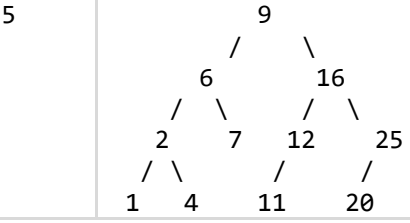
VD2: ***Đếm node của cây con***.

Cho cây BST có cấu trúc như sau:

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;
```

Tìm tất cả các nút trên cây có số nút con bên trái lớn hơn số nút con bên phải một đơn vị.

Input	Output	Minh họa cây
9 16 6 7 2 12 4 11 25 -1	6 12 16	
9 16 6 7 2 12 4 11 25 1 20 -1	12 25	

Ý tưởng:

- Ở mỗi node, ta chỉ cần đếm và so sánh số lượng node con trái và phải của nó là được.

Code mẫu:

```
int CountNode(TREE T) {
    if (!T)
        return 0;
    else {
        int leftNodes = CountNode(T->pLeft);
        int rightNodes = CountNode(T->pRight);
        if (leftNodes - rightNodes == 1)
            cout << T->key << ' ';

        return 1 + leftNodes + rightNodes;
    }
}
```

VD3: ***Xây dựng cây BST có nhiều node trùng nhau***.

Cho một danh sách số nguyên kết thúc bằng -1, các số nguyên có thể xuất hiện nhiều lần. Hãy tạo cây BST dựa trên danh sách trên và đếm số lần xuất hiện của mỗi giá trị trong cây theo thứ tự giảm dần của giá trị.

Input	Output
9 7 5 3 6 3 7 5 3 3	3 xuat hien 4 lan 5 xuat hien 2 lan 6 xuat hien 1 lan 7 xuat hien 2 lan
-1	

Ý tưởng:

- Tạo thêm một trường nữa cho kiểu dữ liệu node là **count**, có chức năng lưu trữ số lần xuất hiện của một giá trị **x** nào đó khi **x == key**.
- Do đề yêu cầu thứ tự các key *tăng dần*, ta duyệt theo LNR.

Code mẫu:

```
struct TNODE {
    int key;
    int cnt;
    TNODE* pLeft;
    TNODE* pRight;
};

typedef TNODE* TREE;

TNODE* CreateNode(int x) {
    TNODE* p = new TNODE;
    p->key = x;
    p->cnt = 1;
    p->pRight = nullptr;
    p->pLeft = nullptr;
    return p;
}

void Insert(TREE& Root, int x) {

    TREE T = Root;

    if (!Root)
        Root = CreateNode(x);
    else while(T) {
        if (x == T->key) {
            T->cnt++;
            return;
        }
        if (x < T->key) {
            if (T->pLeft)
                T = T->pLeft;
            else {
                T->pLeft = CreateNode(x);
                return;
            }
        } else {
            if (T->pRight)
                T = T->pRight;
```

```
            else {
                T->pRight = CreateNode(x);
                return;
            }
        }
    }
}

void CreateTree (TREE& T) {
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        Insert(T, x);
    }
}

void PrintTree(TREE& T) {
    if (!T)
        return;

    if (T->pLeft)
        PrintTree(T->pLeft);

    cout << T->key << " xuat hien " << T->cnt << " lan\n";

    if (T->pRight)
        PrintTree(T->pRight);
}
```

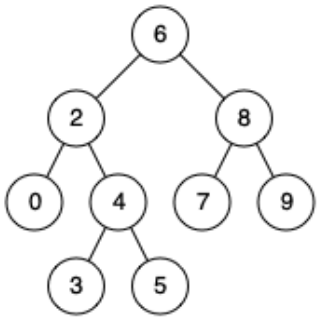
VD4: ***Tìm nút tổ tiên chung thấp nhất (LCA) của 2 nút bất kỳ***.

Cho 1 cây BST có cấu trúc như sau:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

Cho 2 nút p, q và nút T là gốc của cây, hãy tìm tổ tiên chung thấp nhất của p và q.

Ví dụ: Cho cây sau:



- *Tổ tiên chung thấp nhất của 2 và 8 là 6.*
- *Tổ tiên chung thấp nhất của 3 và 4 là 6 (2).*

Ý tưởng:

- Duyệt từ gốc lên lá, nếu có nút k nào mà **|con trái| < k |con phải|** thì đó chính là nút cần tìm (dựa trên định nghĩa cơ bản nhất trong mối quan hệ cha con trong cây BST).
- Trong ví dụ (2), ta thấy cả 3 và 4 đều nằm ở bên trái nút 6. Cho nên, khi duyệt mà chưa tìm thấy k nào thỏa |k->left| < |k| < |k->right| thì ta vẫn cứ duyệt sang phải và trái như quy tắc bình thường cho đến duyệt trúng k.

Code mẫu:

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root)
        if (p->val > root->val && q->val > root->val)
            root = root->right;
        else if (p->val < root->val && q->val < root->val)
            root = root->left;
        else
            return root;
    return nullptr;
}
```

VD5: ***Kiểm tra cây BST***.

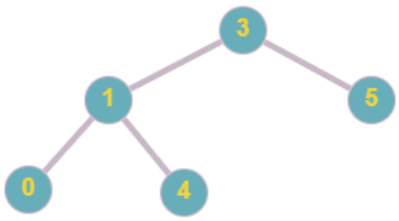
Cho 1 cây có cấu trúc như sau:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

Cho T là gốc của cây, hãy kiểm tra xem cây gốc T có phải cây tìm kiếm nhị phân không.

Ý tưởng:

- Ở mô hình cặp cha – con cục bộ, **|con trái| < |cha| < |con phải|**, như ví dụ trên. Tuy nhiên, chỉ dùng điều kiện này là không đủ để kiểm tra cây BST, ví dụ như trường hợp sau:



  Rõ ràng là 0 < 1 < 4, nhưng cây trên không phải BST vì 4 > root.
- Tổng quát hóa: **|Các nút của cây con trái phải| < |Cha| < |Các nút của cây con phải|**.

Code mẫu:

```
bool isValid(TreeNode* root, long minVal, long maxVal) {
    if (!root)
        return true;

    if (!(root->val > minVal && root->val < maxVal))
        return false;

    return isValid(root->left, minVal, root->val) &&
        isValid(root->right, root->val, maxVal);
}

bool isValidBST(TreeNode* root) {
    return isValid(root, LONG_MIN, LONG_MAX);
}
```

VD5: ***Kiểm tra cây nhị phân hoàn chỉnh***.

Cho 1 cây có cấu trúc như sau:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

Cho T là gốc của cây, hãy kiểm tra xem cây gốc T có phải cây nhị phân hoàn chỉnh không.
Yêu cầu kỹ thuật: Dùng queue.

Ý tưởng:

- Khi dùng queue, ta dễ dàng thu được các node cùng cấp.
- Tại một node cha, nếu cha không có con trái, và nếu cây đang xét thực sự là cây nhị phân hoàn chỉnh thì ta đã xét đến các node lá. Điều đó có nghĩa là ***các node con cùng cấp tiếp theo khi được duyệt bắt buộc phải là lá***.
- Để đảm bảo tính "lệch trái nhất có thể" đối với các node lá, ta phải ưu tiên ***kiểm tra con trái trước, con phải sau***.

Code mẫu:

```
bool IsCompleteBT(TREE root) {
    if (!root)
        return true;

    queue <TNODE*> nodes;
    nodes.push(root);
    bool mustBeLeaf = false;

    while (!nodes.empty()) {
        TNODE* p = nodes.front();
        nodes.pop();

        if (p->pLeft) {
            if (mustBeLeaf)
                return false;
            else
                nodes.push(p->pLeft);
        } else
            mustBeLeaf = true;

        if (p->pRight) {
            if (mustBeLeaf)
                return false;
            else
                nodes.push(p->pRight);
        } else
            mustBeLeaf = true;
    }
    return true;
}
```

VD6: ***Dựng BST cân bằng nhất từ danh sách đã được sắp xếp***.

Cho 1 cây có cấu trúc như sau:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

Cho một danh sách nums đã được sắp xếp và phân biệt. Hãy từ danh sách đó tạo một cây BST cân bằng.

Chú ý: Cho sẵn hàm `InsertNode(T, x)` dùng để chèn node có giá trị `x` vào cây `T`.

Ý tưởng:

- Nếu lần lượt duyệt và insert các node theo thứ tự tăng dần hoặc giảm dần, cây sẽ bị duỗi sang trái (tăng dần) và duỗi sang phải (giảm dần), tức là không cân bằng.
- Cách làm đúng ở đây là duyệt theo kiểu tìm kiếm nhị phân:
    - Insert node giữa vào trước.
    - Insert node giữa của đoạn trái.
    - Insert node giữa của đoạn phải.

Code mẫu:

```
void buildBST(TreeNode*& T, vector<int>& nums, int l, int r) {
    if (l > r)
        return;
    int mid = (l + r) / 2;
    InsertNode(T, nums[mid]);
    buildBST(T, nums, mid + 1, r);
    buildBST(T, nums, l, mid - 1);
}

TreeNode* sortedArrayToBST(vector<int>& nums) {
    TreeNode* T = nullptr;

    buildBST(T, nums, 0, nums.size() - 1);

    return T;
}
```

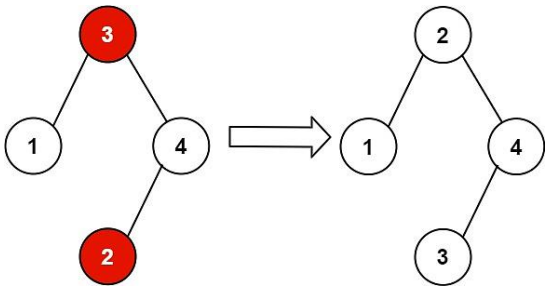
VD7: ***Phục hồi cây BST từ cây bị lỗi***.

Cho 1 cây có cấu trúc như sau:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

Cho một cây root nhưng giá trị của một số node trên cây bị sai lệch. Hãy sửa lại cho đúng (không thay đổi hình dạng cây ban đầu).

Ví dụ:



Trong cây trên, node 3 và 2 bị sai, ta đảo 2 node này lại thì sẽ cho ra cây BST đúng.

Ý tưởng:

- Như ta đã biết, duyệt cây theo thứ tự LNR sẽ cho ra một danh sách tăng dần. Nhưng đối với các cây bị sai, ví dụ như cây trên, LNR cho ra "1 3 2 4" không theo thứ tự.
- Ta chỉ cần sắp xếp danh sách này lại theo thứ tự tăng dần là xong.

Code mẫu:

```
void LNR(TreeNode* root, vector <TreeNode*>& path) {
    if (!root)
        return;

    LNR(root->left, path);

    path.push_back(root);

    LNR(root->right, path);
}

void recoverTree(TreeNode* root) {
    if (!root)
        return;

    vector <TreeNode*> path;
    LNR(root, path);

    vector <int> nodeValues;
    for (auto node : path)
        nodeValues.push_back(node->val);

    sort(nodeValues.begin(), nodeValues.end());
```

```
    for (int i = 0; i < path.size(); i++)
        path[i]->val = nodeValues[i];
}
```

VD8: ***Dung hợp các cây BST***.

Cho 1 cây có cấu trúc như sau:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

Bạn được cung cấp n gốc của các cây tìm kiếm nhị phân (BST) riêng biệt, lưu trong một mảng trees (đánh chỉ số từ 0). Không có hai cây nào có gốc (root) cùng giá trị và mỗi cây này chỉ có tối đa 2 con.

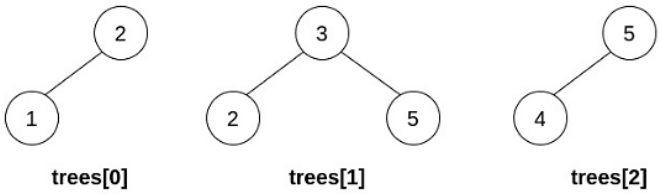
Trong một thao tác, bạn có thể thực hiện như sau:

- Chọn hai chỉ số phân biệt i và j sao cho giá trị tại một trong các lá (leaf) của trees[i] bằng với giá trị gốc (root) của trees[j].
- Thay thế nút lá trong trees[i] bằng cây trees[j] (tức là nối cây trees[j] vào vị trí nút lá đó).
- Xóa trees[j] khỏi trees.

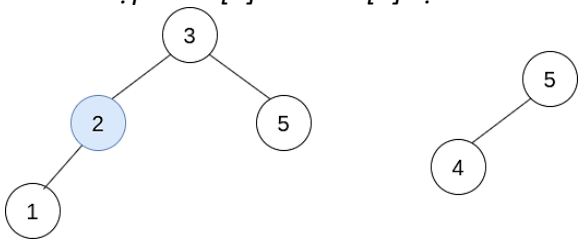
Nhiệm vụ:

- Trả về gốc của cây BST cuối cùng nếu bạn có thể thực hiện n − 1 lần thao tác để ghép tất cả các cây lại thành một cây BST hợp lệ.
- Ngược lại, nếu không thể tạo ra một BST hợp lệ, hãy trả về null.

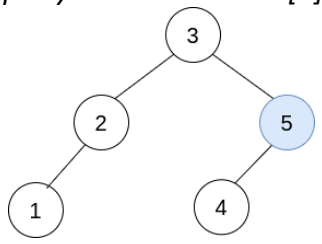
Ví dụ 1:



Bước 1: Gộp trees[0] và trees[1] tại 2:



Bước 2: Gộp cây vừa rồi với trees[2] tại 5:



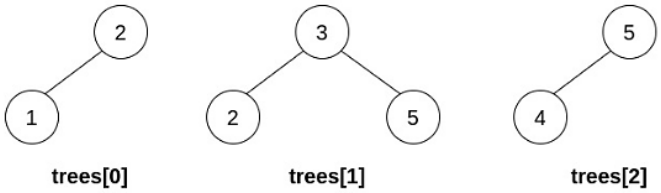
Ví dụ 2:



Không gộp được.

Ý tưởng:

- Xét ví dụ 1, giả sử có danh sách các root và danh sách các lá ứng với các root



roots	2	3	5

leaves	1	2	4	5

- Nếu như trees[i] là đáp án thì chắc chắn trees[i] không thể là lá của một cây khác. Có nghĩa là, trees[i] không tồn tại trong leaves (*nút 3*). Như vậy, ta đã xác định được gốc của cây cần tìm.
- Tại một nút trees[i] không phải gốc, tức là nó có tồn tại trong leaves, ta trực tiếp thay thế nó bằng nút đã tồn tại tương ứng trong leaves.

Code mẫu:

Cấu trúc roots và leaves:

```
unordered_map <int, TreeNode*> roots;
unordered_set <int> leaves;
```

Thành phần đầu tiên của roots chính là val của root.
Ta không cần lưu `TreeNode*` cho leaves vì chỉ cần `val` của leaf là ta có thể gọi roots tương ứng (nếu có một root là leaf)

Hàm tìm gốc:

```
TreeNode* getRoot(vector<TreeNode*>& trees) {
    for (auto root : trees)
        if (!leaves.count(root->val))
            return root;
    return nullptr;
}
```

Phương thức `.count(x)` sẽ trả về số lần xuất hiện của `x` trong cấu trúc.

Hàm xây dựng cây từ gốc đã tìm được:

```
TreeNode* getTree(TreeNode* root) {
    if (!root)
        return nullptr;

    TreeNode* tmp =  roots.count(root->val) ? roots[root->val] : root;
    roots.erase(root->val);

    root->left = getTree(tmp->left);
    root->right = getTree(tmp->right);

    return tmp;
}
```

`tmp` sẽ nhận 2 trường hợp:

- *Nếu `root` là gốc đáp án (`.count(x) == 0`) thì giữ nguyên đó và gọi đệ quy xử lý các con của `root`.*
- *Nếu `root` có tồn tại trong leaves (`.count(x) > 0`) thì lấy nút gốc tương ứng giá trị với `root` và nối vào `root`, đồng thời xóa bỏ nó khỏi `root` vì ta đã gộp nó vào cây đáp án rồi.*

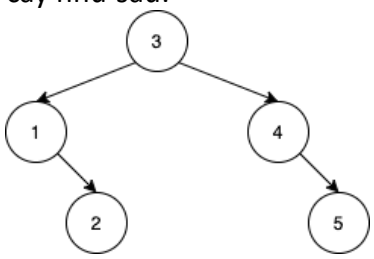
Cuối cùng, nếu cây được tạo ra là cây BST và toàn bộ roots đã được gộp thì đó là đáp án cần tìm.

```
isBST(new_root, 5000000, -5000000)
&& !roots.size()
```

VD9: ***Đếm số cách tạo ra cùng 1 cây BST***.

Cho một danh sách nums chứa các số nguyên. Một cây BST được tạo ra từ nums bằng cách lần lượt thêm từng phần tử của nó vào cây. Hãy đếm số hoán vị khác của nums mà vẫn cho ra cây giống với ban đầu.

Ví dụ: Với nums = [3,4,5,1,2], ta có cây như sau:



Đáp án của bài toán này sẽ là 5 vì có tới 5 hoán vị khác của nums vẫn cho ra cây trên.

1. [3,1,2,4,5]
2. [3,1,4,2,5]
3. [3,1,4,5,2]
4. [3,4,1,2,5]
5. [3,4,1,5,2]

Chú ý:

- Danh sách nums có tối đa 1000 phần tử.
- Lấy kết quả bài toán chia dư cho $10^9 + 1$.

Ý tưởng:

- Ta dễ dàng nhận thấy phần tử đầu tiên luôn là gốc.
- Ta chia danh sách thành 2 phần: Các phần tử nhỏ hơn gốc (thuộc về cây con trái) và các phần tử lớn hơn gốc (thuộc về cây con phải).
- Các phần tử trong 2 danh sách này có thể tráo đổi tùy ý miễn sao thứ tự xuất hiện của chúng so với các nút trong cùng danh sách là giống nhau.
- Ví dụ như với nums = [3,4,5,1,2], ta có 2 cây con là [1,2] và [4,5]. Ta có thể trộn 4 nút này lại miễn sao 1 xuất hiện trước 2, 4 xuất hiện trước 5. Chẳng hạn [1,4,2,5] là hợp lệ nhưng [2,1,4,5] thì không.
- Gọi t_l là cây con trái và t_r là cây con phải. Cho một danh sách có $|t_l| + |t_r|$ ô trống. Ta cần chọn $|t_l|$ ô trống để đặt vào các nút con trái theo thứ tự, phần còn lại sẽ đặt các nút con phải theo thứ tự. Vậy đáp án cho 2 cây con cùng cấp sẽ là:
$$C_{|t_l|+|t_r|}^{|t_l|} = C_{|t_l|+|t_r|}^{|t_l|} = \frac{(|t_l| + |t_r|)!}{|t_r|!.|t_l|!}$$
- Gọi $w(t)$ là số cách tạo ra cây t. Theo quy tắc nhân:
$$w(t) = w(t_l).w(t_r).C_{|t_l|+|t_r|}^{|t_l|}$$
- Đáp án của bài toán sẽ là:
$$w(root) - 1$$
- Trong lập trình, có một phương pháp tính nhanh tổ hợp, đó là dùng tam giác Pascal.
$$C_n^k = Pascal_{k,n}$$

Code mẫu:

Thuật toán xây dựng tam giác Pascal (đã chia dư):

```
int pascal[MAX][MAX];

void buildPascal() {
    for (int n = 0; n < MAX; n++) {
        pascal[n][0] = pascal[n][n] = 1;
        for (int k = 1; k < n; k++)
            pascal[n][k] = (pascal[n - 1][k - 1] + pascal[n - 1][k]) % MOD;
        }
}
```

Hàm đếm số cách tại 1 nút:

```
int countWays(const vector <int>& nums) {
    if (nums.size() == 1)
        return 1;

    vector <int> leftTree;
    vector <int> rightTree;

    for (int i = 1; i < nums.size(); i++)
        if (nums[i] < nums[0])
            leftTree.push_back(nums[i]);
        else
            rightTree.push_back(nums[i]);

    return countWays(leftTree) *
            countWays(rightTree) *
            pascal[leftTree.size() + rightTree.size()][leftTree.size()];
}
```

6. Các thuật toán tìm kiếm.

VD1: ***Chèn phần tử mới vào sao cho thứ tự vốn có của danh sách không đổi***.

Cho một danh sách nums đã được xếp theo thứ tự tăng dần và một số nguyên target.

- Nếu target tồn tại trong nums, hãy in ra index của nó.
- Nếu không, hãy in ra index sao cho khi chèn target vào index đó, nums vẫn tăng dần.

Input	Output
1 3 5 6 5	2
1 3 5 6 2	1
1 3 5 6 7	4

Ý tưởng: Trong quá trình binary search, tại vị trí l = r, nếu nums[l] = target thì hãy in ra; nếu không:

- Nếu target > nums[l], in ra l + 1.
- Nếu không, in ra l.

Code mẫu:

```
int searchInsert(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    int mid;
    while (l < r) {
        mid = (l + r) / 2;
        if (target == nums[mid])
            return mid;
        else if (target > nums[mid])
            l = mid + 1;
        else if (target < nums[mid])
            r = mid - 1;
    }

    // Khi l == r
    if (target > nums[l])
        return l + 1;
    else
        return l;
}
```

VD2: ***Kiểm tra số chính phương với độ phức tạp $O(\log n)$***.

Trả về true nếu n ($1 \le n \le 2^{31} - 1$) là số chính phương và ngược lại.

Input	Output
2147483647	false

Ý tưởng:

- Bản chất của bài toán này là ta đang tìm một số mid sao cho $mid^2 = num$.
- Chú ý: Biểu thức **`mid == num / mid`** và **`num / mid > mid`** sẽ sai về mặt lập trình.

Code mẫu:

```
bool isPerfectSquare(int num) {
    int l = 1, r = num;
    while (l <= r) {
        long long mid = l + (r - l) / 2;
        if (mid * mid == num)
            return true;
        else if (num > mid * mid)
            l = mid + 1;
        else
            r = mid - 1;
    }
    return false;
}
```

VD1: ***Tìm tải trọng nhỏ nhất có thể***.

Một băng chuyền chứa n gói hàng cần được vận chuyển từ một cảng đến cảng khác trong days ngày.

Gói hàng thứ i trên băng chuyền có trọng lượng weights[i]. Mỗi ngày, ta chất hàng lên tàu theo đúng thứ tự trên băng chuyền (weights). Tuy nhiên, tổng trọng lượng hàng chất lên tàu trong một ngày không được vượt quá sức chứa tối đa của tàu.

Hãy trả về sức chứa nhỏ nhất của tàu sao cho tất cả các gói hàng trên băng chuyền được vận chuyển trong vòng days ngày.

Input	Output	Giải thích
1 2 3 4 5 6 7 8 9 10 5	15	Với tải trọng là 15: • Ngày 1: Vận chuyển 1, 2, 3, 4, 5. • Ngày 2: Vận chuyển 6, 7. • Ngày 3: Vận chuyển 8. • Ngày 4: Vận chuyển 9. • Ngày 5: Vận chuyển 10. Với tải trọng này, tất cả hàng được vận chuyển trong 5 ngày. Đây là tải trọng nhỏ nhất có thể để làm việc đó.
3 2 2 4 1 4 3	6	

Ý tưởng:

- Giả định mid là đáp án cần tìm, ta sẽ dùng binary search để tìm mid, với đáp án là mid nhỏ nhất có thể.
- Gọi hàm **`possible(capacity)`**. Hàm này sẽ trả về true nếu với tải trọng capacity, toàn bộ hàng sẽ được vận chuyển xong trong vòng tối đa days ngày.

- Với mid:
  - Nếu possible(mid) là true, ta cho r = mid – 1 (giảm khoảng tìm xuống) để tìm mid nhỏ nhất có thể.
  - Nếu không, ta cho l = mid + 1, để tìm mid mà possible(mid) true.
- Vấn đề tìm l và r ban đầu:
  - Nếu days = 1, toàn bộ hàng phải được vận chuyển trong một ngày, có nghĩa là tải trọng (r) phải bằng tổng khối lượng các hàng.
  - Nếu days = n, mỗi ngày chỉ vận chuyển một hàng, có nghĩa là tải trọng (l) phải bằng khối lượng lớn nhất.

Code mẫu:

```
bool possible(vector<int>& weights, int capacity, int days) {
    int current_weights = 0;
    int cnt_days = 0;
    for (int i = 0; i < weights.size(); i++) {
        if (current_weights + weights[i] <= capacity)
            current_weights += weights[i];
        else {
            cnt_days++;
            current_weights = weights[i];
        }
    }
    if (current_weights <= capacity)
        cnt_days++;
    return (cnt_days <= days);
}

int shipWithinDays(vector<int>& weights, int days) {
    int r = accumulate(weights.begin(), weights.end(), 0);
    int l = *max_element(weights.begin(), weights.end());
    int ans = r;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (possible(weights, mid, days)) {
            ans = mid;
            r = mid - 1;
        }
        else
            l = mid + 1;
    }
    return ans;
}
```

VD1: ***Tìm phần tử trong mảng xoay***.

Cho nums là một mảng xoay. Cho một target; nếu target tồn tại trong nums, trả về vị trí của target; nếu không, trả về -1.

Input	Output
4 5 6 7 0 1 2 0	4
4 5 6 7 0 1 2 0	-1

Ý tưởng:

- Ở bước đầu, ta có 3 trường hợp của m như sau:
  - Trường hợp 1: m = k.

```
4  5  6  7 | 0  1  2
l        m |       r
```

    - Nếu như nums[l] ≤ target ≤ nums[m], ta biết được target có thể nằm trong đoạn tăng dần bên trái, ta tăng giảm m và l theo ý tưởng gốc của binary search.
    - Nếu không, ta cũng biết được target có thể nằm trong đoạn tăng dần bên phải, ta tăng giảm m và r theo ý tưởng gốc của binary search.
  - Trường hợp 2: m > k.

```
4  5  6 | 0  1  2  3
l       m |       r
```

    - Nếu như nums[m] ≤ target ≤ nums[r], ta theo ý tưởng gốc.
    - Nếu không, ta vẫn theo ý tưởng gốc.
  - Trường hợp 3: m < k.

```
2  3  4  5  6 | 0  1
l        m    |    r
```

    - Nếu như nums[l] ≤ target ≤ nums[m], ta theo ý tưởng gốc.
    - Nếu không, ta vẫn theo ý tưởng gốc. Ở đây, ta không quan tâm đến việc đoạn nửa phải ({6; 0; 1}) đã được xếp tăng dần hay chưa, vì khi gọi đệ quy sẽ lại quy về 2 trường hợp trên.
- Trường hợp 1 và 3 khá giống nhau, ta nên tổng quát hóa thành điều kiện nums[l] ≤ nums[m]. Như vậy, ta không cần phải tìm k.

Code mẫu:

```
int search(vector<int>& nums, int target) {
    int l = 0, r = nums.size()-1;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (nums[mid] == target)
            return mid;
        else if (nums[l] <= nums[mid]) {
            if (nums[l] <= target && target <= nums[mid])
                r = mid - 1;
            else
                l = mid + 1;
        } else {
            if (nums[mid] <= target && target <= nums[r])
                l = mid + 1;
            else
                r = mid - 1;
        }
    }
```

```
    return -1;
}
```

VD2: ***Tìm min trong mảng xoay***.

Cho nums là một mảng xoay, nums[i] ≤ 5000. Hãy tìm min trong nums.

Input	Output
4 5 6 7 0 1 2	0

Ý tưởng:

- Nhận xét: Tương tự bài toán trên.
- Nếu nums[l] ≤ nums[mid], ta đang ở phần đầu, ans có thể là nums[l]. Sau đó, ta thử cho l = mid + 1 để xem min có thể xuất hiện ở phần sau không.
- Nếu nums[l] > nums[mid], ta đang ở phần sau, ans có thể là nums[mid]. Sau đó, ta thử cho r = mid – 1 để xem min có thể xuất hiện ở phần đầu không.

Code mẫu:

```
int findMin(vector<int>& nums) {
    int ans = 5000;
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (nums[l] <= nums[mid]) {
            ans = min(ans, nums[l]);
            l = mid + 1;
        } else {
            ans = min(ans, nums[mid]);
            r = mid - 1;
        }
    }
    return ans;
}
```

7. Các thuật toán sắp xếp.

Selection sort:

```
for (int i = 0; i < n-1; i++) {
    int indexOfMin = i;
        for (int j = i + 1; j < n; j++) {
            if (A[j] < A[indexOfMin]) {
                indexOfMin = j;
            }
        }
    swap(A[i], A[indexOfMin]);
}
```

Sequential insertion sort:

```
void insertionSort(int *a, int n) {
    for (int i = 1; i < n; i++) {
        int selected = a[i];
        int index = i;
        while (index > 0 && a[index - 1] > key) {
            a[index] = a[index - 1];
            index--;
        }
        a[index] = selected;
    }
}
```

Binary insertion sort:

```
int binarySearch(int *a, int l, int r, int x) {
    int res = l;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (x > a[mid]) {
            res = mid;
            l = mid + 1;
        }
        else
            r = mid - 1;
    }
    return res + 1;
}

void binaryInsertionSort(int *a, int n) {
    for (int i = 1; i < n; i++) {
        int selected = a[i];
        int pos = binarySearch(a, 0, i, selected);
        for (int j = i; j > pos; j--) {
            a[j] = a[j - 1];
        }

        a[pos] = selected;
    }
}
```

Merge sort:

```
void merge(int *a, int l, int mid, int r) {
    int i = l, j = mid + 1;

    vector<int> temp;

    while(i <= mid && j <= r) {
        if(a[i] <= a[j]) {
            temp.push_back(a[i]);
            i++;
        } else {
            temp.push_back(a[j]);
            j++;
        }
    }

    while(i <= mid) {
        temp.push_back(a[i]);
        i++;
    }

    while(j <= r) {
        temp.push_back(a[j]);
        j++;
    }

    for(int k = l; k <= r; k++) {
        a[k] = temp[k];
    }
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int mid = (l + r) / 2;
        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, r);
        merge(arr, l, mid, r);
    }
}
```

Quick sort:

```
void QuickSort(int a[], int left, int right) {
    int i, j, pivot;
    pivot = a[(left + right) / 2];
    i = left; j = right;
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            Swap(a[i], a[j]);
            i++; j--;
        }
    }
    if (left < j) QuickSort(a, left, j);   // Gọi ds con từ left đến j
    if (i < right) QuickSort(a, i, right); // Gọi ds con từ i đến right
}
```

Merge sort:

```
void Heapify(int a[], int n, int i) {
    int childLeft = i*2 + 1;
    int childRight = i*2 + 2;
    int max = i;

    if(childLeft < n && a[max] < a[childLeft])
        max = childLeft;
    if(childRight < n && a[max] < a[childRight])
        max = childRight;

    if(max != i) {
        swap(a[max], a[i]);
        Heapify(a, n, max);
    }
}

void buildHeap(int a[], int n) {
    for(int i = n / 2 - 1; i >= 0; i--)
        Heapify(a, n, i);
}

void HeapSort(int a[], int n) {
    buildHeap(a, n);

    for(int i = n - 1; i > 0; i--) {
        swap(a[0], a[i]);
        Heapify(a, i, 0);
    }
}
```

VD1: ***MEX - Minimum Excluded Value***.

Cho một danh sách có n (n > 0) các số tự nhiên. Hãy xây dựng hàm in ra số tự nhiên nhỏ nhất không nằm trong mảng đó.

Input	Output
5 0 1 9 4 4	3
5 1 5 3 4 5	0

Ý tưởng:

- Nhận xét: Cho 1 dãy các số tự nhiên, số tự nhiên nhỏ nhất luôn thuộc đoạn $[0; n-1]$. Trong trường hợp trong dãy không bị khuyết số nào, $max = n - 1$, đáp án sẽ là n.
- Đáp án chính là con số đầu tiên không xuất hiện trong mảng trong thuật toán Counting sort.

Code mẫu:

```
int MEX(vector <int>& A) {
    int maxVal = *max_element(A.begin(), A.end());
    vector <bool> exists(maxVal + 2, false);

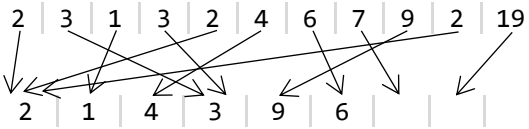
    for (int num : A)
        if (num >= 0)
            exists[num] = true;

    for (int i = 0; i <= maxVal; i++)
        if (!exists[i])
            return i;

    return A.size();
}
```

VD2: ***Sắp xếp theo thứ tự được cho trước***.

Cho 2 vector: arr1 và arr2 (|arr2| < |arr1|; 0 ≤ arr1[i], arr2[i] ≤ 1000). Tất cả những phần tử của arr2 đều phân biệt và thuộc arr1.
Hãy sắp xếp arr1 sao cho *vị trí của các phần tử trong arr1 đều tương ứng với vị trí các phần tử trong arr2*, nếu không tương ứng, nếu có một phần tử của arr1 không thuộc arr2, chèn chúng *vào sau arr1 theo thứ tự tăng dần*.

Input	Output	Giải thích
2 3 1 3 2 4 6 7 9 2 19 2 1 4 3 9 6	2 2 2 1 4 3 3 9 6 7 19	
28 6 22 8 44 17 22 28 8 6	22 28 8 6 17 44	

Ý tưởng:

- Đây là một biến thể của counting sort: Thay vì sắp xếp các phần tử tăng dần, bài toán đòi hỏi sắp xếp theo thứ tự được cho trước (arr2).
- Ta vẫn sẽ ánh xạ theo chiều tăng dần, nhưng sẽ sửa phần tử của arr1 theo chiều của arr2. Để làm được điều này, ta thông qua 2 bước:
  - Tạo mảng **`bool existsOnArr2[]`** tương ứng với `count`, nếu `existsOnArr2[num]` là `true` thì `num` có tồn tại trong `arr2`.
  - Sau khi chèn lại vào `arr1` các số theo thứ tự `arr2`, ta chèn thêm những số mà không tồn tại trong `arr2`.

Code mẫu:

```
vector<int> relativeSortArray(vector <int>& arr1, vector <int>& arr2) {
    // Xây dựng existsOnArr2
    int maxVal = *max_element(arr1.begin(), arr1.end());

    vector <bool> existsOnArr2(maxVal + 1, false);
    for (int num : arr1)
        existsOnArr2[num] = (find(arr2.begin(), arr2.end(), num) != arr2.end());

    // Xây dựng count
    vector <int> count(maxVal + 1, 0);

    for (int num : arr1)
        count[num]++;

    int index = 0;

    // Xây dựng lại arr1
    for (int num : arr2)
        while (count[num] > 0) {
            arr1[index++] = num;
            count[num]--;
        }

    // Điền những phần tử không tồn tại trong arr2 vào sau arr1
    for (int i = 0; i <= maxVal; i++)
        if (!existsOnArr2[i])
            while (count[i] > 0) {
                arr1[index++] = i;
                count[i]--;
            }

    return arr1;
```

```
}
```

VD1: ***Array Partition***.

Cho một danh sách có n (n > 0, n ⋮ 2) các số nguyên. Chia danh sách này ra thành n / 2 cặp (a, b). Hãy tìm max(sum(min(a, b))).

Input	Output	Giải thích
4 1 4 3 2	4	Có những cách chia như sau: • (1; 4); (3; 2) → sum(min(1; 4), min(3; 2)) = 3. • (1; 3); (2; 4) → sum(min(1; 3), min(2; 4)) = 3. • (1; 2); (3; 4) → sum(min(1; 2), min(3; 4)) = 4. Vậy, max = 4.

Ý tưởng:

- Nhận xét:
  - Không thể dùng chiến lược vét cạn.
  - Xét min(a, b), biểu thức này sẽ nhận giá trị nhỏ nhất. Muốn sum là max, thì a, b cũng phải là lớn nhất.
- Như vậy, ta sắp xếp mảng theo thứ tự tăng dần rồi lấy tuần tự phần tử nhỏ hơn ở từng cặp.

Code mẫu:

```
int arrayPairSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    int sum = 0;
    for (int i = 0; i < nums.size(); i+=2) {
        sum += nums[i];
    }
    return sum;
}
```

VD2: ***Minimum Number of Moves to Seat Everyone***.

Có n ghế ngồi và n học sinh đang đứng trước những cái ghế. Cho mảng seats và students, seats[i] là vị trí của ghế ngồi thứ i, students[i] là vị trí của học sinh thứ i.
Mỗi học sinh chỉ có thể di chuyển đến ghế liền cạnh ghế phía sau mình. Ví dụ, học sinh đang đứng ở vị trí x chỉ có thể di chuyển sang vị trí x + 1 hoặc x – 1.
Hãy tìm số bước di chuyển ít nhất để mỗi học sinh đều có ghế ngồi.

Input	Output	Giải thích
3 1 5 2 7 4	4	1. Di chuyển học sinh 0 từ 2 sang 1. 2. Di chuyển học sinh 2 từ 4 sang 3. 3. Di chuyển học sinh 1 từ 7 sang 6, từ 6 sang 5. Tổng cộng có 4 bước.
4 1 5 9 1 3 2 6	7	

Ý tưởng:

- Việc quan trọng là tìm độ lệch giữa các phần tử của hai mảng với nhau sao cho nhỏ nhất.
- Việc sắp xếp các phần tử của 2 mảng này lại với nhau theo một chiều sẽ giúp các phần tử có độ lớn tương tự nhau trong mảng của chúng được đặt cạnh nhau. Khi đó, ta chỉ cần lấy độ lệch tương ứng giữa các phần tử đó là xong.

Code mẫu:

```
int minMovesToSeat(vector<int>& seats, vector<int>& students) {
    sort(seats.begin(), seats.end());
    sort(students.begin(), students.end());

    int cnt = 0;
    for (int i = 0; i < seats.size(); i++) {
        cnt += abs(seats[i] - students[i]);
    }
     return cnt;
}
```

II. LÝ THUYẾT.

Thuật toán	Độ phức tạp			Ý tưởng
	Tốt nhất	Trung bình	Xấu nhất	
Linear search	$O(1)$	$O(n)$		Xét duyệt từ đầu tới cuối danh sách.
Binary search	$O(1)$	$O(\log n)$		Chia danh sách thành 2 nửa để xét.
Selection sort	$O(n^2)$			Lần lượt đưa phần tử nhỏ nhất trong đoạn unsorted lên đầu đoạn unsorted.
Insertion sort	$O(n)$	$O(n^2)$		Lần lượt đưa phần tử đầu tiên của unsorted vào sorted sao cho sorted giữ nguyên thứ tự.
Merge sort	$O(n.\log n)$			***Top-down***: - *Devide*: Chia mảng thành từng cặp nhỏ có 1 phần tử. - *Merge*: Hợp nhất các mảng con theo thứ tự. ***Bottom-up***: - Ngầm chia mảng từng mảng con có 1 phần tử. - Lần lượt nhóm từng cặp mảng con liên tiếp với nhau. ***Chú ý***: Cả 2 cách đều tốn $O(n)$ ***bộ nhớ phụ***.
Quick sort	$O(n.\log n)$		$O(n^2)$	- ***Partition***: Lần lượt đổi chỗ các cặp phần tử đầu tiên bên trái ≥ pivot với phần tử đầu tiên bên phải ≤ pivot cho đến khi nửa trái ≤ pivot và nửa phải ≥ pivot. - Gọi đệ quy quick sort trên nửa trái và nửa phải pivot. - Trong đó, tốn $O(\log n)$ lần partition, mỗi partition tốn $O(n)$. - Việc chọn pivot là phần tử có giá trị trung tâm của dãy sẽ giúp tiết kiệm thời gian.
Heap sort	$O(n)$	$O(n.\log n)$		- ***Hapify*** tại i: Nếu $\lvert con_{max}\rvert < \lvert cha\rvert$ thì hoán đổi con và cha rồi hapify tại con sau khi hoán đổi. - Trong đó, tốn $O(n)$ lần hapify, mỗi hapify tốn $O(\log n)$.

Phân loại sort:

1. **Stable sort**: Sau khi sắp xếp, các phần tử có giá trị bằng nhau trong danh sách vẫn giữ nguyên vị trí tương ứng so với trước khi sắp xếp.
   VD: ***Insertion sort***, ***Merge sort***.
2. **Online sort**: Ciệc so sánh được thực hiện tuần tự theo chiều dài mảng. Có nghĩa là, một mảng có thể vừa được sắp xếp vừa được bổ sung dữ liệu.
   VD: ***Insertion sort***.
3. **Internal sort**: Toàn bộ dữ liệu nằm hoàn toàn trong bộ nhớ chính (RAM) trong suốt quá trình thực hiện sắp xếp.
   VD: Các thuật toán trên.

Selection sort: Lần lượt đưa phần tử nhỏ nhất trong đoạn unsorted lên đầu đoạn unsorted.

Bước	2	4	1	3	6	5
1	1	2	4	3	6	5
2	1	2	4	3	6	5
3	1	2	3	4	6	5
4	1	2	3	4	6	5
5	1	2	3	4	5	6

Insertion sort: Lần lượt đưa phần tử đầu tiên của unsorted vào sorted sao cho sorted giữ nguyên thứ tự.

Bước	2	4	1	3	6	5
1	2	4	1	3	5	6
2	1	2	4	3	5	6
3	1	2	3	4	5	6
4	1	2	3	4	5	6
5	1	2	3	4	5	6

Top-down merge sort:

1. Chia mảng thành từng cặp nhỏ có 1 phần tử.
2. Hợp nhất các mảng con theo thứ tự.

Chia	2	4	1	3	6	5
	2	4	1	3	6	5
	2	4	1	3	6	5
	2	4	1	3	6	5
Hợp	2	1	4	3	5	6
	1	2	4	3	5	6
	1	2	3	4	5	6

Bottom-up merge sort: Lần lượt nhóm từng cặp mảng con liên tiếp với nhau, vừa nhóm vừa sắp xếp.

Width	2	4	1	3	6	5
1	2	4	1	3	5	6
2	1	3	2	4	5	6
3	1	2	3	4	5	6

Hoare quick sort:

1. Partition:
    a. Chọn pivot.
    b. Từ trái sang, chọn phần tử đầu tiên ≥ pivot (M).
    c. Từ phải sang, chọn phần tử đầu tiên ≤ pivot (m).
    d. Nếu (M) nằm trước hoặc trùng (m) thì hoán đổi 2 phần tử đó.
    e. Tiếp tục dịch M và m để hoán đổi cho đến khi không thể
    f. Lặp lại cho đến khi nào không phân vùng được nữa.
2. Lấy móc giữa m và M làm ranh giới phân chia mảng thành 2 mảng con. Tiếp tục partion với từng mảng con này.

	6	2	5	1	4	3	7
Chọn pivot:	6	2	5	1	4	3	7
	6_M	2	5	$_m1$	4	3	7
	1_m	2_M	5	6	4	3	7
Chọn pivot:	1	2	5	6	4	3	7
	1	2	5	6_M	4	$_m3$	7
	1	2	5	3	$_m4$	6_M	7
Chọn pivot:	1	2	5	3	4	6	7
	1	2	5_M	3	$_m4$	$_m6_M$	7
	1	2	4	$_m3$	5_M	6	7
Chọn pivot:	1	2	4	3	5	6	7
	1	2	4_M	$_m3$	5	6	7
	1	2	$_m3$	4_M	5	6	7
Chọn pivot:	1	2	3	4	5	6	7
	1	$_m2_M$	3	4	5	6	7
	1	2	3	4	5	6	7

Heap sort:

1. ***Heapify tại phần tử i***:
    a. Tìm max giữa |cha|, |con trái| (2 cha + 1) và |con phải| (2 cha + 2). Nếu max này không phải cha:
        i. Hoán đổi cha và max.
        ii. Heapify tại max.
2. ***Quá trình sort***:
    a. ***Build heap***: Heapify các phần tử từ n / 2 – 1 đến 0.
    b. ***Sort***: Duyệt các đỉnh từ n – 1 tới 1.
        i. Hoán đổi phần tử 0 và i.
        ii. Heapify tại i.

	i	*0*	*1*	*2*	*3*	*4*	*5*
		6	4	2	5	1	3
Build heap	*Heapify*	6	4	2_p	5	1	3_l
		6	4	3	5	1	2_p
	Heapify	6	4_p	3	5_l	1_r	2
		6	5	3	4_p	1	2
	Heapify	6_p	5_l	3_r	4	1	2
Sort		6	5	3	4	1	2
	Swap	2	5	3	4	1	6
	Heapify	2_p	5_l	3_r	4	1	6
		5	2_p	3	4_l	1_r	6
		5	4	3	2_p	1	6
	Swap	1	4	3	2	5	6
	Heapify	1_p	4_l	3_r	2	5	6
		4	1_p	3	2_l	5	6
		4	2	3	1_p	5	6
	Swap	1	2	3	4	5	6
	Heapify	1_p	2_l	3_r	4	5	6
		3	2	1_p	4	5	6
	Swap	1	2	3	4	5	6
	Heapify	1_p	2_l	3	4	5	6
		2	1_p	3	4	5	6
	Swap	1	2	3	4	5	6

ĐỀ THI UIT.

I. LÝ THUYẾT.

Câu 1 a (0.25 điểm) [*HK1 23-24*]
Xét độ phức tạp theo BigO, thuật toán tìm nhị phân tốt hơn thuật toán tìm tuyến tính, vì sao?

Giải:
Thuật toán tìm kiếm nhị phân có độ phức tạp $O(\log n)$, thuật toán tìm kiếm tuyến tính có độ phức tạp $O(n)$, cao hơn tìm kiếm nhị phân.

Câu 1 (1.25 điểm) [*HK1 24-25 1*]
Hãy liên kết các đặc trưng ở cột bên trái với các thuật toán tương ứng ở cột bên phải. Các thuật toán có thể có những đặc trưng giống nhau.

	Đặc trưng
1	Độ phức tạp của thuật toán trong trường hợp xấu nhất là $O(n.\log n)$.
2	Độ phức tạp của thuật toán trong trường hợp trung bình là $O(n^2)$.
3	Thuật toán được phân loại là Online Sorting.
4	Thuật toán sắp xếp không dựa trên kết quả so sánh giá trị (khóa) giữa các phần tử trong danh sách.
5	Thuật toán có số lượng phép so sánh trong mọi trường hợp (xấu nhất, trung bình, tốt nhất) đều như nhau.
6	Thuật toán có số lần hoán vị ít (xấu nhất là n-1 lần).
7	Thuật toán được thiết kế theo chiến lược chia để trị.

Thuật toán sắp xếp	
A	Selection sort.
B	Insertion sort.
C	Quick sort.
D	Merge sort.
E	Heap sort.
F	Counting sort.
G	Radix sort.

Giải:

	Đặc trưng
1	Độ phức tạp của thuật toán trong trường hợp ***xấu nhất là*** $O(n.log\ n)$.
2	Độ phức tạp của thuật toán trong trường hợp ***trung bình là*** $O(n^2)$.
3	Thuật toán được phân loại là ***Online Sorting***.
4	Thuật toán sắp xếp ***không dựa trên kết quả so sánh*** giá trị (khóa) giữa các phần tử trong danh sách.
5	Thuật toán có số lượng phép so sánh trong mọi trường hợp (xấu nhất, trung bình, tốt nhất) đều như nhau.
6	Thuật toán có số lần hoán vị ít (xấu nhất là n-1 lần).
7	Thuật toán được thiết kế theo chiến lược ***chia để trị***.

	Thuật toán sắp xếp
A	Selection sort.
B	Insertion sort.
C	Quick sort.
D	Merge sort.
E	Heap sort.
F	Counting sort.
G	Radix sort.

Lưu ý: Không bắt buộc trả lời đúng tính chất 4.

Câu 1 (1.5 điểm) [*HK2 22-23*]

1. Các thuật toán sắp xếp nào, trong số các thuật toán insertion sort, heap sort, merge sort và quick sort, được thiết kế theo chiến lược chia để trị? (0.5 điểm)
2. Cho dãy số A = {1, 2, 3, 4, 5, 6, 7, 8, 9}. Hãy cho biết:
    a. Những thuật toán sắp xếp nào, trong số các thuật toán heap sort, quick sort và merge sort, sẽ gặp thuận lợi khi sắp dãy số A theo thứ tự giảm dần? (0.5 điểm)
    b. Thuận lợi đó, theo từng thuật toán đã trả lời ở câu 1.2 a, là gì? (0.5 điểm)

Giải:

1.
Merge sort, Quick sort.

2.

4. Heap sort: Do dãy đã được sắp xếp tăng dần nên không cần build heap để tìm min.
5. Quick sort: Do dãy đá được sắp xếp tăng dần nên luôn chọn được phần tử trung vị hoặc gần trung vị khi lấy pivot là phần tử giữa mảng

II. MÔ PHỎNG THUẬT TOÁN.

1. Tìm kiếm.

Câu 1 b (0.25 điểm) [*HK1 23-24*]

6. Thuật toán tìm nhị phân với thứ tự tăng dần sẽ có kết quả tìm khóa **4** trong mảng gồm các khóa 1, 2, 3, 4, 5, 6, 7, 8, 9 sau bao nhiêu lần so sánh giá trị khóa?

Giải:

Số bước	1	2	3	**4**	5	6	7	8	9
1	l				mid				r
2	l	mid		r					
3			l mid	r					
4				l mid r					

Đáp án: 4.

2. Sắp xếp.

Câu 1 (2.5 điểm) [*HK2 18-19*]

a. Trình bày các bước giải thuật sắp xếp Quick sort (không viết chương trình) để sắp xếp mảng số nguyên N phần tử giảm dần, cho biết độ phức tạp giải thuật. (1.5 điểm)

b. Trình bày các bước (vẽ từng bước) áp dụng giải thuật trong câu 1.a để sắp xếp mảng số nguyên {10, 5, 30, 70, 40, 80, 90} giảm dần. (1 điểm)

Giải:

a.

Các bước:

1. Chọn pivot là phần tử trung vị của danh sách từ vị trí l đến r.
2. Lần lượt hoán đổi các phần tử xung quanh pivot sao cho các phần tử bên trái > pivot, các phần tử bên phải < pivot.
3. Gọi đệ quy từ l đến vị trí pivot, từ vị trí pivot đến r.

Độ phức tạp: $O(\log n)$.

b.

Chọn pivot	10	5	30	70	40	80	90
	10_i	5	30	70	40	80	$_j90$
	90	5_i	30	70	40	$_j80$	10
	90	80	30_i	$_j70$	40	5	10
	90	80	$_j30$	70_i	40	5	10
Chọn pivot	90	80	70	30	40	5	10
	90	$_j80_i$	70	30_i	$_j40$	5	10
	90	80	70	40	30_i	5	10
Chọn pivot	90	80	70	40	30	5	10
	90	80	70	40	30	5_i	$_j10$
	90	80	70	40	30	10	5

Câu 1 b c (1.75 điểm) [*HK2 21-22*]

1. Viết hàm sắp xếp mảng 1 chiều giảm dần với thuật toán Insertion sort (0.75 điểm)
2. Chạy từng bước thuật toán đã viết ở trên với dãy số sau: 3, 8, 4, 5, 9, 1, 2, 6 (1 điểm)

Giải:

```
void InsertionSort(int A[], int n) {
    for (int i = 1; i < n; i++) {
        int x = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] < x) {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = x;
    }
}
```

Bước	3	8	4	5	9	1	2	6
1	8	3 |	4	5	9	1	2	6
2	8	4	3 |	5	9	1	2	6
3	8	5	4	3 |	9	1	2	6
4	9	8	5	4	3 |	1	2	6
5	9	8	5	4	3	1 |	2	6
6	9	8	5	4	3	2	1 |	6
7	9	8	6	5	4	3	2	1 |

Câu 2 (0.5 điểm) [*HK1 23-24*]
Nếu dùng thuật toán Insertion Sort (Chèn trực tiếp) để sắp xếp một dãy số tăng dần, thì khi hoàn thành việc sắp xếp mảng A={**3,2,4,5,1,5**}, số lần dịch chuyển phần tử sang phải một vị trí là bao nhiêu?

Giải:

Số lần dịch chuyển	3	2	4	5	1	5
1	2	3 |	4	5	1	5
1	2	3	4 |	5	1	5
1	2	3	4	5 |	1	5
5	1	2	3	4	5 |	5
5	1	2	3	4	5	5 |

Đáp án: 5.

Câu 2 (0.5 điểm) [*HK1 24-25 1*]
Cho một mảng gồm 7 số nguyên như sau: **4, 9, 5, 6, 10, 2, 3**. Hãy cho biết mảng sẽ thay đổi qua từng bước như thế nào khi áp dụng thuật toán Quick Sort theo mã giả bên dưới, để sắp xếp mảng theo thứ tự **giảm dần**.

Mã giả:

```
void QuickSort(int a[], int left, int right)
      int i, j, x;
      x = a[(left+right)/2];
      i = left; j = right;
      while(i < = j)

      while(a[i] > x) i++;
      while(a[j] < x) j--;

      if(i <= j)
            Doicho(a[i],a[j]);
            i++ ; j--;

      if(left<j) QuickSort(a, left, j);
      if(i<right) QuickSort(a, i, right);
```

Giải:

Chọn pivot:	4	9	**5**	6	10	2	3
	10	9	5	6	4	2	3
	10	9	6	5	4	2	3
Chọn pivot:	10	**9**	6	5	**4**	2	3
	10	9	6	5	4	2	3
Chọn pivot:	10	9	6	5	4	**2**	3
	10	9	6	5	4	3	2

Câu 14 (1 điểm) [*HK1 23-24*]
Quỳnh được giao công việc cài đặt một hàm nhận một danh sách số nguyên và trả về một tập hợp số nguyên (các phần tử không trùng nhau). Các phần tử trong tập hợp được sắp thứ tự tăng dần.

Ví dụ: Input là 1, 5, 2, 7, 4, 1, 3, 2, 3, 5 thì output là 1, 2, 3, 4, 5, 7.

Quỳnh sử dụng hàm sort, có độ phức tạp $O(n.\log n)$, và cài đặt hàm **`ListToSet`** như sau.

```
vector <int> ListToSet(vector <int> A) {

    vector <int> r;
    long long unsigned int i, j;

    for (i = 0; i < A.size(); i++) {
        for (j = 0; j < i; j++) {
            if (A[j] == A[i]) break;
        }
        if (j >= i) r.push_back(A[i]);
    }

    sort(r.begin(), r.end());

    return r;
}
```

Mentor của Quỳnh nói rằng, cách này chưa tốt, phải sắp xếp trước rồi xử lý. Mentor đưa cho Quỳnh hàm **`List2Set`** như bên dưới, và yêu cầu Quỳnh hoàn thành nó sao cho thời gian thực hiện tốt hơn hàm **`ListToSet`**. Bạn cũng phải hoàn thành hàm **`List2Set`** này.

Giải:

```
vector<int> List2Set(vector<int> A) {
    vector<int> r;
    sort(A.begin(), A.end());

    r.push_back(A[0]);

    for (int i = 1; i < A.size(); i++) {
        if (A[i] != A[i-1])
            r.push_back(A[i]);
    }

    return r;
}
```

2. Danh sách liên kết, stack và queue.

Câu 1 (2 điểm) [*HK2 17-18*]
Cho cấu trúc dữ liệu danh sách liên kết đơn như dưới đây:

```
struct Node{
    int info;
    Node *Next;
};

struct List{
    Node *Head ;
    Node *Tail;
};
```

Anh / chị hãy viết hàm **`void RPrint(List ls)`** để in lên màn hình các phần tử trong danh sách ls theo thứ tự từ phần tử cuối đến phần tử đầu (không dùng mảng phụ hoặc danh sách phụ).

Giải:

```
void RPrint(List ls) {
    if (!ls.pHead)
        return;

    int temp = ls.pHead->info;

    ls.pHead = ls->Next;
    RPrint(ls);

    cout << temp << ' ';
}
```

Câu 2 (2 điểm) [*HK2 17-18*]
Cho chuỗi các thao tác như sau:

`RTU*I**MUN***EB***1*ON`

Biết rằng: với mỗi chữ cái tượng trưng cho thao tác thêm chữ cái tương ứng vào Stack (ngăn xếp); với mỗi dấu * tượng trưng cho thao tác lấy nội dung một phần tử trong stack và in ra màn hình. Anh / chị hãy cho biết sau khi hoàn tất chuỗi thao tác trên, những chữ cái nào còn trong stack và những chữ cái nào được in ra màn hình.

Giải:

Màn hình	Stack
	R
	R T
	R T U
U	R T
	R T I
I	R T
T	R
	R M
	R M U
	R M U N
N	R M U
U	R M
M	R
	R E
	R E B
B	R E
E	R
R	
	1
1	
	O
	O N

Màn hình: `UITNUMBER1`.
Trong stack còn lại: `O N`.

Câu 2 (2 điểm) [*HK2 22-23*]
Telegram đã ra đời được 10 năm và được biết đến như là một dịch vụ nhắn tin tức thời miễn phí, đa nền tảng và mã hóa. Màn hình của dịch vụ này chỉ hiển thị tin nhắn của k người sau cùng. Khi có một tin nhắn được gởi đến bạn thì tin nhắn sẽ được hiển thị vào đầu danh sách tin nhắn. Các tin nhắn sẽ được nhóm lại theo số điện thoại (SĐT) của người gởi.
Cho một chuỗi tuần tự các SĐT có gởi tin nhắn cho bạn:

1. Hãy mô tả cấu trúc dữ liệu (CTDL) sử dụng để hiển thị tin nhắn của k người sau cùng và các tin nhắn sẽ được nhóm lại theo SĐT của người gởi (xem mô tả ví dụ input, ouput bên dưới) (0.5 điểm)
2. Viết chương trình bằng C++ hiện thực hoá yêu cầu, sử dụng CTDL trong câu 2.1 (1.5 điểm)

Ví dụ Input:

- Dòng 1: số k.
- Dòng 2: số n: số lượng tin nhắn được gởi đến bạn.
- Dòng 3: danh sách SĐT của n tin nhắn theo thứ tự thời gian từ trước đến sau.

Lưu ý: Giới hạn SĐT chỉ có tối đa 3 chữ số.

Ví dụ Output: Danh sách các SĐT của các tin nhắn hiển thị trên màn hình theo thứ tự SĐT có tin nhắn mới hơn sẽ xuất hiện trước kèm theo số lượng tin nhắn từ SĐT đó.

Input	Output
5 12 903 901 902 904 976 976 973 986 976 904 905 986	986(2) 905 904(2) 976(3) 973

Giải:

1.
Sử dụng cấu trúc stack. Các số điện thoại sẽ được đưa vào và lấy ra từ 1 đầu.

2.

```
#include<iostream>
#include<stack>

using namespace std;

int main() {

    stack <int> s;
    int SL[1000]= {0};

    int n, k, x;
    cin >> k;
    cin >> n;

    for (int i = 0; i < n; i++) {
        cin >> x;
        s.push(x);
        SL[x]++;
    }

    while ((k) && !s.empty()) {
        x = s.top();
        if (SL[x] != 0) {
            cout << x;
            if (SL[x] > 1)
                cout << "(" << SL[x] << ")";
                cout << " ";
                k--;
                SL[x] = 0;
            }
            s.pop();
    }
    return 0;
}
```

Câu 12 (1 điểm) [*HK1 23-24*]
Cho kiểu dữ liệu **`DblStack`**, là một stack chứa các giá trị double, và một hàm **`CreateDblItem`** được khai báo như bên dưới.

```
struct DblStack {
    double data;
    DblStack *next;
};

DblStack* CreateDblItem(double x) {
    DblStack r = new DblStack;
    if (r == NULL) exit(1);
    r->data = x;
    r->next = NULL;
    return r;
};
```

Hãy cài đặt hàm push (0.5đ) và hàm pop (0.5đ) ở bên dưới, lần lượt để đưa một số double vào stack và xóa bỏ phần tử ở đỉnh stack.

Giải:

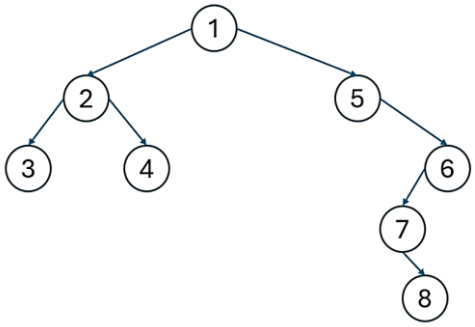
```
void push(DblStack*& stack, double x) {
    DblStack* temp = CreateDblItem(x);
    if (stack)
        temp->next = stack;
    stack = temp;
}

void pop(DblStack*& stack) {
    if (!stack)
        return;

    DblStack* temp = stack;
    stack = stack->next;
    delete temp;
}
```

3. Cây nhị phân, cây nhị phân tìm kiếm.

Câu 3 (0.5 điểm) [*HK1 23-24*]
Cho cây nhị phân T_1:

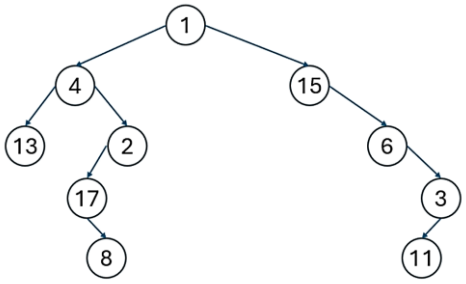


Hỏi khóa **7** xuất hiện ở vị trí thứ mấy (vị trí đầu tiên là 0) trong kết quả duyệt LNR?

Giải:

LNR: 3 2 4 1 5 **7** 8 6.
Đáp án: 5.

Câu 4 (0.5 điểm) [*HK1 23-24*]
Cho cây nhị phân T_2:



Hỏi mức nào trên cây T_2 có tổng giá trị khóa cao nhất?

Giải:

Tổng các mức:

0. 1 = 1.
1. 4 + 15 = 19.
2. 13 + 2 + 6 = 21.
3. 17 + 3 = 20.
4. 8 + 11 = 19.

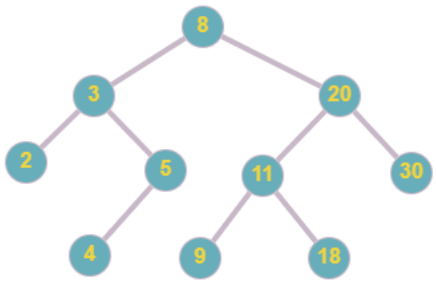
Đáp án: Mức 2.

Câu 3 (1 điểm) [*HK1 24-25 1*]
Xét giải thuật tạo cây nhị phân tìm kiếm, thứ tự các giá trị nhập vào cây T1 như sau: **8, 3, 5, 2, 20, 11, 30, 9, 18, 4**. Yêu cầu:

a) Cho biết kết quả duyệt cây T1 theo thứ tự Right – Left – Node (RLN);.
b) Hãy vẽ lại hình ảnh cây T1 khi xóa 2 lần nút gốc sao cho cây vẫn là cây nhị phân tìm kiếm.

Giải:

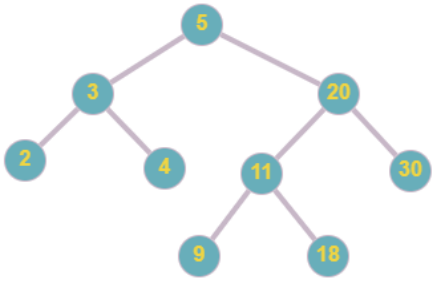
Cây ban đầu:



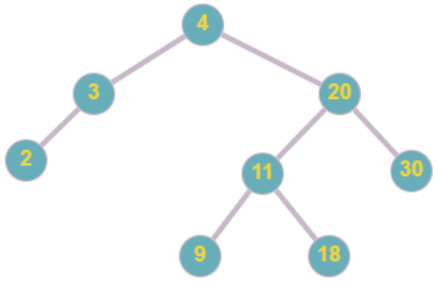
a. 30 18 9 11 20 4 5 2 3 8.

b.

Xóa lần 1:



Xóa lần 2:

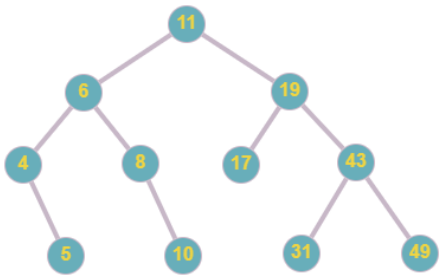


Câu 2 (4 điểm) [*HK2 18-19*]
Cho dãy số sau: 11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31.
Hãy thực hiện các yêu cầu sau:

a. Xây dựng cây nhị phân tìm kiếm từ dãy số đã cho vào cây theo thứ tự thêm các số từ trái sang phải của dãy số (1 điểm)
b. Duyệt cây trong câu 2.a theo Node-Left-Right, Right-Left-Node (1 điểm)
c. Xóa khỏi cây lần lượt các nút 8, 11, 43, 6 (vẽ hình từng trường hợp) sao cho cây vẫn là cây nhị phân tìm kiếm sau khi xoá nút. (1 điểm)
d. Viết hàm in ra màn hình các nút trên cây có duy nhất một nút con (1 điểm)

Giải:

a.

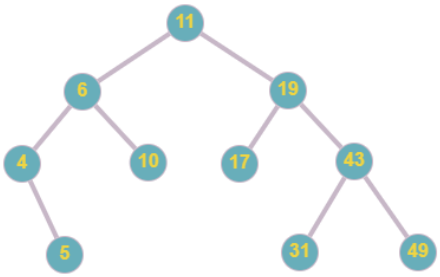


b.

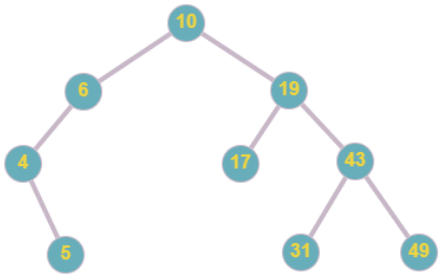
- NLR: 11 6 4 5 8 10 19 17 43 31 49.
- RLN: 49 31 43 17 19 10 8 5 4 6 11.

c.

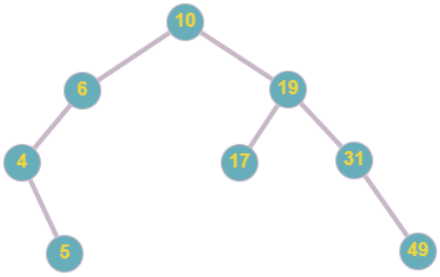
Xóa 8: Nối con 10 vào 6.



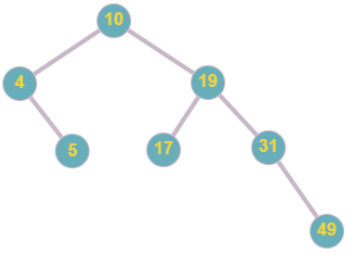
Xóa 11: Thế bằng 10 rồi xóa.



Xóa 43: Thế bằng 31 rồi xóa.



Xóa 6: Nối con 4 vào 10.



d.

```
void haveOneChild(TREE T) {
    if (!T)
        return;

    if (!T->left && !T->right)
        return;

    if (!T->left || !T->right)
        cout << T->key << ' ';

    haveOneChild(T->left);
    haveOneChild(T->right);
}
```

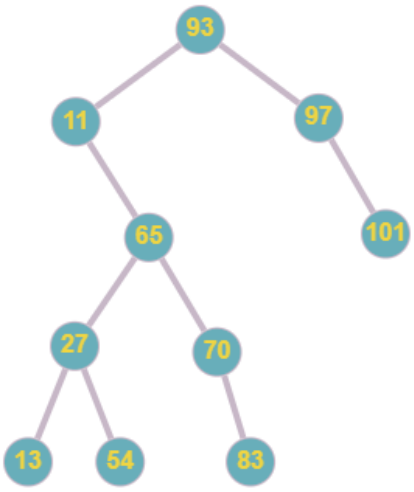
Câu 3 (4 điểm) [*HK2 17-18*]

Cho dãy số như sau : {93, 11, 97, 65, 27, 70, 13, 83, 54, 101}. Anh / chị hãy thực hiện :

a) Xây dựng cây nhị phân tìm kiếm từ dãy số trên lần lượt từ trái sang phải (1.5 điểm)
b) Xóa lần lượt theo thứ tự các nút {54, 11, 93} (1.5 điểm).
c) Viết hàm đếm số nút trên cây chỉ có duy nhất 1 nút con phải (1 điểm)

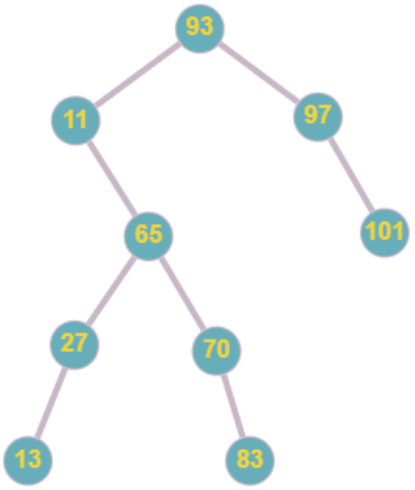
Giải:

a.

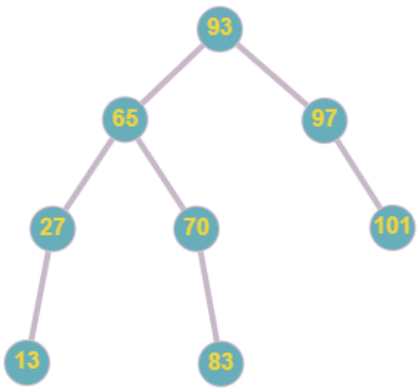


b.

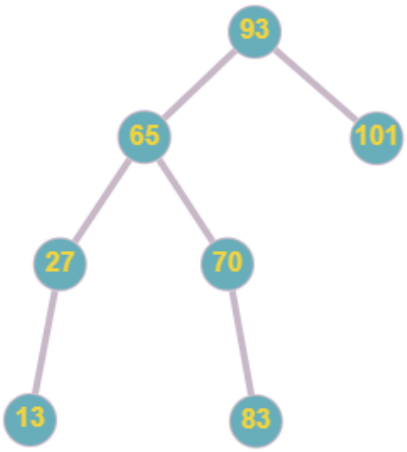
Xóa 54:



Xóa 11: Gắn con 65 với 93:



Xóa 97: Gắn con 101 với 93:



c.

```
int count1LChild(TNODE* T) {
    if (!T)
        return 0;

    return (!T->left && T->right ? 1 : 0) +
           count1LChild(T->right) +
           count1LChild(T->left);
}
```

Câu 1 (3 điểm) [*HK2 21-22*]

Cho dãy ký tự như sau: **R, E, T, A, V, X, L, G, S, I**. Hãy thực hiện các yêu cầu sau:

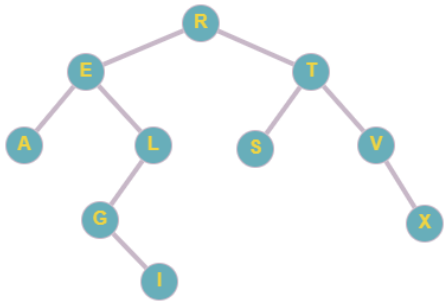
a) Vẽ cây nhị phân tìm kiếm bằng cách thêm lần lượt từng ký tự vào cây theo thứ tự từ trái qua phải của dãy ký tự trên, biết rằng giá trị của từng ký tự tương ứng theo thứ tự xuất hiện của ký tự trong từ điển, biết rằng giá trị của từng ký tự tương ứng theo thứ tự xuất hiện của ký tự trong từ điển như sau: A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z (1 điểm).

b) Cho biết kết qủa duyệt cây theo RNL, NRL (1 điểm)

c) Huỷ lần lượt từng nút L, T, E, R trên cây, mỗi lần huỷ 1 nút vẽ lại cây nối tiếp theo như thứ tự huỷ (1 điểm)

Giải:

a.

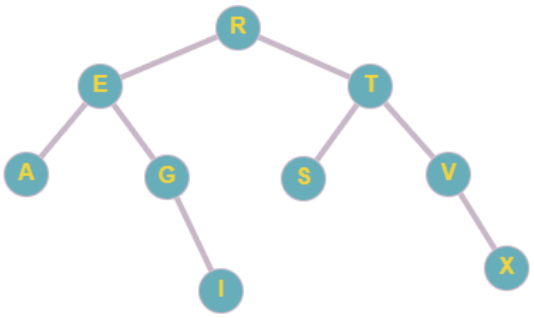


b.

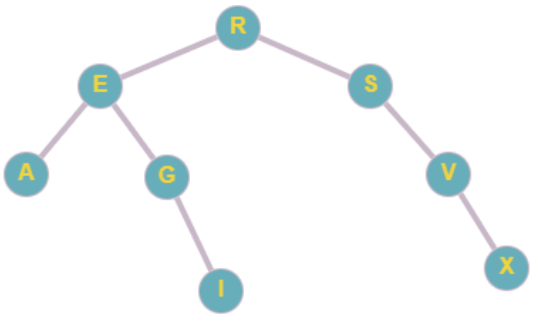
- RNL: X V T S R L I G E A.
- NRL: R T V X S E L G I A.

c.

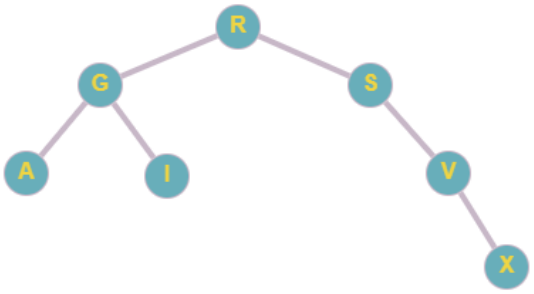
Xóa L: Nối con G vào E:



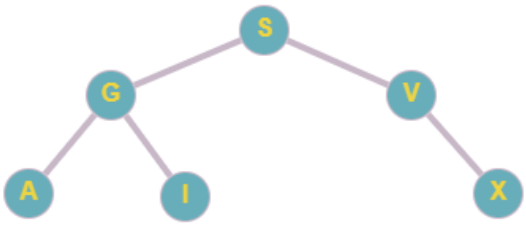
Xóa T: Thế T bằng S rồi xóa.



Xóa E: Thế E bằng G rồi xóa.

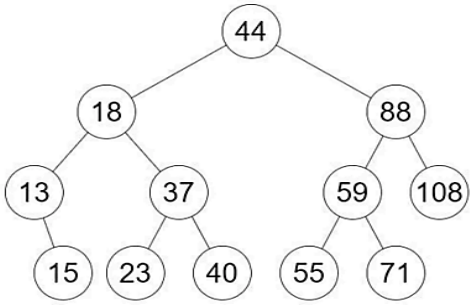


Xóa R: Thế R bằng S rồi xóa.



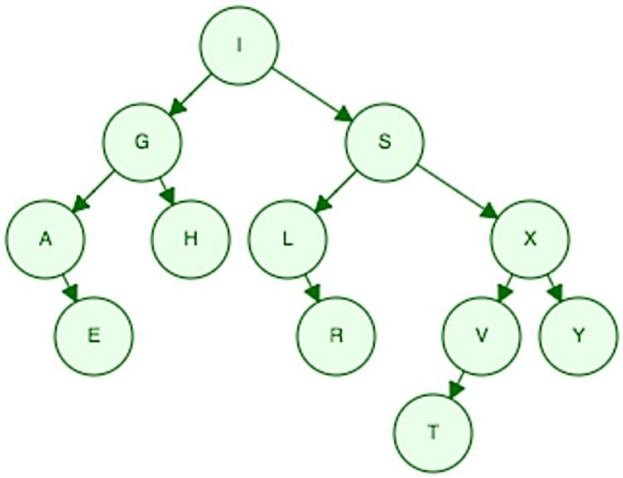
Câu 3 (1.5 điểm) [*HK2 22-23*]

1. Cho dãy ký tự như sau: I, S, G, L, X, V, A, T, E, R, H, Y, hãy vẽ cây nhị phân tìm kiếm khi thêm từng ký tự vào cây theo thứ tự từ trái qua phải của dãy ký tự, biết rằng giá trị của từng ký tự tương ứng theo thứ tự xuất hiện của ký tự trong từ điển như sau: A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z (0.5 điểm)
2. Cho biết kết qủa duyệt cây nhị phân tìm kiếm (hình bên) theo RNL, NRL (0.5 điểm)
3. Viết hàm đếm số nút có 2 nút con trên cây nhị phân tìm kiếm (0.5 điểm)



Giải:

1.



2.

- RNL: 108 88 71 59 55 44 40 37 23 18 15 13.
- NRL: 44 88 108 59 71 55 18 37 40 23 13 15.

3.

```
int count2child(TNODE* T) {
    if (!T)
        return 0;
    else if (T->pRight && T->pLeft)
        return 1 + count2child(T->pRight) + count2child(T->pLeft);
    else
        return count2child(T->pRight) + count2child(T->pLeft);
}
```

Câu 13 (1 điểm) [*HK1 23-24*]
Cho kiểu dữ liệu TREE, để biểu diễn cây nhị phân tìm kiếm, được khai báo như bên dưới. Hãy cài đặt hàm **ChonKhoa** để trả về một danh sách các khóa k trên cây nhị phân tìm kiếm sao cho a < k < b, với a, b là các số cho trước. Yêu cầu dùng queue thay cho hàm đệ quy.

```
struct TNode {
    int key;
    TNode *left, *right;
};

typedef TNode *TREE;
```

Giải:

```
vector <int> ChonKhoa(TREE T, int a, int b) {
    vector <int> ans;

    queue <TREE> qu;
    qu.push(T);

    while (!qu.empty()) {
        TNODE* node = qu.front(); qu.pop();

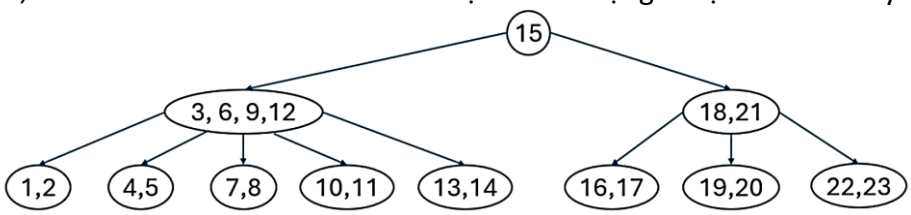
        if (!node)
            continue;

        if (a < node->key && node->key < b)
            ans.push_back(node->key);
        else if (a > node->key)
            qu.push(node->right);
        else
            qu.push(node->left);
    }

    return ans;
}
```

4. Cây B-Tree.

Câu 6 (0.5 điểm) [*HK1 23-24*]
Cho B-Tree bậc 5, T4. Biết thao tác tìm khóa trên một nút sử dụng thuật toán tìm tuyến tính.



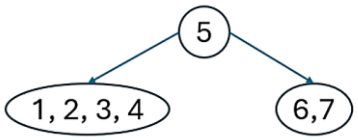
Hỏi số lần so sánh giá trị khóa khi có kết quả tìm khóa **14** trên T_4?

Giải:

Node đang xét	Node
15	{15}
3 6 9 12	{3, 6, 9, 12}
13 14	{13, 14}

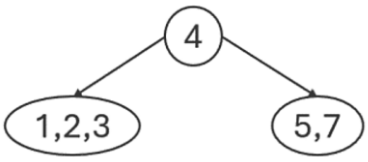
Đáp án: 7.

Câu 5 (0.5 điểm) [*HK1 23-24*]
Cho B-Tree bậc 5 T_3:



Hãy vẽ cây T_3 sau khi xóa khóa 6 (0.25đ)? Và cho biết tên thao tác đã điều chỉnh các nút trên T_3 (0.25đ)?

Giải:
Khi xóa key 6, ta thấy node này đã thiếu key, nhưng do node anh em của nó thừa key nên ta dùng underflow.



Câu 4 (1 điểm) [*HK1 24-25 1*]
Tạo một cây B-Tree T2 bậc 5 với thứ tự thêm các giá trị vào cây như sau: **1, 12, 8, 2, 25, 6, 14, 28, 17**.
Yêu cầu:

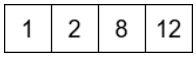
a) Vẽ cây T2 ;
b) Sau đó, lần lượt xoá 2 giá trị là 12 và 25 ra khỏi cây T2, hãy vẽ trạng thái cây sau mỗi lượt xóa. Quy ước: Trong quá trình xóa, ưu tiên thực hiện thủ tục nhường khóa (underflow) trước thủ tục gộp (catenation).

Giải:

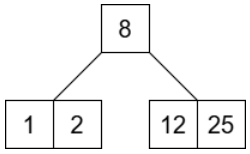
a.

6, 14, 28, 17

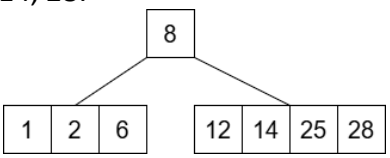
Thêm 1, 12, 8, 2:



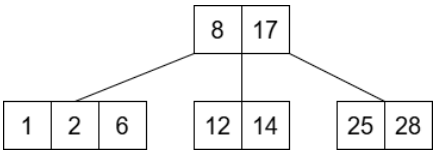
Thêm 25: Tách 8:



Thêm 6, 14, 28:

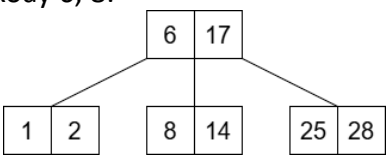


Thêm 17: Tách 17:

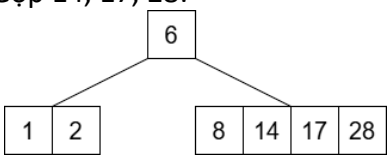


b.

Xóa 12: Xoay 6, 8:



Xóa 25: Gộp 14, 17, 28:



Câu 4 (1.5 điểm) [*HK2 22-23*]

Cho biết B-Tree bậc 5 là một cây đa nhánh thỏa mãn đồng thời tất cả các tính chất sau:

- Tất cả node lá phải nằm trên cùng một mức
- Tất cả các node, trừ node gốc và node lá có tối thiểu là 2 khóa và 3 con.
- Tất cả các node có tối đa là 4 khóa và 5 con
- Một node không phải là lá và có n khóa thì bắt buộc phải có n+1 con.

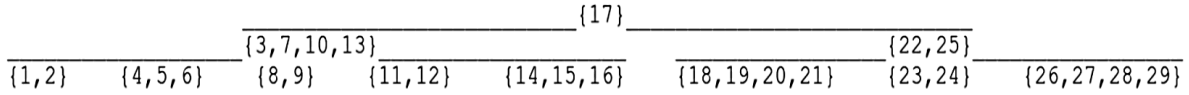
Lần lượt thêm các giá trị sau đây vào cây: 9, 8, 23, 2, 14, 17, 27, 11, 1, 24, 16, 5, 7, 13, 4, 18, 25, 19, 22, 26, 15, 29, 3.

1. Vẽ trạng thái của cây sau khi thêm toàn bộ các giá trị trên. (1 điểm)
2. Cho biết cấu trúc BTree_node và một hàm duyệt cây được định nghĩa như sau:

```
class BTree_node{
    vector<int> keys;
    vector<BTree_node*>children;
};

void dfs2(BTree_node* root){
    stack<BTree_node*> q;
    q.push(root);
    while(q.empty() != true){
        BTree_node* x = q.top(); q.pop();
        for (BTree_node* i : x->keys) cout << i << " ";
        cout << endl;
        for(int i: x->children) q.push(i);
    }
}
```

Với cây B-Tree bậc 5 có hình dáng như bên dưới thì hàm duyệt cây trên sẽ cho ra output là gì? (0.5 điểm)



Giải:

1.



2.

Node	**Stack**
	{17}
{17}	{3, 7, 10, 13}
{22, 25}	{3, 7, 10, 13} {18, 19, 20, 21} {23, 24} {26, 27, 28, 29}
{26, 27, 28, 29}	{3, 7, 10, 13} {18, 19, 20, 21} {23, 24}
{23, 24}	{3, 7, 10, 13} {18, 19, 20, 21}
{18, 19, 20, 21}	{3, 7, 10, 13}
{3, 7, 10, 13}	{1, 2} {4, 5, 6} {8, 9} {11, 12} {14, 15, 16}
{14, 15, 16}	{1, 2} {4, 5, 6} {8, 9} {11, 12}
{11, 12}	{1, 2} {4, 5, 6} {8, 9}
{8, 9}	{1, 2} {4, 5, 6}
{4, 5, 6}	{1, 2}
{1, 2}	

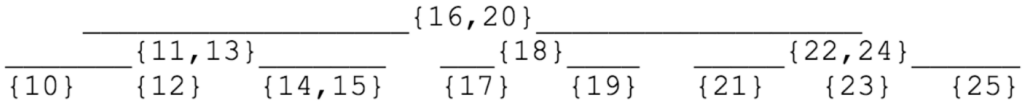
Câu 3 (2 điểm) [*HK2 21-22*]

Cho biết cây B-Tree bậc 3 là một cây thỏa mãn các tính chất sau:

- Tất cả node lá nằm trên cùng một mức.
- Tất cả các node, trừ node gốc và node lá, có tối thiểu 2 node con.
- Tất cả các node có tối đa 3 con.
- Tất cả các node, trừ node gốc, có từ 1 cho đến 2 khóa (keys).
- Một node không phải lá và có n khóa thì phải có n+1 node con.

Hãy thực hiện các yêu cầu sau:

1. Cho dãy số: *12, 17, 20, 23, 15, 11, 24, 13, 19, 22, 18, 21, 16*. Hỏi khi lần lượt thêm các số trong dãy theo thứ tự từ trái qua phải vào một cây B-Tree bậc 3 rỗng thì:
   a. Các khóa nào khi thêm vào cây sẽ làm phát sinh thao tác tách (split) node? (0.5 điểm)
   b. Vẽ cây B-Tree trước và sau khi thêm các khóa ở câu a (1 điểm)
2. Cho cây B-Tree bậc 3 như hình sau:



   Hãy lần lượt tiến hành xóa các khóa sau khỏi cây: *13, 24, 19* và vẽ cây B-Tree trước và sau khi xóa mỗi khóa trên (0.5 điểm)

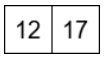
   Lưu ý khi xoá:

   - Khi khóa cần xóa (gọi là x) không nằm ở node lá, chọn khóa thế mạng là khóa có giá trị lớn nhất mà nhỏ hơn x.
   - Thao tác nhường khoá (underflow) sẽ được thực hiện khi hai node liền kề có tổng số khóa ≥ 2. Khi có một node không còn đáp ứng đủ số lượng khóa tối thiểu, ưu tiên thực hiện underflow thay cho catenation (hợp) vì thao tác này không làm thay đổi số khóa của node cha.
   - Khi có 02 lựa chọn node liền kể để thực hiện catenation, ưu tiên chọn catenation giữa node bị thiếu khóa với node liền trước.

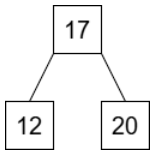
Giải:

1.

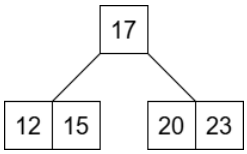
Thêm 12, 17:



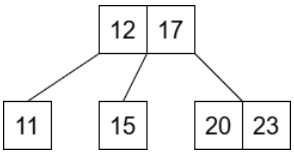
Thêm **20**: Tách 17:



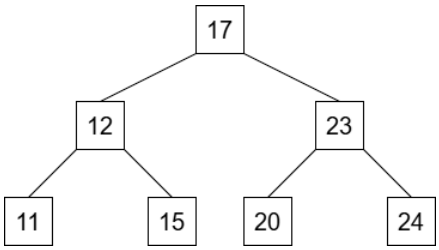
Thêm 23, 15:



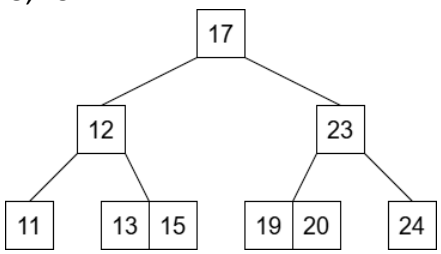
Thêm **11**: Tách 12:



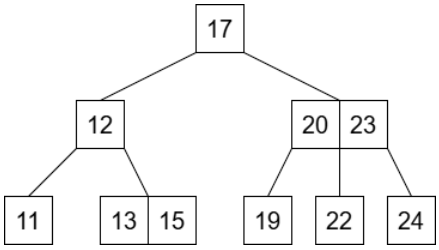
Thêm **24**: Tách 23 → tách 17:



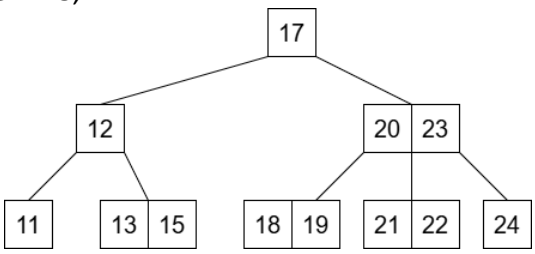
Thêm 13, 19:



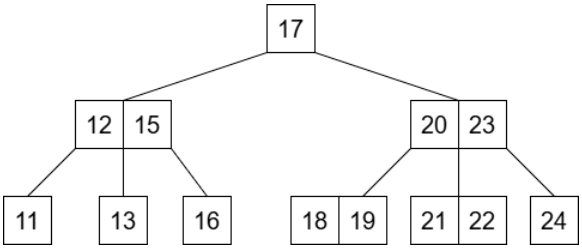
Thêm **22**: Tách 20:



Thêm 18, 21:

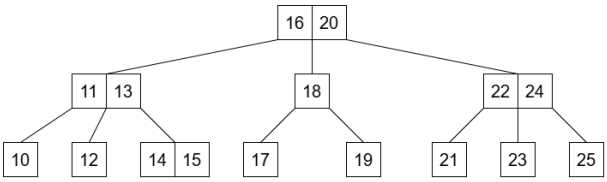


Thêm **16**: Tách 15:



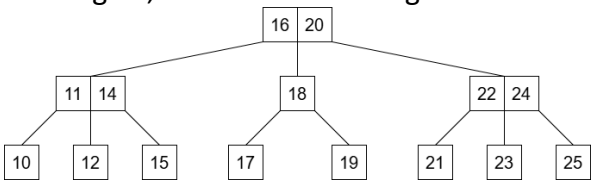
2.

Cây ban đầu:



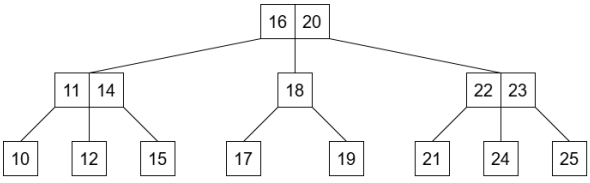
Xóa 13:

Thế bằng 14, và sau đó xóa thẳng 13:

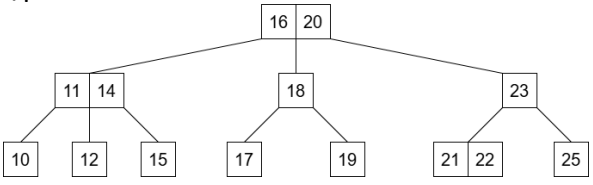


Xóa 24:

Hoán đổi 24 và 23:

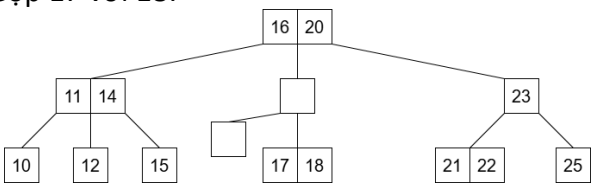


Gộp 21 với 22:

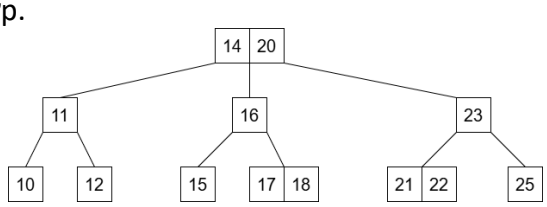


Xóa 19:

Gộp 17 với 18:



Đối với node cha bị thiếu, xoay 14, 16, đồng thời con 15 cũng bị xoay theo và gắn vào vị trí phù hợp.



5. Bảng băm.

Câu 5 (1 điểm) [*HK1 24-25 1*]
Thêm các khoá **37, 28, 24, 7, 71** vào một bảng băm địa chỉ mở HT, có kích thước M = 13, sử dụng hàm băm h_1(key) = key % M. Biết rằng, phương pháp giải quyết xung đột là băm kép (double hashing), với hàm băm phụ h_2(key) = 11 – (key %11), và hàm băm lại h(key, i) = (h_1(key) + i* h_2(key)) % M. Yêu cầu:

a) Hãy trình bày từng bước việc thêm các khóa vào HT bằng cách điền kết quả tính toán vào Bảng 1;
b) Cho biết vị trí thêm các khóa ở Bảng 2.

Bảng 1: Minh họa các bước tính toán trong quá trình thêm 5 khóa trên:

key	h_1(key)	h_2(key)	Kết quả băm lại lần 1 (nếu có)	Kết quả băm lại lần 3 (nếu có)	Kết quả băm lại lần 3 (nếu có)
37					
28					
24					
7					
71					

Bảng 2: Bảng băm HT sau khi thêm 5 khóa trên:

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Key													

Giải:

key	h_1(key)	h_2(key)	Kết quả băm lại lần 1 (nếu có)	Kết quả băm lại lần 3 (nếu có)	Kết quả băm lại lần 3 (nếu có)
37	11				
28	2				
24	11	9	7		
7	7	4	11	2	6
71	6	6	12		

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Key			28				7	24				37	71

Câu 5 (1.5 điểm) [*HK2 22-23*]

1. Cho bảng băm T có các đặc điểm sau:
   - Kích thước bảng băm m=7.
   - Giá trị khóa k là các số nguyên k∈[1,50].
   - Sử dụng hàm băm theo phép chia.
   - Xử lý đụng độ (hay va chạm, collision) theo phương pháp thăm dò tuyến tính.

   Hãy cho biết công thức của hàm băm h(k) và hàm băm lại hi(k), hay h(k, i), sao cho xác suất xảy ra đụng độ trong T không vượt quá n/m khi T có n phần tử. (0.5 điểm)
2. Cho bảng băm T có các đặc điểm sau:
   - Giá trị khóa k là một số nguyên dương.
   - Kích thước bảng băm m=11.
   - Hàm băm h(k) = floor(((0.618 * k) mod 1) * m).
   - Hàm băm lại h(k, i) = floor(((0.618 * (k + i)) mod 1) * m).

   Cho biết:
   - Hàm floor(x) trả về giá trị nguyên lớn nhất nhỏ hơn hoặc bằng x. Ví dụ floor(4.8) = 4.
   - Biểu thức x mod 1 sẽ lấy phần thập phân của x. Ví dụ, 8.492 mod 1 = 0.492.
   - Các số thực chỉ lấy đến ba số sau phần thập phân.

   Giả sử bảng băm T đã chứa các khóa như sau:

Chỉ số	0	1	2	3	4	5	6	7	8	9	10
Khóa			2				9				

   Hãy trình bày quá trình:
   a. Thêm khóa có giá trị 1 vào T. (0.5 điểm)
   b. Giả sử khóa 2 đã bị xóa (ô có chỉ số 2 trên T chứa ký hiệu DELETED). Tìm khóa có giá trị 1 trên T. (0.5 điểm)

Giải:

1.

Theo định lý họ hàm băm phổ quát:

$$h(k) = [(2k + 1)\ \%\ 53]\ \%\ 7, \qquad h_i(k) = [h(k) + i]\%7$$

2.

a.

Ta có:

- $h(1) = floor\left(((0{,}618\ .\ 1)\ \%\ 1)\ .\ 11\right) = 6$, T[6] = 9, băm lại.
- $h(1;\ 1) = floor\left(\left((0{,}618\ .\ (1 + 1))\ \%\ 1\right)\ .\ 11\right) = 2$, T[2] = 2, băm lại.
- $h(1;\ 2) = floor\left(\left((0{,}618\ .\ (1 + 2))\ \%\ 1\right)\ .\ 11\right) = 9$, T[9] trống, gán T[9] = 1.

Chỉ số	0	1	2	3	4	5	6	7	8	9	10
Khóa			2				9			1	

b.
Ta có:

- $h(1) = 6$, T[6] = 9, băm lại.
- $h(1;\ 1) = 2$, T[2] = DELETED, băm lại.
- $h(1;\ 2) = 9$, T[9] = 1, đã tìm thấy.

Câu 7 8 9 (1.5 điểm) [*HK1 23-24*]
Cho bảng băm địa chỉ mở HT_1, có kích thước M=11, hàm băm f và hàm băm lại f' lần lượt là:

$$f(k) = k\ \%\ M, \qquad f'(k,i) = (f(k) + i^2)\ \%\ M$$

Trong đó, phép toán % là phép toán lấy phần dư của phép chia nguyên.

Chỉ số	0	1	2	3	4	5	6	7	8	9	10
Khóa	11	1			15	5				9	

1. Hỏi số lần **so sánh giá trị khóa** khi có kết quả tìm khóa 4 trên HT_1? (0.5 điểm)
2. Hỏi hệ số tải của HT_1 (lấy 2 số lẻ sau phần thập phân)? (0.5 điểm)
3. Hỏi vị trí khi thêm khóa 22? (0.5 điểm)

Giải:

1.
Ta có:

1. $f(4) = 4\ \%\ 11 = 4$, HT[4] = 15, băm lại.
2. $f'(4,1) = (4 + 1^2)\%\ 11 = 5$, HT[5] = 5, băm lại.
3. $f'(4,2) = (4 + 2^2)\%\ 11 = 8$, HT[8] trống, HT[4] = 4.

Đáp án: 3.

2.

$$load_{factor} = \frac{5}{11} \approx 0{,}45$$

3.
Ta có:

1. $f(22) = 22\ \%\ 11 = 0$, HT[4] = 11, băm lại.
2. $f'(22,1) = (22 + 1^2)\%\ 11 = 1$, HT[1] = 1, băm lại.
3. $f'(22,2) = (22 + 2^2)\%\ 11 = 4$, HT[4] = 15, băm lại.
4. $f'(22,3) = (22 + 3^2)\%\ 11 = 9$, HT[9] = 9, băm lại.
5. $f'(22,4) = (22 + 4^2)\%\ 11 = 5$, HT[5] = 5, băm lại.
6. $f'(22,5) = (22 + 4^2)\%\ 11 = 3$, HT[3] trống, HT[3] = 22.

Đáp án: 3.

Câu 4 (2 điểm) [*HK2 21-22*]
Để việc tìm kiếm thông tin mặt hàng được nhanh chóng, người ta dùng một bảng băm theo phương pháp thăm dò, làm việc trên mã quản lý của mặt hàng. Mã quản lý này là một con số nguyên. Bảng băm có:

- Hàm băm: ***h(key) = (key % M)***.
- Hàm băm lại (hàm thăm dò): ***prob(key, i) = (h(key) + i*i + i) % M***.

Trong đó:

- key là giá trị khóa.
- i là một số nguyên cho biết lần băm lại (thăm dò) thứ i.
- M là kích thước bảng băm.

Giả sử M = 7, cho trường hợp T của bảng băm đã chứa dữ liệu như bên dưới. Biết "-" là ký hiệu vị trí trống trong bảng băm.

	Bảng băm T
0	-
1	-
2	16
3	-
4	-
5	12
6	13

a) Trình bày từng bước việc tìm mã quản lý 23 trong bảng băm T. (0.5 điểm).
b) Trình bày từng bước việc thêm các mã quản lý sau vào bảng băm T theo đúng thứ tự liệt kê là 11, 20, 27 (1.5 điểm).

Giải:

a.
Ta có:

- $h(23) = 23\ \%\ 7 = 2$, T[2] = 26, băm lại.
- $prob(23, 1) = (2 + 1^2)\ \%\ 7 = 3$, T[3] trống, không tìm thấy 23.

b.

Thêm 11:

- $h(11) = 11 \,\%\, 7 = 4$, T[4] trống, T[4] = 11.

	Bảng băm T
0	-
1	-
2	16
3	-
4	11
5	12
6	13

Thêm 20:

- $h(20) = 20 \,\%\, 7 = 6$, T[6] = 13, băm lại.
- $prob(20, 1) = (6 + 1^2) \,\%\, 7 = 0$, T[0] trống, T[0] = 20.

	Bảng băm T
0	20
1	-
2	16
3	-
4	11
5	12
6	13

Thêm 27:

- $h(27) = 27 \,\%\, 7 = 6$, T[6] = 13, băm lại.
- $prob(27, 1) = (6 + 1^2) \,\%\, 7 = 0$, T[0] = 20, băm lại.
- $prob(27, 2) = (6 + 2^2) \,\%\, 7 = 3$, T[3] trống, T[3] = 27.

	Bảng băm T
0	20
1	-
2	16
3	27
4	11
5	12
6	13

Câu 3 (2 điểm) [*HK2 18-19*]
Cho bảng băm A kích thước 13 phần tử và tập khóa K = {10, 26, 52, 76, 13, 8, 3, 33, 60, 42}, ta cần nạp các giá trị khóa K vào bảng băm A sử dụng hàm băm H(K) = K % 13. Hãy vẽ bảng băm khi thêm từng khóa K vào bảng A, trong trường hợp xảy ra đụng độ, sử dụng phương pháp dò tuyến tính để giải quyết đụng độ.

Giải:

Thêm 10:

- H(10) = 10 % 13 = 10, T[10] trống, T[10] = 10.

0	1	2	3	4	5	6	7	8	9	10	11	12
										10		

Thêm 26:

- H(26) = 26 % 13 = 0, T[0] trống, T[0] = 2.

0	1	2	3	4	5	6	7	8	9	10	11	12
26										10		

Thêm 52:

- H(52) = 52 % 13 = 0, T[0] = 26, băm lại.
- H’(52, 1) = (0 + 1) % 13 = 1, T[1] trống, T[1] = 52.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52									10		

Thêm 76:

- H(76) = 76 % 13 = 11, T[11] trống, T[11] = 76.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52									10	76	

Thêm 13:

- H(13) = 13 % 13 = 0, T[0] = 26, băm lại.
- H’(13, 1) = (0 + 1) % 13 = 1, T[1] = 52, băm lại.
- H’(13, 2) = (0 + 2) % 13 = 2, T[2] trống, T[2] = 13.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52	13								10	76	

Thêm 8:

- H(8) = 8 % 13 = 8, T[8] trống, T[8] = 8.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52	13						8		10	76	

Thêm 3:

- H(3) = 3 % 13 = 3, T[3] trống, T[3] = 3.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52	13	3					8		10	76	

Thêm 33:

- H(33) = 33 % 13 = 7, T[7] trống, T[7] = 33.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52	13	3				33	8		10	76	

Thêm 60:

- H(60) = 60 % 13 = 8, T[8] = 8, băm lại.
- H'(60, 1) = (8 + 1) % 13 = 9, T[9] trống, T[9] = 60.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52	13	3				33	8	60	10	76	

Thêm 42:

- H(42) = 42 % 13 = 3, T[3] = 3, băm lại.
- H'(42, 1) = (3 + 1) % 13 = 4, T[4] trống, T[4] = 42.

0	1	2	3	4	5	6	7	8	9	10	11	12
26	52	13	3	42			33	8	60	10	76	

Câu 4 (2 điểm) [*HK2 17-18*]

Cho tập khóa K = {12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5} và hàm băm:

H(key) = (2 * key + 5) %11 (key: khóa cần băm).

Anh / chị hãy thực hiện :

a. Vẽ hình từng bước việc lưu trữ từng khóa trong K vào bảng băm có kích thước M = 11, dùng hàm băm H và phương pháp nối kết trực tiếp để xử lý xung đột. (1 điểm)

b. Định nghĩa cấu trúc dữ liệu cho bảng băm ở câu a (1 điểm)

Giải:

a.

Vị trí của các khóa cần thêm là:

- 12: H(12) = (2 * 12 + 5) % 11 = 7.
- 44: H(44) = (2 * 44 + 5) % 11 = 5.
- 13: H(13) = (2 * 13 + 5) % 11 = 9.
- 88: H(88) = (2 * 88 + 5) % 11 = 5.
- 23: H(23) = (2 * 23 + 5) % 11 = 7.
- 94: H(94) = (2 * 94 + 5) % 11 = 6.
- 11: H(11) = (2 * 11 + 5) % 11 = 5.
- 39: H(39) = (2 * 39 + 5) % 11 = 6.
- 20: H(20) = (2 * 20 + 5) % 11 = 1.
- 16: H(16) = (2 * 16 + 5) % 11 = 4.
- 5: H(5) = (2 * 5 + 5) % 11 = 4.

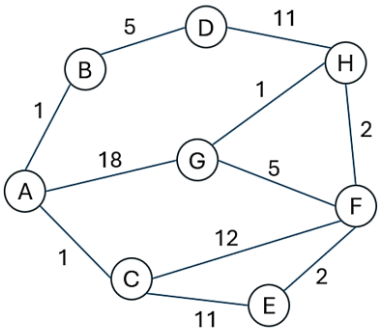
Index	Node
0	
1	20
2	
3	
4	16, 5
5	44, 88, 11
6	94, 39
7	12, 23
8	
9	13
10	
11	

b.

```
vector <vector <int>> table (M);
```

6. Đồ thị.

Câu 10 (0.5 điểm) [*HK1 23-24*]
Cho đồ thị vô hướng G:



Hỏi độ dài đường đi ngắn nhất từ A đến các đỉnh còn lại?

Giải:

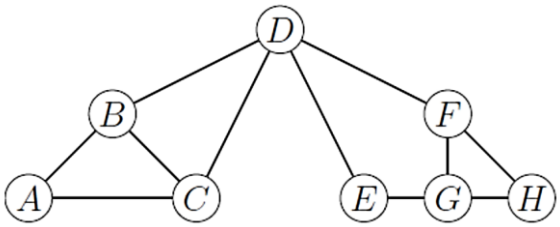
A	B	C	D	E	F	G	H
--	1	1	6	12	13	16	15

Câu 6 (1.25 điểm) [*HK1 24-25 1*]

Bạn đang chơi trò chơi MazeCraft, có giao diện như Hình 1. Nhận thấy rằng, bạn có thể chuyển đổi mê cung thành một đồ thị, trong đó các đỉnh đại diện cho các ô và các cạnh đại diện cho các lối đi, bạn muốn sử dụng các thuật toán tìm kiếm trên đồ thị vừa học để tìm đường đi qua mê cung.



Hãy xem xét đồ thị G1 (Hình 2) đã được chuyển đổi dưới đây.



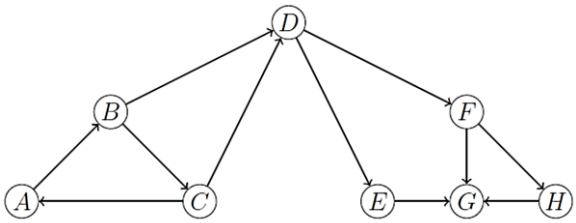
Giả sử rằng, đồ thị được biểu diễn bằng danh sách kề (adjacency list) và tất cả các danh sách kề được sắp xếp, nghĩa là ứng với mỗi đỉnh i, ta cần lưu trữ một danh sách có thứ tự gồm các đỉnh kề với đỉnh i. Các đỉnh kề này được sắp xếp theo thứ tự bảng chữ cái.
Thứ tự bảng chữ cái là A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z.

Hãy thực hiện các yêu cầu sau:

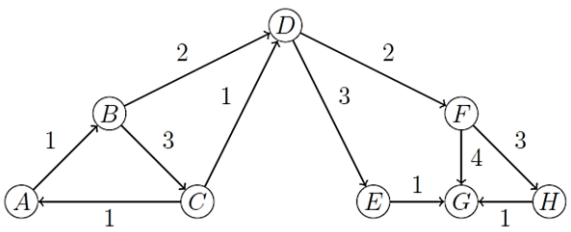
a) Giả sử bạn muốn tìm một đường đi từ đỉnh A đến đỉnh H. Nếu bạn sử dụng thuật toán tìm kiếm theo chiều rộng (Breadth-First Search, BFS), hãy viết lại đường đi kết quả dưới dạng một chuỗi các đỉnh.

b) Nếu bạn sử dụng thuật toán tìm kiếm theo chiều sâu (Depth-First Search, DFS) để tìm một đường đi từ đỉnh A đến đỉnh H, hãy viết lại đường đi kết quả dưới dạng một chuỗi các đỉnh, kèm theo chú thích là bạn chọn cài đặt theo cách thức nào.
Ví dụ, có 2 cách cài đặt DFS phổ biến là sử dụng Ngăn xếp để lưu các đỉnh chờ duyệt (Stack-based implementation) hoặc dùng kỹ thuật Đệ quy quay lui (Recursion-based implementation, Backtracking).

c) Bây giờ giả sử rằng, các lối đi trong mê cung có hướng. Chạy lại thuật toán tìm kiếm theo chiều sâu (DFS) trên đồ thị có hướng G2 (Hình 3) bên dưới.



d) Giả sử mỗi lối đi trong mê cung gây ra một lượng sát thương khác nhau cho bạn trong trò chơi. Đồ thị được bổ sung thêm các trọng số để biểu diễn sát thương gây ra bởi mỗi cạnh. Sử dụng thuật toán Dijkstra để tìm đường đi từ đỉnh A đến đỉnh H trong đồ thị G3 (Hình 4) với sát thương thấp nhất có thể. Hãy ghi lại thứ tự các đỉnh được loại bỏ khỏi hàng đợi ưu tiên (priority queue) khi chạy thuật toán Dijkstra.



Giải:

a.

Đỉnh đã xét	Queue
	A
A	B C
B	C D
C	D
D	E F
E	F G
F	G H
G	H
H	

b.

Đỉnh đã xét	Stack
	A
A	B C
C	B D
D	B E F
F	B E G H
H	

c.

Đỉnh đã xét	Stack
	A
A	B
B	C D
D	C E F
F	C E G H
H	

d.

Đỉnh đã xét	Priority queue
	(A, 0)
(A, 0)	(B, 1)
(B, 1)	(D, 2) (C, 4)
(D, 2)	(C, 4) (F, 5) (E, 6)
(C, 4)	(F, 5) (E, 6)
(F, 5)	(E, 6) (H, 8) (G, 9)
(E, 6)	(G, 7) (H, 8)
(G, 7)	(H, 8)
(H, 8)	

Câu 11 (2 điểm) [*HK1 23-24*]
Cho các bản đồ thể hiện không quá 10000 giao lộ (ngã ba, ngã tư, ..). Mỗi giao lộ được đánh số từ 0 đến 9999. Các bản đồ này có thể biểu diễn theo đồ thị đơn, vô hướng và được biểu diễn trên máy tính bằng ma trận, với kiểu dữ liệu Bando như sau:

```
struct Bando {
    int N;
    bool Duong[10000][10000];
};
```

Trong đó, N là số giao lộ; `Duong[i][j] = true` cho biết giao lộ `i` có đường đi trực tiếp đến giao lộ `j`; `Duong[i][j] = false` cho biết giao lộ `i` không có đường đi trực tiếp đến giao lộ `j`.

1. Cài đặt hàm **`DemNgaNam`** để đếm số ngã năm trên bản đồ `m` (1đ).
2. Cài đặt hàm **`CoDuongdi`** trả về 1 nếu bản đồ `m` có đường đi từ giao lộ `s` đến giao lộ `e`, trả về `0` trong các trường hợp còn lại. Yêu cầu phải dùng queue thay cho hàm đệ quy (1đ).

Giải:

1.

```
int DemNgaNam (BanDo m) {
    int ans = 0;

    for (int i = 0; i < m.N; i++) {
        int count = 0;
        for (int j = 0; j < m.N; j++)
            count += m.Duong[i][j];
        ans += (count == 5);
    }

    return ans;
}
```

2.

```
int CoDuongdi (BanDo m, int s, int e) {

    vector <bool> close (m.N, false);

    queue <int> qu;
    qu.push(s);
    close[s] = true;

    while (!qu.empty()) {
        int i = qu.front(); qu.pop(); close[i] = true;
        if (i == e)
            return 1;
        for (int j = 0; j < m.N; j++)
            if (!close[j] && m.Duong[i][j])
                qu.push(j);
    }

    return 0;
}
```

III. MÔ HÌNH HÓA BÀI TOÁN VÀ CÀI ĐẶT THUẬT TOÁN.

Câu 7 (1.5 điểm) [*HK1 24-25 1*]
Hãy khai báo kiểu dữ liệu danh sách liên kết đơn mà mỗi phần tử chứa thông tin về một quốc gia gồm tên quốc gia, dân số, diện tích. Với các kiểu dữ liệu vừa khai báo, hãy
xây dựng các hàm sau:

a) Hàm nhập danh sách các quốc gia bằng cách thêm từng quốc gia vào đầu danh sách.
b) Hàm sắp xếp danh sách các quốc gia theo tên quốc gia dùng thuật toán Selection Sort.

Giải:

```
struct quocGia {
    string ten;
    int danSo;
    float dienTich;
    quocGia* pNext;
};


struct node {
    quocGia info;
    node* pNext;
};
```

a.

```
node* createNode(string ten, int danSo, float dienTich) {
    node* tam = new node;
    node->info.ten = ten;
    node->info.danSo = danSo;
    node->info.dienTich = dienTich;
    tam->pNext = nullptr;
    return tam;
}


void Nhap(node*& list) {
    list = nullptr;

    int n;
    cin >> n;

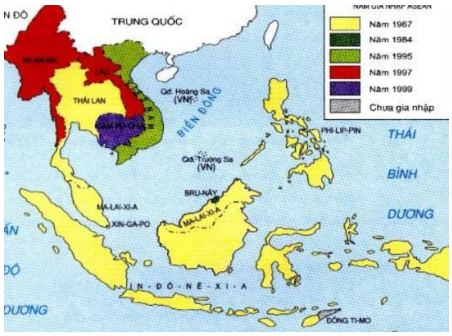
    for (int i = 0; i < n; i++) {
        string ten;
        int danSo;
        float dienTich;
        cin >> ten >> danSo >> dienTich;

        if (!list)
            list = createNode(ten, danSo, dienTich);
        else {
            node* tam = createNode(ten, danSo, dienTich);
            tam->pNext = list;
            list = tam;
        }
    }
}
```

b.

```
void setlectionSort(node*& list) {
    node* i = list;
    while (i) {
        node* min = i;
        node* j = i->pNext;
        while (j) {
            if (j->info.ten < min->info.ten)
                min = j;
            j = j->pNext;
        }
        swap(min->info, i->info);
        i = i->pNext;
    }
}
```

Câu 7 (1.5 điểm) [*HK1 24-25 1*]
Cho bài toán "Tô màu bản đồ" được đặt ra như sau: Có một bản đồ các quốc gia trên thế giới, ta muốn tô màu các quốc gia này sao cho hai nước có cùng ranh giới được tô khác màu nhau. Yêu cầu tìm cách tô sao cho số màu sử dụng là ít nhất.



Hãy thực hiện các yêu cầu sau:

a) Xác định thông tin đầu vào (input), thông tin đầu ra (output) của bài toán. (0.5đ)
b) Hãy mô hình hóa bài toán trên thành một bài toán trên đồ thị, tức cho biết đỉnh và cạnh của đồ thị biểu diễn thông tin gì, đồ thị có trọng số hay không, yêu cầu xử lý gì với đồ thị đó. (0.5đ)
c) Xây dựng các cấu trúc dữ liệu phù hợp nhất có thể để biểu diễn đồ thị trên máy tính theo input đã cho ở Câu a. (0.5đ)
   Cấu trúc được xem là tốt nếu đạt được các tiêu chuẩn sau: Tiết kiệm tài nguyên; Hỗ trợ một số thao tác cơ bản như "Kiểm tra hai đỉnh có kề nhau không", "Tìm danh sách các đỉnh kề với một đỉnh cho trước" với ràng buộc là không phải duyệt qua danh sách tất cả các cạnh của đồ thị.
d) Giả sử chương trình đã có sẵn các hàm hỗ trợ nhập thông tin đầu vào và xuất thông tin đầu ra theo cấu trúc dữ liệu ở Câu c, cùng với các chỉ thị #include cần thiết. Hãy cài đặt một (hoặc một số) hàm cho biết tên của quốc gia có đường biên giới chung với nhiều quốc gia khác nhất trong bản đồ.

Sinh viên được phép sử dụng Standard Template Library-STL với những cấu trúc dữ liệu (vector, stack, queue, list, map, set, pair, ...) cũng như giải thuật được xây dựng sẵn.

Giải:

a.
Input:

1. Số lượng các quốc gia và số lượng biên giới giữa các quốc gia.
2. Thông tin các quốc gia: Tên quốc gia, biên giới (quốc gia nào giáp với quốc gia nào).

Output:

- Danh sách màu sắc cụ thể cho từng quốc gia sao cho số lượng màu dùng là ít nhất.

b.

- Mỗi đỉnh ứng với một quốc gia.
- Nếu quốc gia A và B có biên giới chung thì A có cạnh nối đến B.
- Đồ thị vô hướng.
- Không có trọng số.
- Bài toán trở thành: Tìm cách tô màu cho các đỉnh sao cho 2 đỉnh kề nhau có màu khác nhau, số màu sử dụng là ít nhất.

c.
Sử dụng cấu trúc **`map <string, set<string>> graph`** làm danh sách kề. Trong đó:

- Thành phần thứ nhất của map là tên 1 đỉnh.
- Thành phần thứ hai của map là danh sách các đỉnh kề với thành phần thứ nhất.

d.

```
string maxAdj(map <string, set <string>> graph) {
    string ans;
    int max = -1;
    for (auto v : graph)
        if (v.second.size() > max)
            ans = v.first;
    return ans;
}
```

Câu 4 (1.5 điểm) [*HK2 18-19*]
Mạng xã hội là là dịch vụ kết nối các thành viên, người dùng trên Internet lại với nhau dựa theo những tiêu chí, sở thích nào đó, với nhiều mục đích khác nhau, là nơi trao đổi thông tin, chia sẻ suy nghĩ, ý tưởng mà không bị giới hạn về không gian và thời gian. Các thành viên giao tiếp với các thành viên khác trong mạng, mỗi thành viên sẽ là một chủ thể trao đổi thông tin và tương tác với người khác.

Bạn được giao nhiệm vụ xây dựng một mạng xã hội thu nhỏ với các chức năng : kết bạn với nhau, xem thông tin của bạn của mình post trên mạng và like một post của bạn giống như cách trên Facebook, hãy đề xuất ý tưởng :

a. Mô tả cấu trúc dữ liệu bạn nghĩ sẽ sử dụng phù hợp với các chức năng của mạng đã mô tả ở phần trên. (1 điểm)
b. Mô tả các giải thuật (các bước thực hiện giải thuật, không cài đặt) để hiện thực các chức năng của mạng đã mô tả ở phần trên và phù hợp với cấu trúc dữ liệu bạn đã đề nghị trong câu 4.a. (0.5 điểm)

Giải:

a.
Thông tin của người dùng phục vụ cho các tính năng:

- Kết bạn → Danh sách bạn bè.
- Đăng post → Danh sách post.
- Thả like post → Danh sách những ai đã thả like.

```
struct post{
    string content;
    set <user> likes;
};

struct user{
    string id, password;
    set <post> posts;
    set <user> friends;
};
```

Các người dùng có thể được coi như một đỉnh của đồ thị vô hướng không trọng số. Trong struct user có trường friends chính là danh sách kề lưu trữ các bạn của người dùng.

Danh sách các user:

```
vector <user> users;
```

b.

Thuật toán kết bạn / hủy kết bạn:

- *Input*: Người dùng A, B. *Output*: Thông báo đã kết bạn / hủy kết bạn thành công.
- Khi người dùng A kết bạn với người dùng B thì push người dùng B vào friends của A và ngược lại.
- Nếu B hủy kết bạn với A thì tìm kiếm và xóa B ra khỏi friends của A và ngược lại.

Thuật toán upload / delete post:

- *Input*: Người dùng A. *Output*: Thông báo đã upload / delete post thành công.
- Khi người dùng A upload một post mới thì đưa post đó vào trường posts.
- Ngược lại thì tìm kiếm và xóa post khỏi posts.

Thuật toán thả like / unlike:

- *Input*: Người dùng A, B. *Output*: Thông báo đã like / unlike post thành công.
- Khi người dùng B thả like vào 1 post của người dùng A thì đưa người dùng B vào trường likes của A. Khi unlike thì làm ngược lại.

Do cả friends, posts, likes đều dùng set nên không cần kiểm tra việc 1 phần tử có nằm sẵn trong danh sách hay không. Nếu dùng các cấu trúc khác như mảng, vector,… thì sẽ cần.

Câu 5 (1 điểm) [*HK2 21-22*]
Trong các ứng dụng thực tế, chẳng hạn trong mạng lưới giao thông đường bộ, đường thủy hoặc đường hàng không, người ta không chỉ quan tâm đến việc tìm đường đi giữa hai địa điểm mà còn phải lựa chọn một hành trình tiết kiệm nhất (theo tiêu chuẩn không gian, thời gian hay chi phí). Vấn đề này có thể được mô hình hóa thành một bài toán trên đồ thị, trong đó mỗi địa điểm được biểu diễn bởi một đỉnh, cạnh nối hai đỉnh biểu diễn cho "đường đi trực tiếp" giữa hai địa điểm (tức không đi qua địa điểm trung gian) và trọng số của cạnh là khoảng cách giữa hai địa điểm.

Bài toán có thể phát biểu dưới dạng tổng quát như sau: Cho một đơn đồ thị có hướng và có trọng số dương G = (V, E), trong đó V là tập đỉnh, E là tập cạnh (cung) và các cạnh đều có trọng số, hãy tìm một đường đi (không có đỉnh lặp lại) ngắn nhất từ đỉnh xuất phát S thuộc V đến đỉnh đích F thuộc V.

Giả sử thông tin đầu vào của bài toán (Input) được nhập vào chương trình như sau:

Input	Giải thích
7 A B 1 B E 3 E D 3 C B 4 A D 7 E C 2 C D 1 A E	• Dòng đầu tiên chứa một số nguyên dương e cho biết số cạnh của đồ thị. • Với e dòng tiếp theo, mỗi dòng chứa hai chuỗi u, i và một số nguyên dương x, thể hiện thông tin có một cạnh nối từ đỉnh u sang đỉnh i trong đồ thị với độ dài (trọng số) là x. • Dòng cuối cùng chứa hai chuỗi s và f, đây là đỉnh bắt đầu và đỉnh kết thúc của đường đi cần tìm ***Lưu ý***: Không biết trước số đỉnh và danh sách các đỉnh.

Hãy thực hiện các yêu cầu sau:

a) Xây dựng các cấu trúc dữ liệu phù hợp nhất có thể để biểu diễn đồ thị trên máy tính theo input đã cho. (0.5 điểm)
Cấu trúc được xem là tốt nếu đạt được các tiêu chuẩn sau: Tiết kiệm tài nguyên; Hỗ trợ một số thao tác cơ bản như "Kiểm tra hai đỉnh có kề nhau không", "Tìm danh sách các đỉnh kề với một đỉnh cho trước" với ràng buộc là không phải duyệt qua danh sách tất cả các cạnh của đồ thị.
b) Viết hàm nhập theo Input ở đầu bài và lưu trữ thông tin của đồ thị vào cấu trúc dữ liệu đã đề xuất ở câu a. (0.5 điểm)

KHÔNG YÊU CẦU tìm cách giải cho bài toán này. Sinh viên ĐƯỢC PHÉP sử dụng Standard Template Library-STL với những cấu trúc dữ liệu (vector, stack, queue, list, map, set, pair, ...) cũng như giải thuật được xây dựng sẵn.

Giải:

a.
Sử dụng cấu trúc **`map <string, set<pair<int, string>>>`** để làm danh sách kề.

b.

```
void Input(map <string, set <pair <int, string> > >& graph, int& e) {
    cin >> e;

    for (int i = 0; i < e; i++) {
        string u, v;
        int x;
        cin >> u >> v >> x;
        graph[u].insert({x, v});
    }
}
```

Câu 6 (2 điểm) [*HK2 22-23*]
Hãy thực hiện các yêu cầu sau:

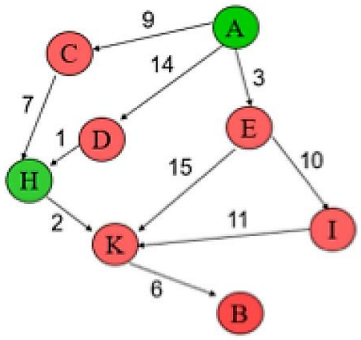
1. Xây dựng các CTDL phù hợp nhất có thể để biểu diễn đồ thị trên máy tính theo input đã cho (0.5 điểm).
   Cấu trúc được xem là tốt nếu đạt được các tiêu chuẩn sau: Tiết kiệm tài nguyên; Hỗ trợ một số thao tác cơ bản như:"Kiểm tra hai đỉnh có kề nhau không","Tìm danh sách các đỉnh kề với một đỉnh cho trước" với ràng buộc là không phải duyệt qua danh sách tất cả các cạnh của đồ thị.
2. Dưới đây là một ví dụ về mã giả của thuật toán Dijkstra để tìm một đường đi ngắn nhất từ đỉnh xuất phát s đến đỉnh đích g trong đồ thị. Hãy minh hoạ các bước thực hiện của thuật toán theo mã giả đã cho để "Tìm đường đi ngắn nhất từ đỉnh A đến đỉnh H" trong đồ thị bên dưới (0.75 điểm).

Gọi:

- Open: tập các đỉnh có thể xem xét lựa chọn cho bước đi tiếp theo, các đỉnh có thể được xem xét lại, đỉnh chờ duyệt.
- Close: tập các đỉnh đã xét/đã duyệt, không xem xét lại
- s : đỉnh xuất phát
- g: đỉnh đích
- p: đỉnh đang xét, đỉnh hiện hành
- Hàm d(u) dùng để lưu trữ độ dài đường đi ngắn nhất từ đỉnh nguồn s đến đỉnh u
- w(u,i) : trọng số của cạnh nối từ đỉnh u tới đỉnh i
- parent(q)=p: lưu thông tin cha con, đỉnh p là cha của đỉnh q

Các bước thực hiện chính:

- Bước 1: Khởi tạo
  - Open = {s}
  - Close ={}
  - d(s)=0
- Bước 2: While (Open ≠ {})
  - Chọn p thuộc Open có d(p) nhỏ nhất (xóa p ra khỏi Open). Nếu có nhiều đỉnh cùng giá trị d(p) nhỏ nhất thì chọn đỉnh nhập vào sau trong danh sách đỉnh.
  - Nếu p là đỉnh đích thì xuất đường đi, kết thúc thuật toán.
  - Nếu p đã duyệt rồi thì bỏ qua, không xem xét lại, trở lại đầu vòng lặp.
  - Đánh dấu p đã duyệt qua rồi (tức thêm p vào Close).
  - Với mỗi đỉnh q kề với p, nếu q không thuộc Close:
    - Nếu q đã có trong Open và d(q)> d(p)+w(p,q) thì cập nhật các thông tin:
      d(q) = d(p) + w(p,q) parent(q)=p
    - Nếu q chưa có trong Open:
      - d(q) = d(p) +w(p,q).
      - parent(q)=p.
      - Thêm q vào Open.
- Bước 3: Kết luận "Không tìm thấy đường đi"



3. Hãy thiết kế CTDL cho các đối tượng được đề cập trong thuật toán 6.2 (như Open, Close, hàm d(u), parent) và viết hoàn chỉnh hàm Dijkstra để giải quyết bài toán nêu trên (0.75 điểm).

Giải:

1.
Sử dụng danh sách kề có trọng số, triển khai bằng `map <int, set<pair<int, int>>> graph`.

2.

p	Open	Close
	(A, 0)	
(A, 0)	(C, 9) (D, 14) (E, 3)	(A, 0)
(E, 3)	(C, 9) (D, 14) (K, 18) (I, 13)	(E, 3)
(C, 9)	(D, 14) (K, 18) (I, 13) (H, 16)	(C, 9)
(I, 13)	(D, 14) (K, 18) (H, 16)	(I, 13)
(D, 14)	(K, 17) (H, 15)	(D, 14)
(H, 15)	(K, 17)	(H, 15)

Truy xuất tìm đường đi:

- Cha(H) = D.
- Cha(D) = A.

Vậy đường đi ngắn nhất từ A đến H là ADH.

3.

Hàm tìm đường đi:

```
int Dijkstra(int s, int e,
             const map<int, set<pair<int, int>>>& graph,
             map<int, int>& parent)
{
    map<int,int> d;
    map<int,bool> closed;

    // Khởi tạo
    for (auto& [u, adj] : graph) {
        d[u]      = INT_MAX;
        closed[u] = false;
        parent[u] = -1;
    }

    // Min-heap
    priority_queue<
       pair<int,int>, vector<pair<int, int>>, greater<>> open;

    d[s] = 0;
    open.push({0, s});

    while (!open.empty()) {
        auto [du, u] = open.top();
        open.pop();
        if (closed[u] || du > d[u])
            continue;
        closed[u] = true;
        if (u == e)
            break;

        // Xét các cạnh kề
        for (auto& vw : graph.at(u)) {
            int v = vw.first, w = vw.second;
            if (!closed[v] && d[u] + w < d[v]) {
                d[v] = d[u] + w;
                parent[v] = u;
                open.push({d[v], v});
            }
        }
    }

    // Bạn có thể lưu parent, d ra biến ngoài hoặc return d[e]
    return d[e];
}
```

Hàm truy xuất:

```
vector<int> findPath(int s, int e, const map <int, int>& parent) {
    vector<int> path;

    if (s == e) {
        path.push_back(s);
        return path; // Chỉ có 1 đỉnh trên đường đi
    }

    if (!parent.count(e) || parent.at(e)==-1)
        return path;  // Không tồn tại đường
```

```
    // Truy xuất
    for (int u = e; u != -1; u = parent.at(u)) {
        path.push_back(u);
    }

    reverse(path.begin(), path.end());
    return path;
}
```